



Università
degli Studi
di Palermo



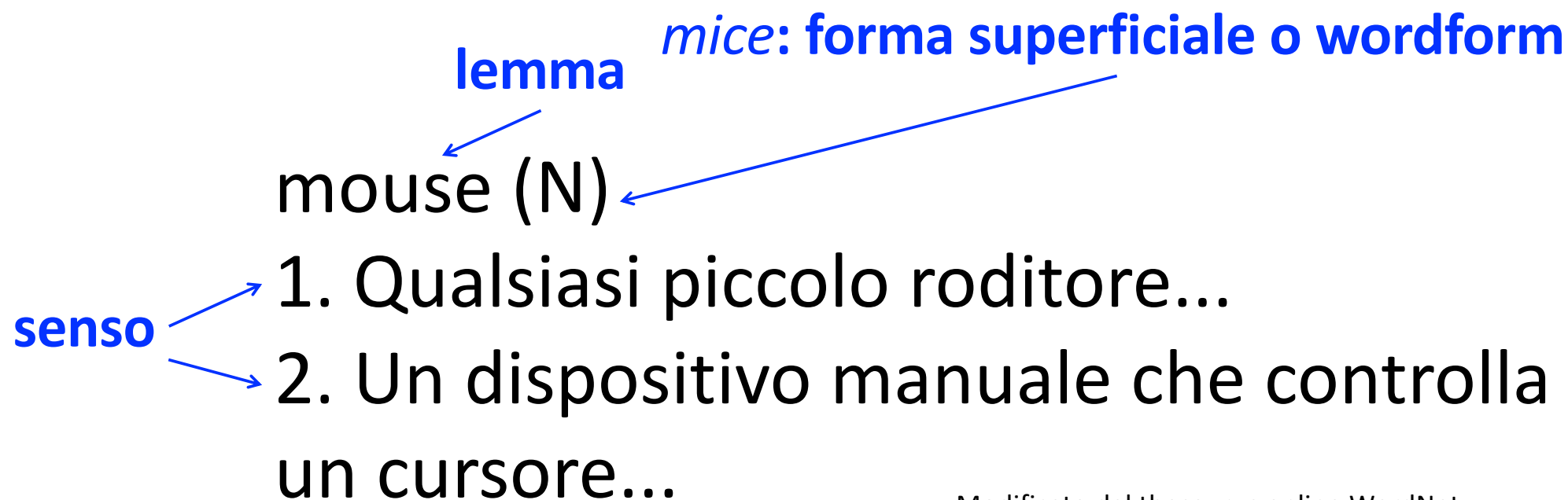
Natural Language Processing e Large Language Model

INTELLIGENZA ARTIFICIALE PER L'INGEGNERIA BIOMEDICA (EX ROBOTICA MEDICA)
a.a. 2025/2026

Prof. Roberto Pirrone



Lemmi e sensi



Modificato dal thesaurus online WordNet

Un **senso** (o “concetto”) è la **componente di significato** di una parola. I lemmi possono essere **polisemici**, cioè avere più sensi.

Vector semantics: idea 1

- Wittgenstein, PI #43:
"Il significato di una parola è il suo uso nel linguaggio"

Vector semantics: idea 1

- Definiamo “uso”:
le parole sono definite dal loro “ambiente” (le parole attorno a loro)
- Zellig Harris (1954):
«Se **A** e **B** hanno ambienti quasi identici, diciamo che sono **sinonimi.**»

Definire il significato come un punto nello spazio in base alla distribuzione linguistica.

- Ogni parola → un vettore
- Parole simili sono ***vicine nello spazio semantico***
- Costruiamo questo spazio ***in modo automatico***, osservando ***quali parole compaiono vicine nel testo.***



Definiamo il significato di una parola come un vettore

- Chiamato **embedding** perché è *inserito (embedded)* all'interno di uno spazio
- È il **metodo standard per rappresentare il significato** in NLP.

👉 Ogni moderno algoritmo di NLP utilizza gli. **embeddings** come rappresentazione del significato delle parole.

Utile per analizzare le **similarità** tra parole.

Due (e forse tre) tipi di embeddings

- **tf-idf ed altri modelli basati sul conteggio delle parole**
 - Tipico dell'Information Retrieval
 - Baseline comune per molti approcci
 - Vettori *sparsi*
 - Le parole sono rappresentate da **(una semplice funzione dei)** conteggi delle **parole vicine** nel testo.
- **Word2vec ed altri modelli di embedding neurale**
 - Vettori *densi*
 - La rappresentazione viene creata addestrando un classificatore a ***prevedere se una parola è probabile che compaia nelle vicinanze.***
 - Esiste un'estensione chiamata **contextual embeddings**

Language Model neurali

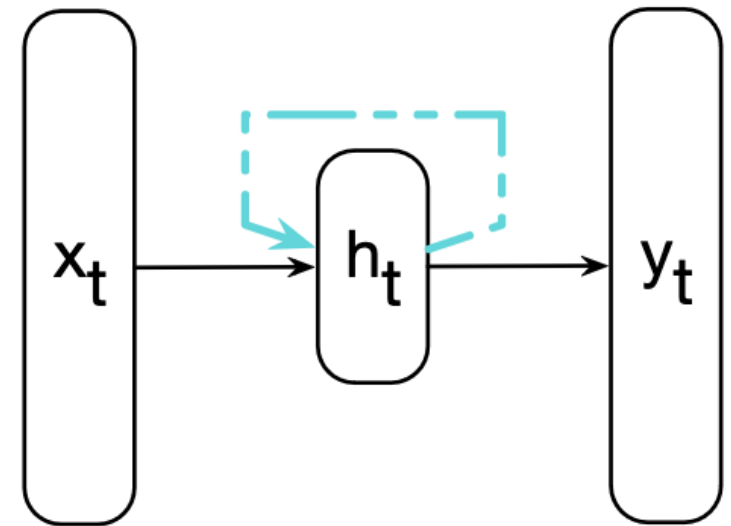
- Language Modeling: calcolare la probabilità di una parola in una sequenza, posto di averne osservate alcune.
- Abbiamo visto gli N-grammi
- I LM neurali li battono di gran lunga
- I LM neurali allo stato dell'arte sono basati su reti neurali molto potenti quali i Transformer
- Anche semplici modelli feedforward hanno una buona performance

La natura del linguaggio

- I **neural language models** usano o sliding windows di dimensione fissa o il padding dell'input fino alla frase più lunga
 - Vedremo di nuovo il padding nei Transformers
- Le **Recurrent Neural Networks** (Reti neurali Ricorrenti – RNN) trattano direttamente la natura sequenziale del linguaggio
 - Le connessioni ricorrenti permettono alla decisione del modello di dipendere da informazioni provenienti da centinaia di parole nel passato

Elman Network (1990)

- Lo stato della rete è rappresentato al tempo t
- L'input dell'**hidden layer** è aumentato con *l'output al punto temporale precedente h_{t-1}*

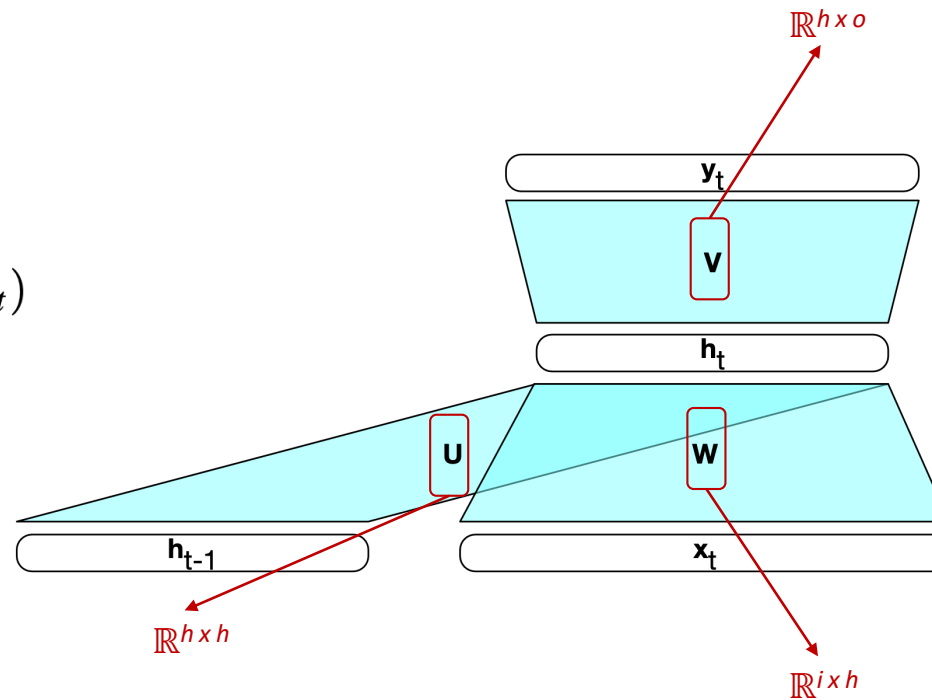


Inferenza nelle RNN

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$$

$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t)$$

Usiamo la softmax per la classificazione



Inferenza nelle RNN

La computazione
itera attraverso la
sequenza di input

function FORWARDRNN($\mathbf{x}$, *network*) **returns** output sequence $\mathbf{y}$

$\mathbf{h}_0 \leftarrow 0$

for $i \leftarrow 1$ **to** LENGTH($\mathbf{x}$) **do**

$\mathbf{h}_i \leftarrow g(\mathbf{U}\mathbf{h}_{i-1} + \mathbf{W}\mathbf{x}_i)$

$\mathbf{y}_i \leftarrow f(\mathbf{V}\mathbf{h}_i)$

return $\mathbf{y}$



Università
degli Studi
di Palermo

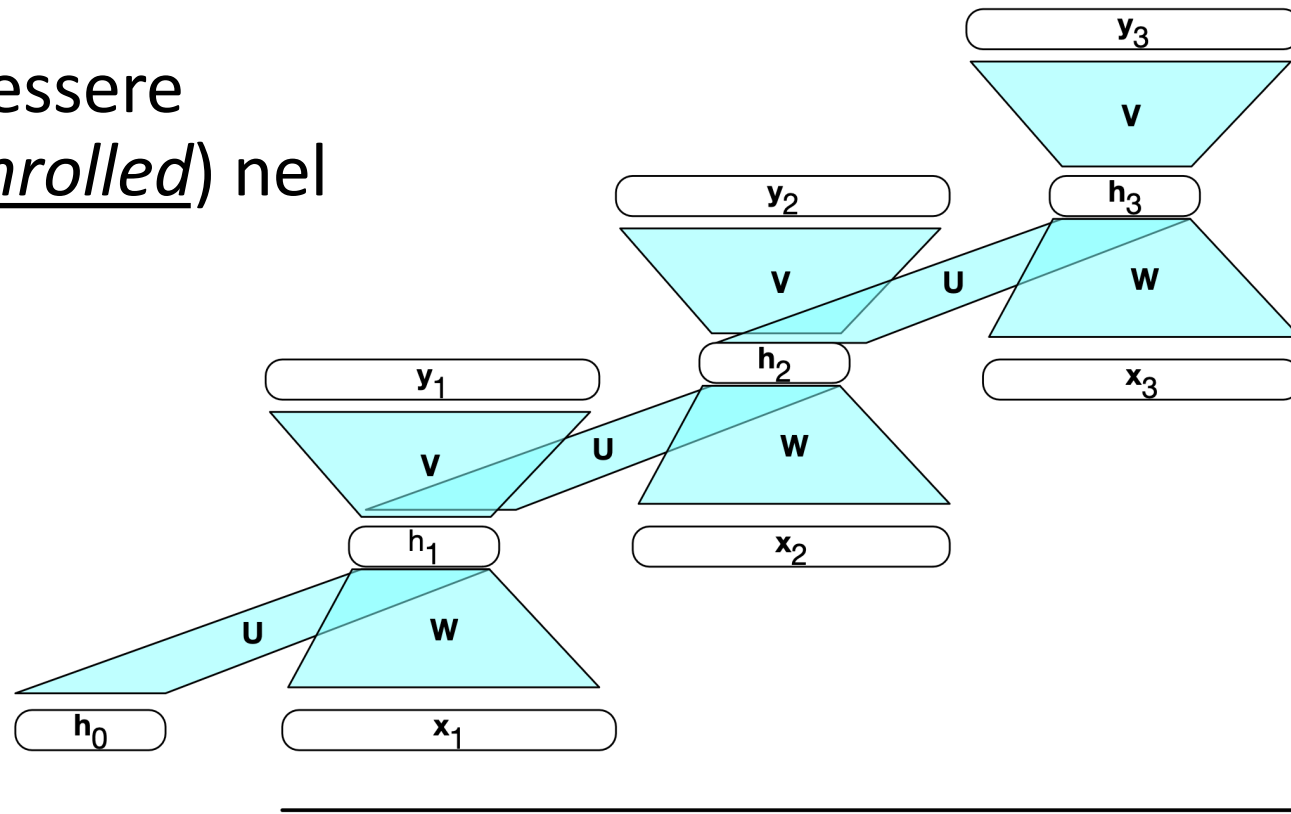


dipartimento
di ingegneria
unipa



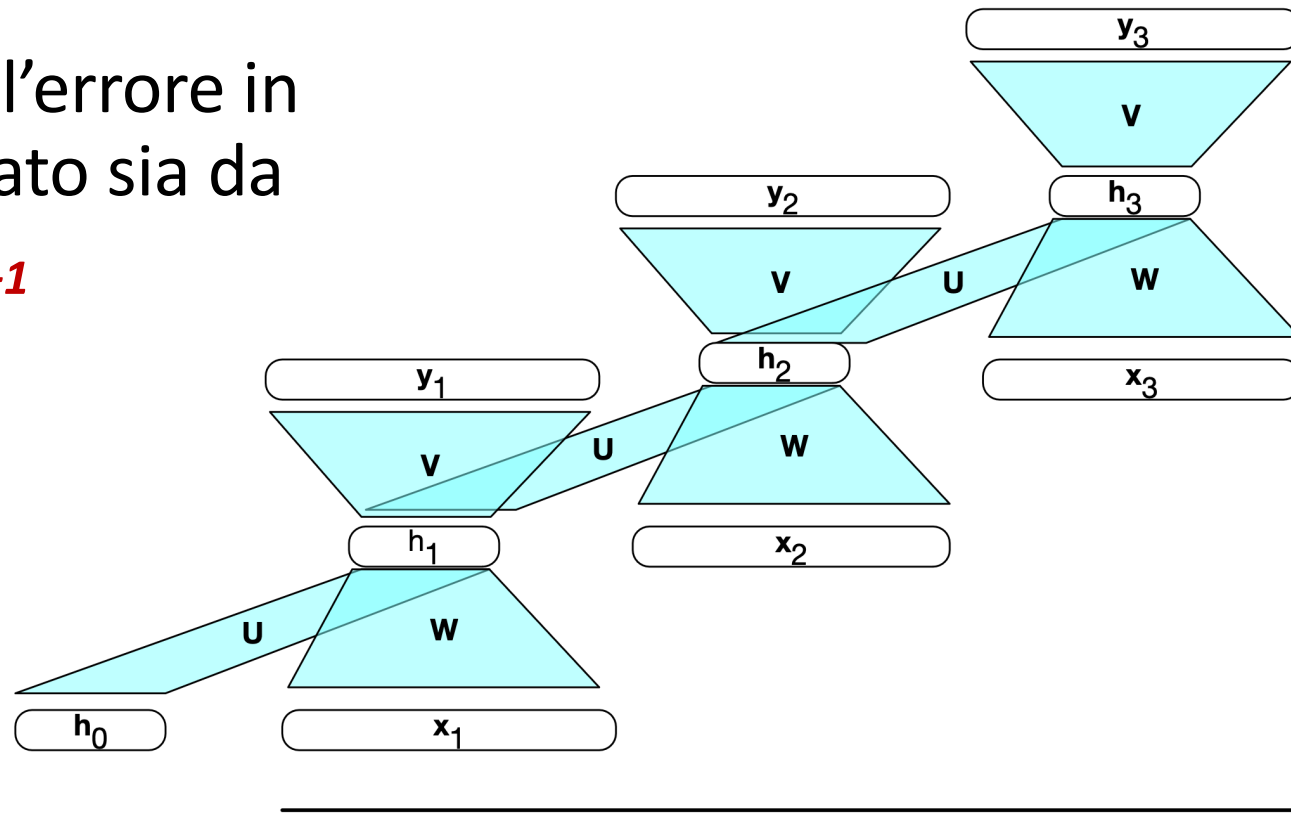
Inferenza nelle RNN

La rete può essere
srotolata (unrolled) nel
tempo



Training delle RNN

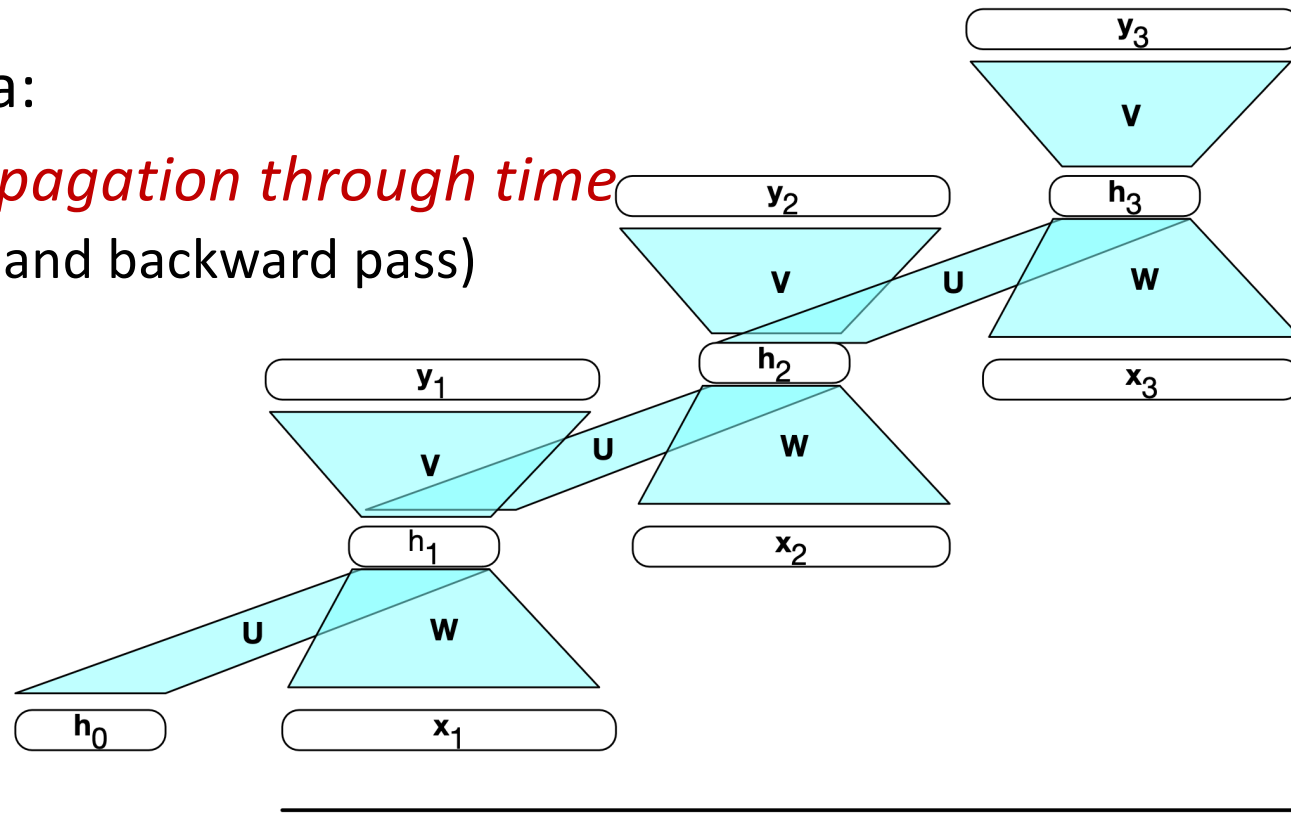
Il calcolo dell'errore in h_i è influenzato sia da h_{i-1} sia da h_{i+1}



Training delle RNN

Vecchia scuola:

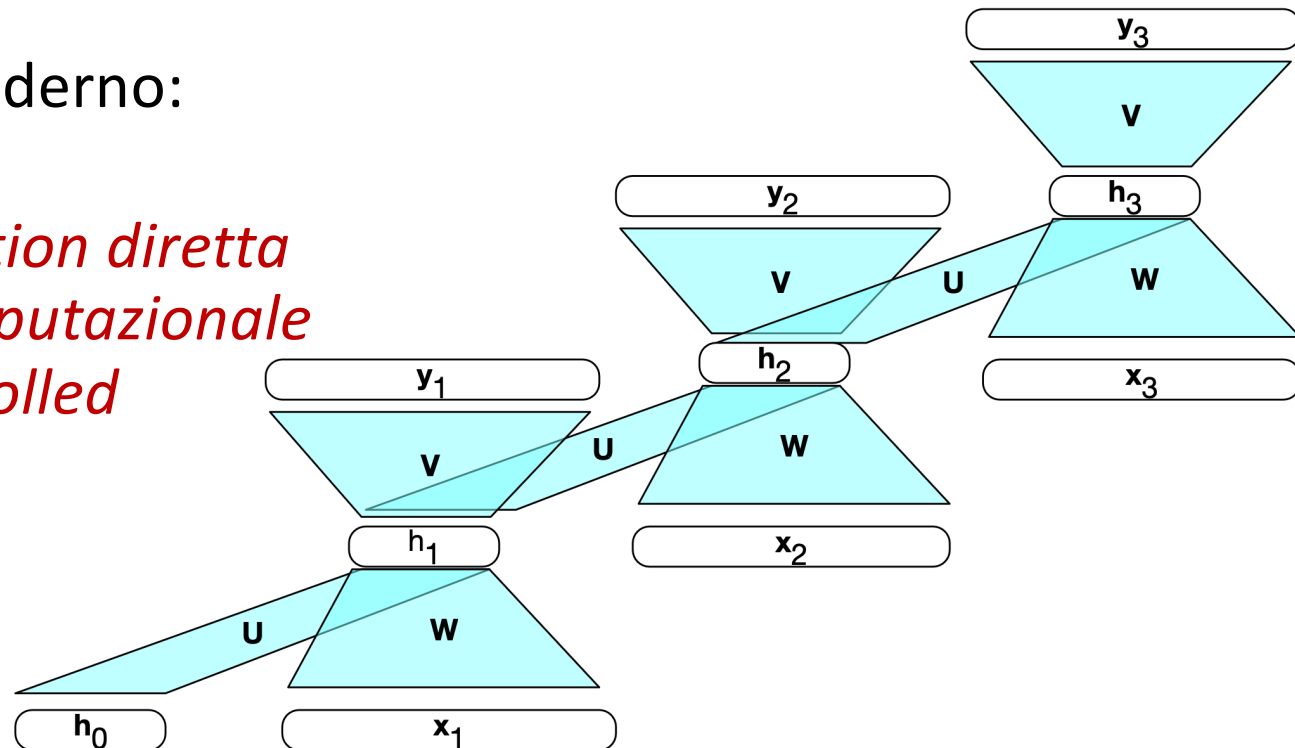
Backpropagation through time
(Forward and backward pass)



Training delle RNN

Approccio moderno:

*Backpropagation diretta
sul grafo computazionale
della rete unrolled*



RNN come Language Model

Richiamo della definizione di language modeling

- Language modeling:

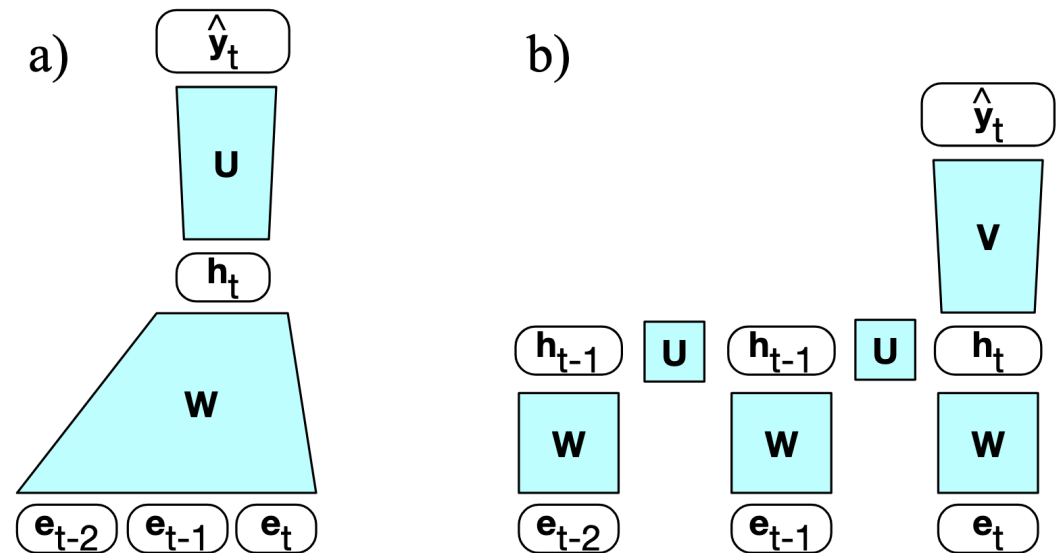
- Predire quanto è probabile una parola data una sequenza di parole osservata $P(\textit{fish}|\textit{Thanks for all the})$

- Possiamo anche predire la probabilità di un'intera sequenza

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{<i})$$

RNN e contesto

- Le RNN non hanno problemi nel trattare un contesto illimitato
- L'hidden layer tiene conto del contesto passato



Embeddings

- Usiamo anche la matrice di **word embeddings E**

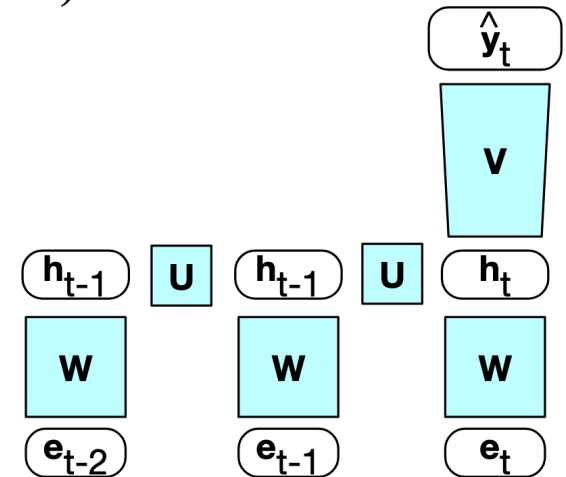
$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$$

In generale E non è addestrata e l'embedding layer è congelato (frozen), ad esempio a una rappresentazione word2vec

b)



Inferenza forward

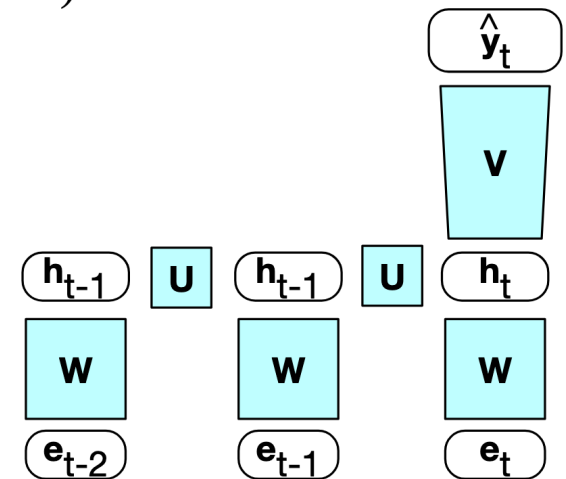
Inferenza

$$P(w_{t+1} = i | w_1, \dots, w_t) = \mathbf{y}_t[i]$$

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:i-1})$$

$$= \prod_{i=1}^n \mathbf{y}_i[w_i]$$

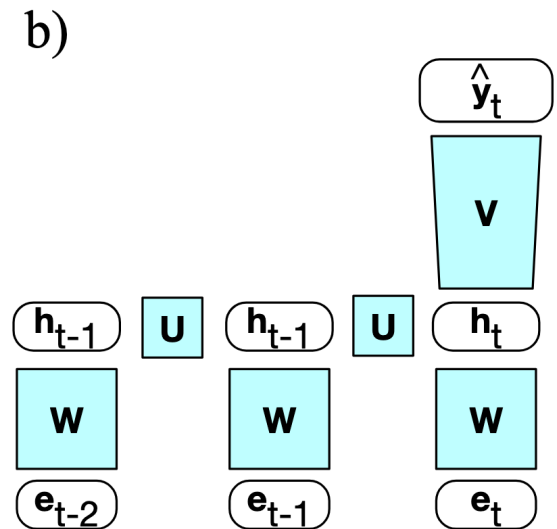
b)



Addestramento

Useremo la
self-supervision:

*la parola corretta da predire
è la successiva*

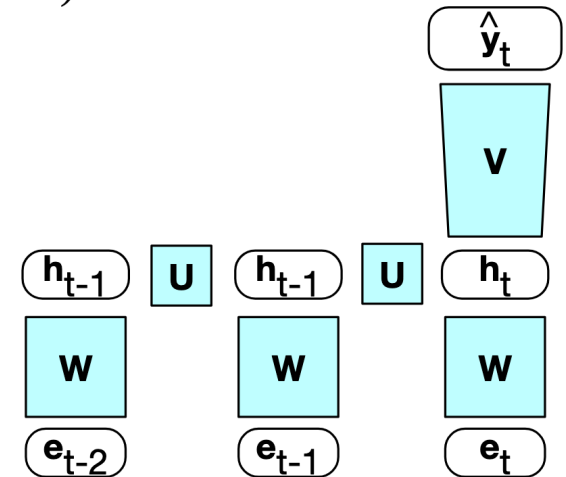


Training

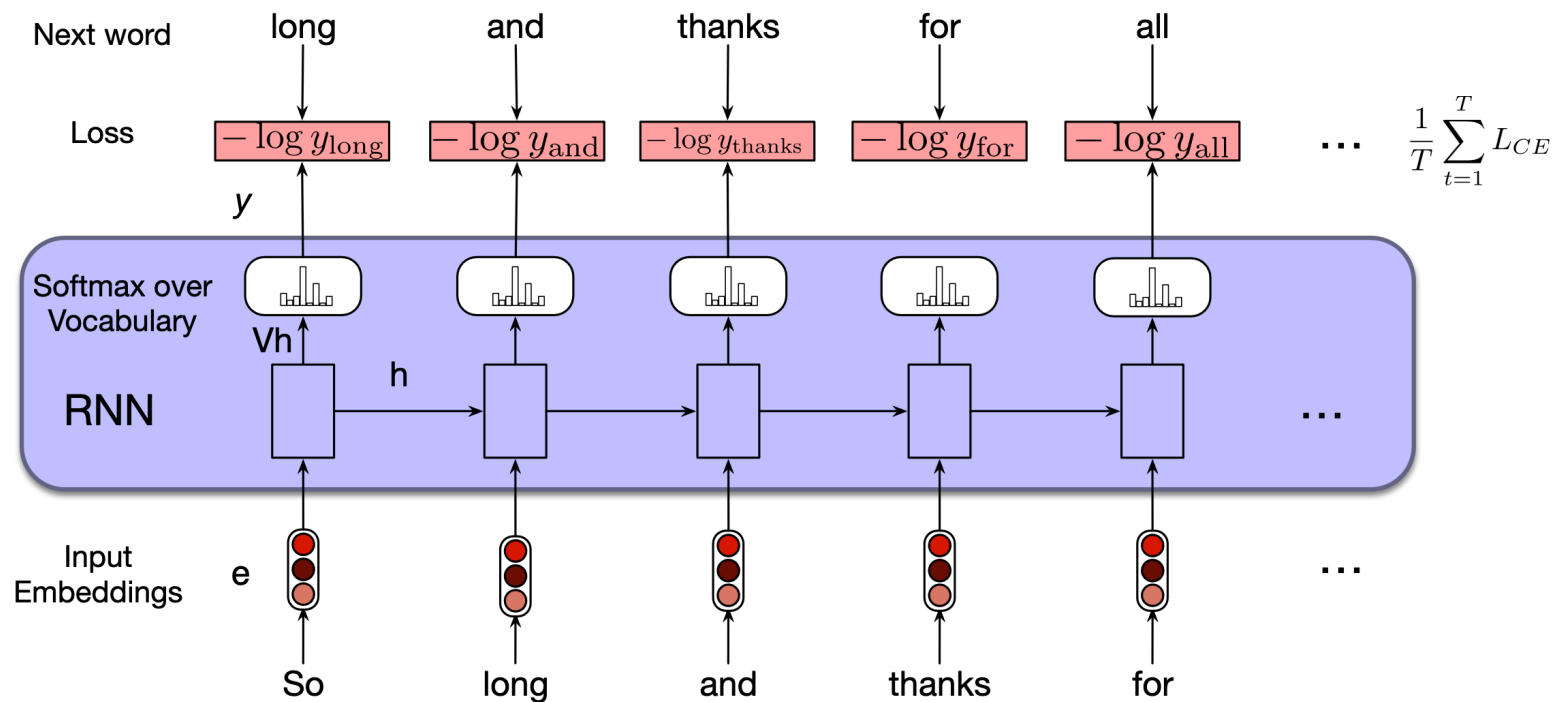
$$\text{Loss: } L_{CE} = - \sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w]$$

$$\text{Cioè: } L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = - \log \hat{\mathbf{y}}_t[w_{t+1}]$$

b)



Teacher forcing



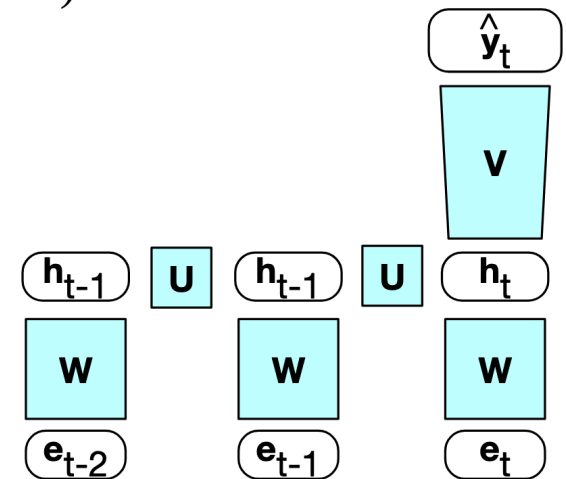
Ad ogni passo usiamo sempre la sequenza corretta per predire la parola successiva

Weight tying

- **E** è la matrice $d_h \times |V|$ dei word embedding appresi
- **V** è la matrice $|V| \times d_h$ degli score delle probabilità delle parole, data l'evidenza di ogni parola memorizzata in **h**

• *Sono davvero diverse??*

b)



Weight tying

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$$

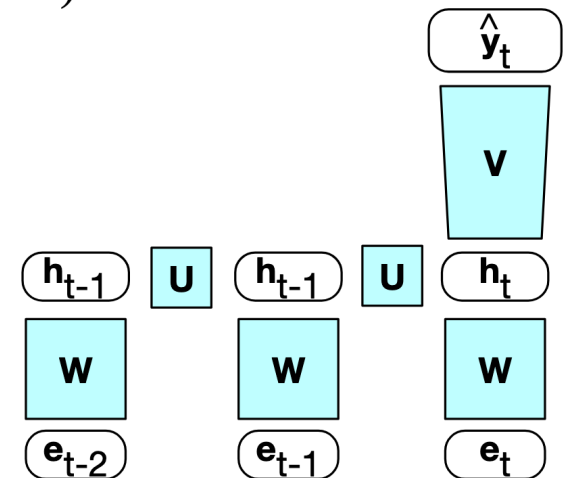
$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$$

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{E}^\top \mathbf{h}_t)$$

b)



RNN per altri task NLP



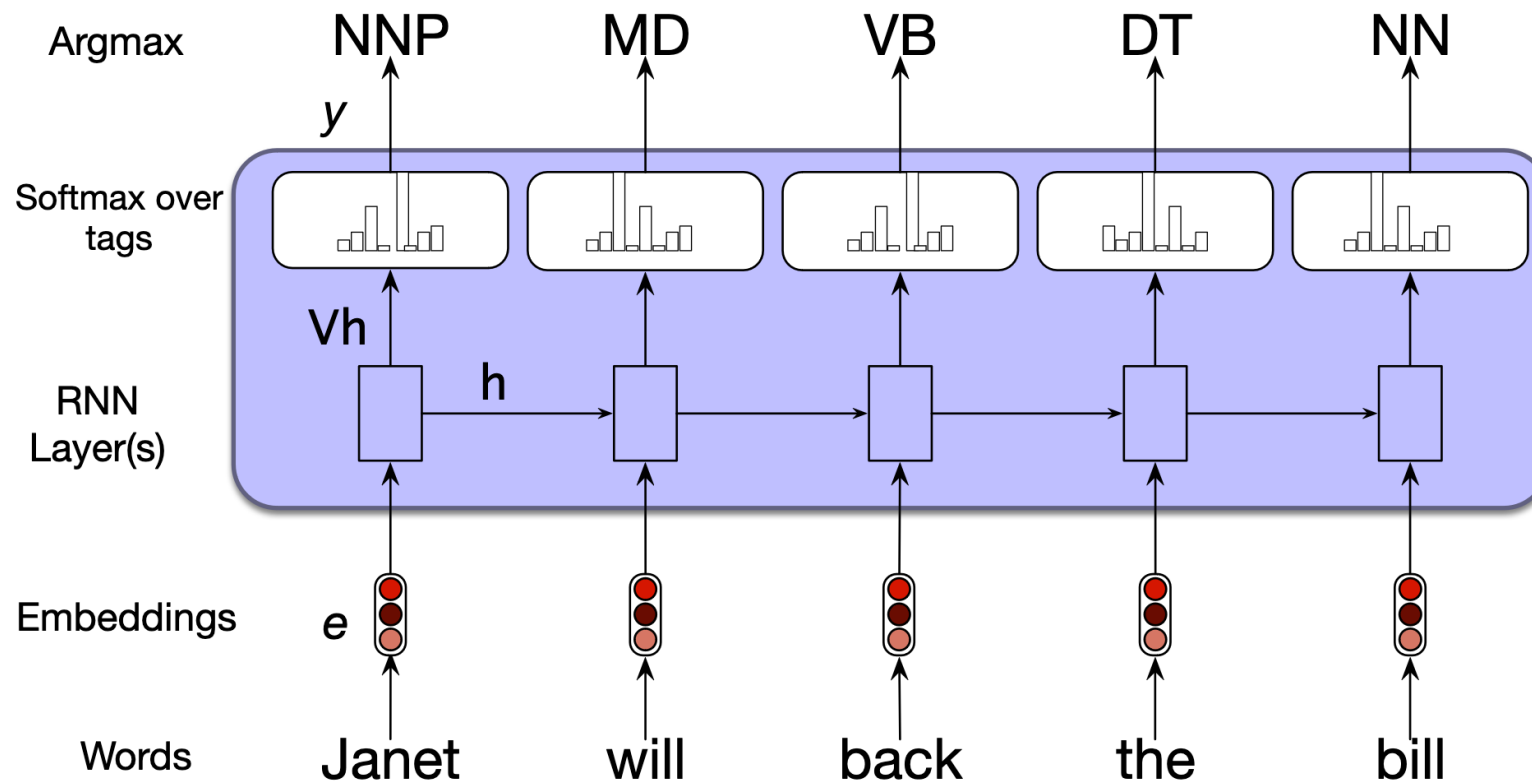
Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



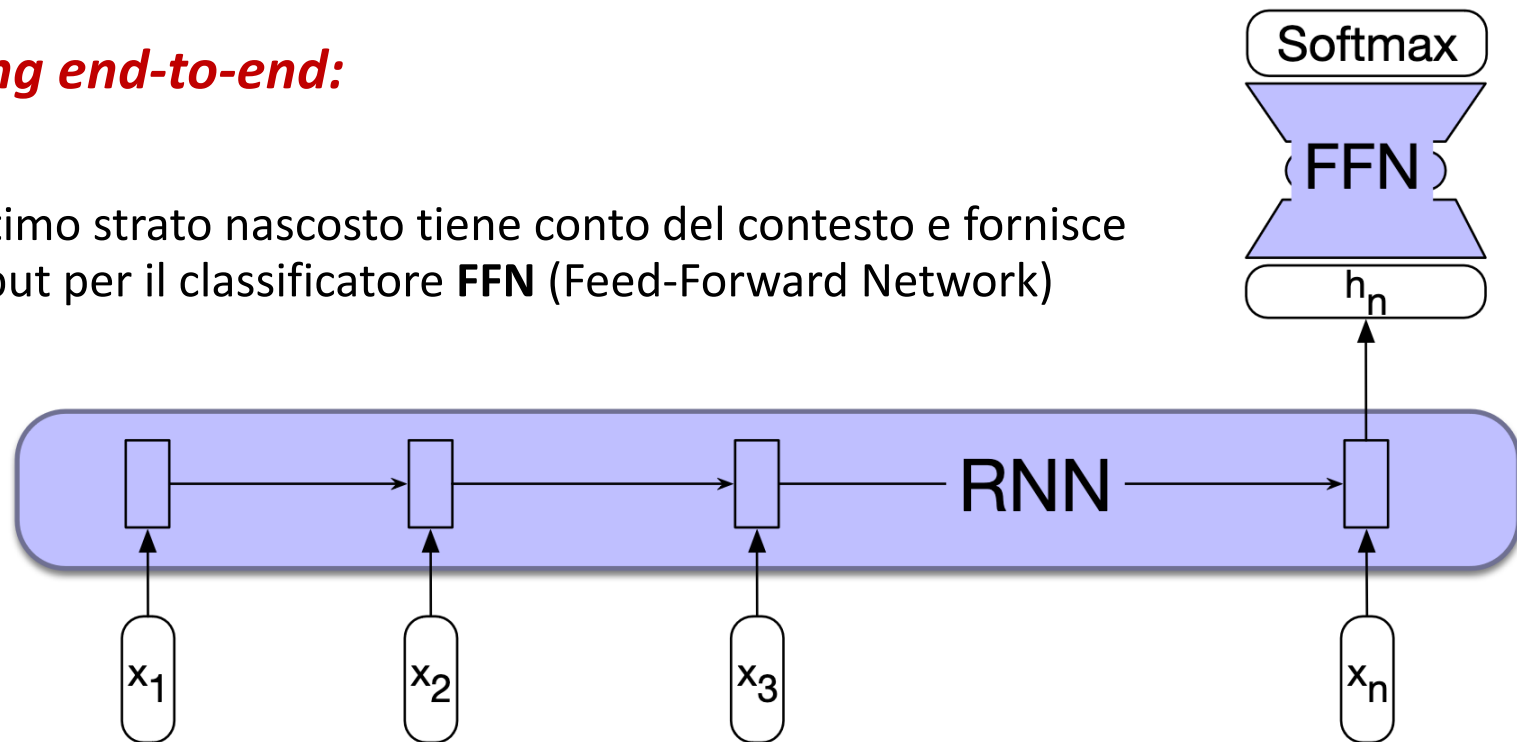
Sequence labeling



Sequence classification

- *Training end-to-end:*

- L'ultimo strato nascosto tiene conto del contesto e fornisce l'input per il classificatore **FFN** (Feed-Forward Network)

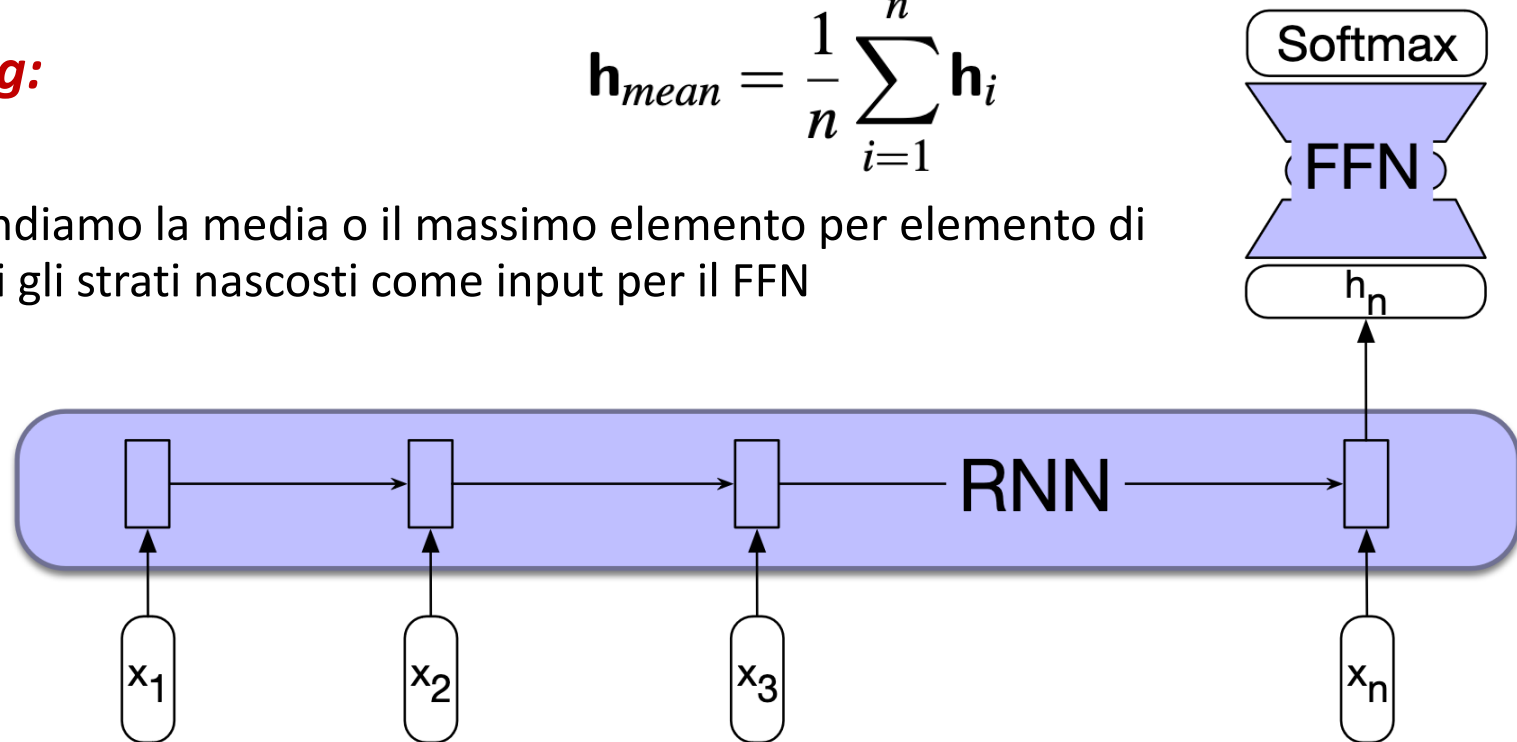


Sequence classification

- **Pooling:**

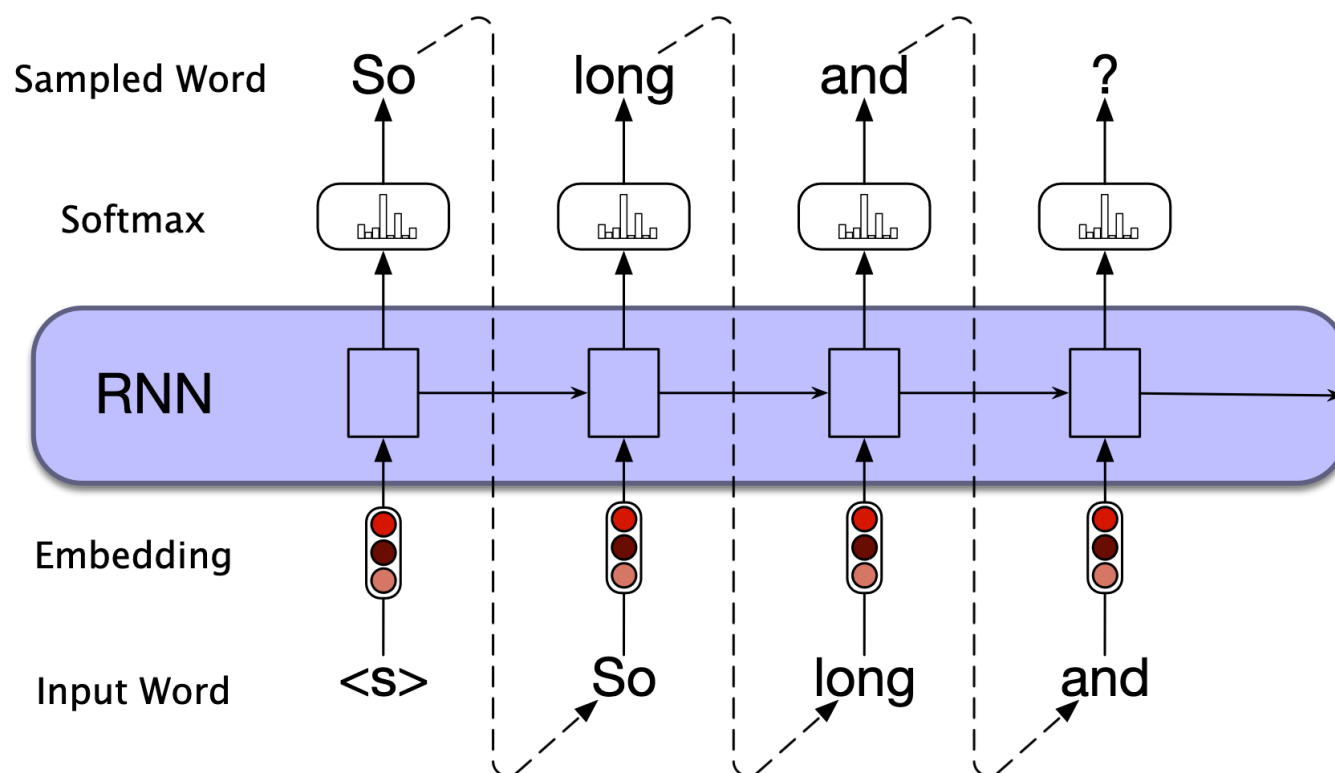
$$\mathbf{h}_{mean} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i$$

- Prendiamo la media o il massimo elemento per elemento di tutti gli strati nascosti come input per il FFN



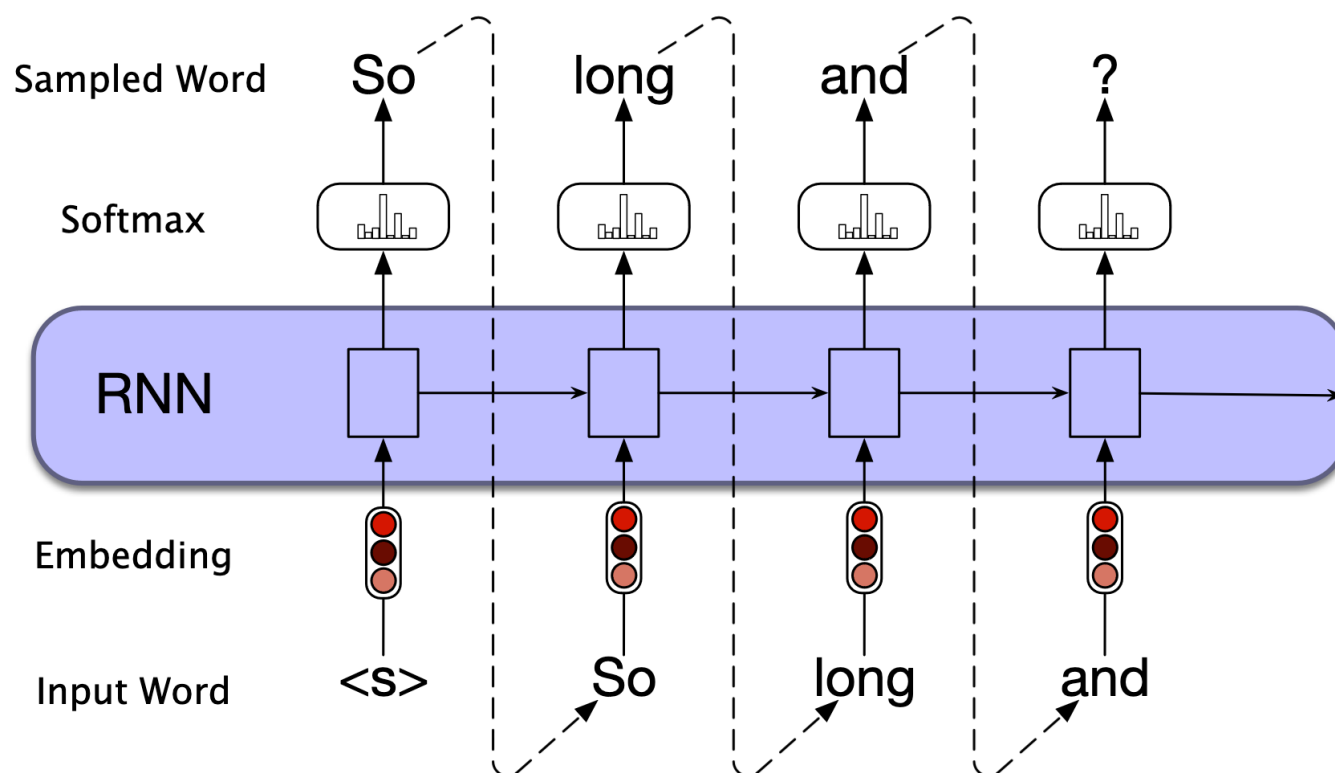
Text generation

Ogni parola è campionata come risultato della ***softmax precedente***



Text generation

L'output campionato è dato in input all'input successivo della RNN, e così via



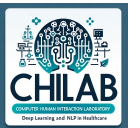
Architetture RNN «Impilate» (Stacked) e Bidirezionali



Università
degli Studi
di Palermo

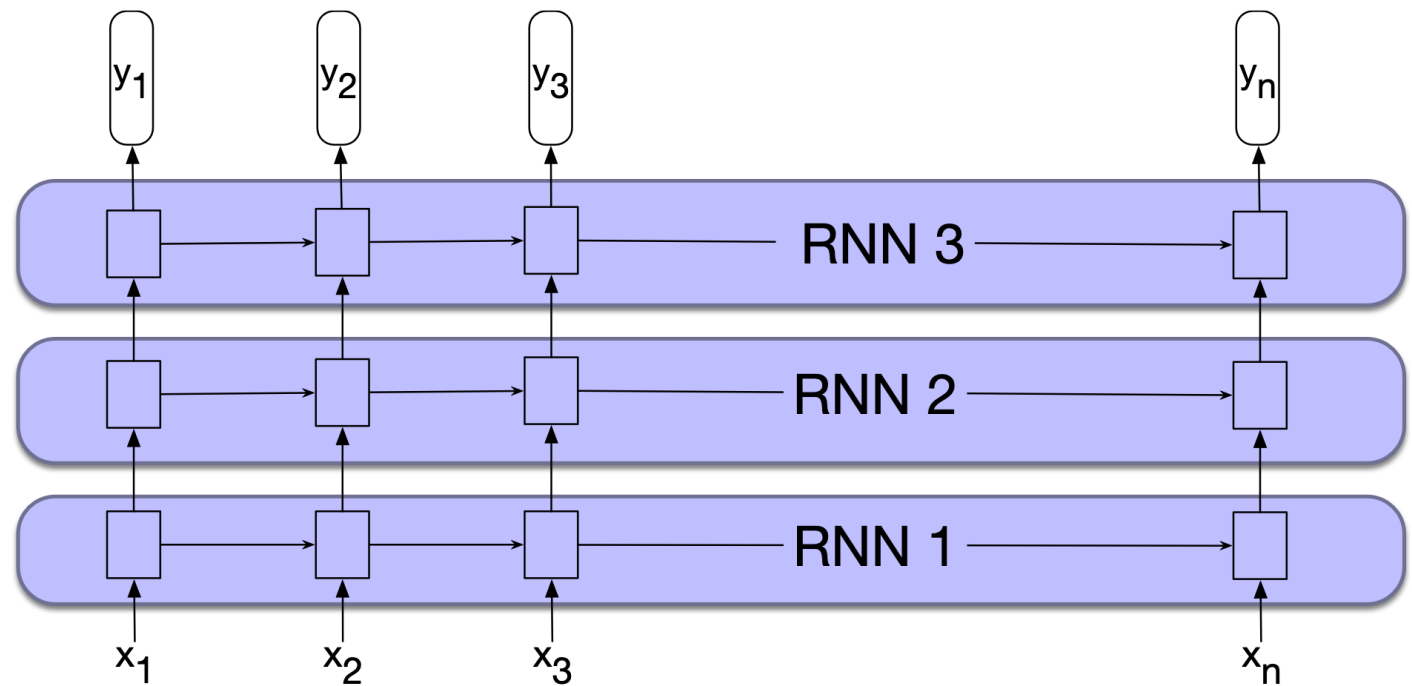


dipartimento
di ingegneria
unipa



Stacked RNN

La sequenza di output di un layer è l'input per quello successivo



RNN Bidirezionali

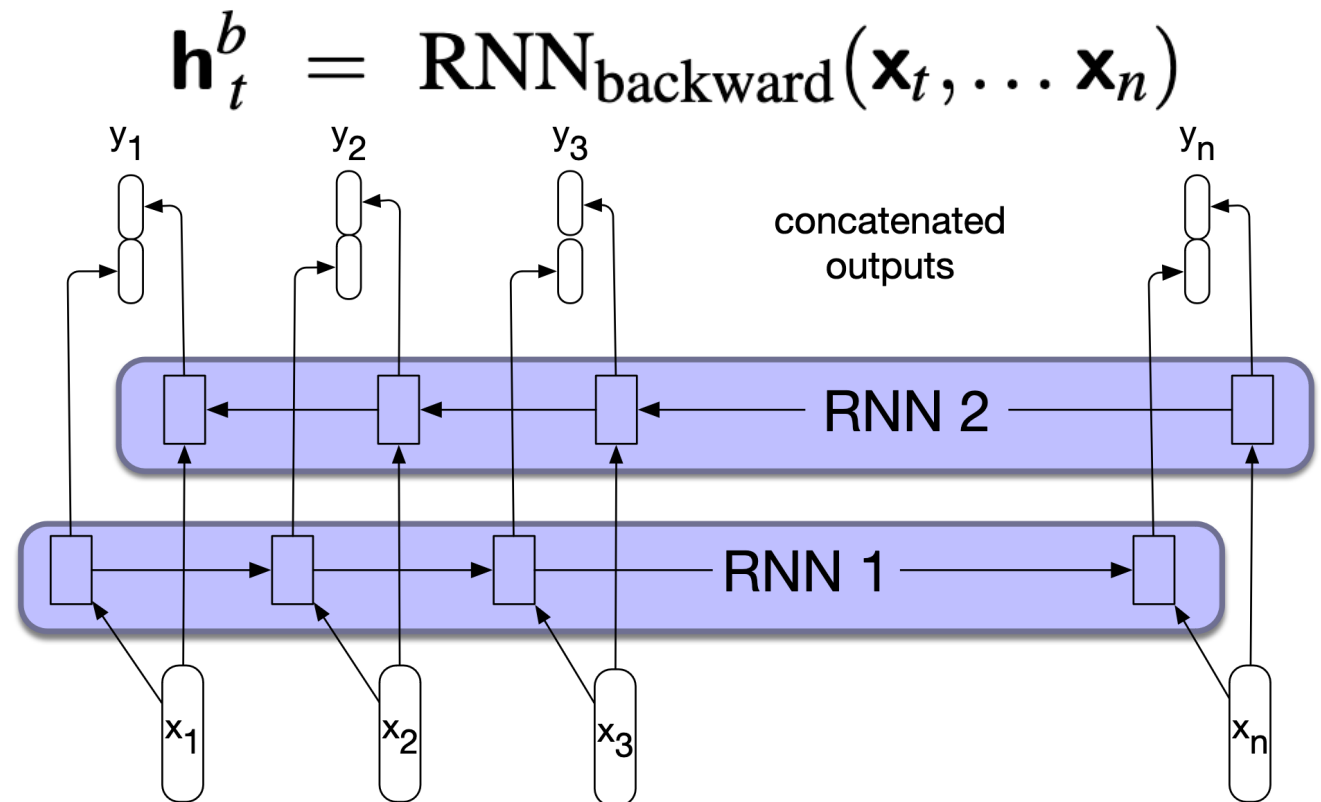
- Le RNN usano informazioni provenienti dal *contesto sinistro* al passo t
 - Lo stato nascosto $\mathbf{h}_t^f$ al passo t è una funzione in avanti del contesto $\mathbf{x}_1 \dots \mathbf{x}_t$

$$\mathbf{h}_t^f = \text{RNN}_{\text{forward}}(\mathbf{x}_1, \dots, \mathbf{x}_t)$$

- Spesso è utile ottenere informazioni anche dal *contesto destro*

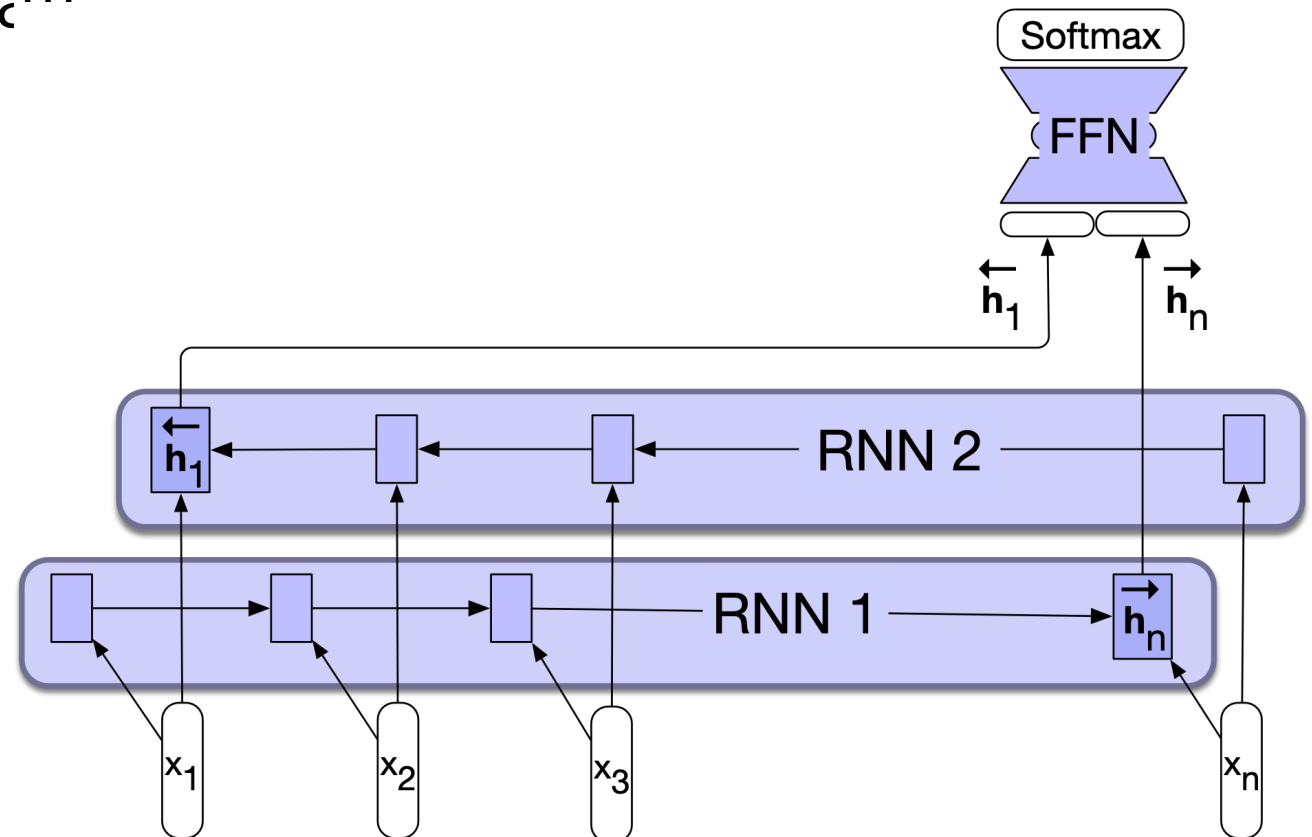
RNN Bidirezionali

- Le RNN bidirezionali sono addestrate in modalità forward e backward
- Gli output forward e backward vengono quindi concatenati



RNN Bidirezionale

Nella text
classification
 $\mathbf{h}_1^b \oplus \mathbf{h}_n^f$ viene
passato al
classificatore FFN



Long Short-Term Memory (LSTM)



Informazione distante

- Le RNN non sono efficaci nel gestire dipendenze tra parole distanti

*L'auto nuova che avevo comprato in concessionaria
il mese scorso **si è rotta***

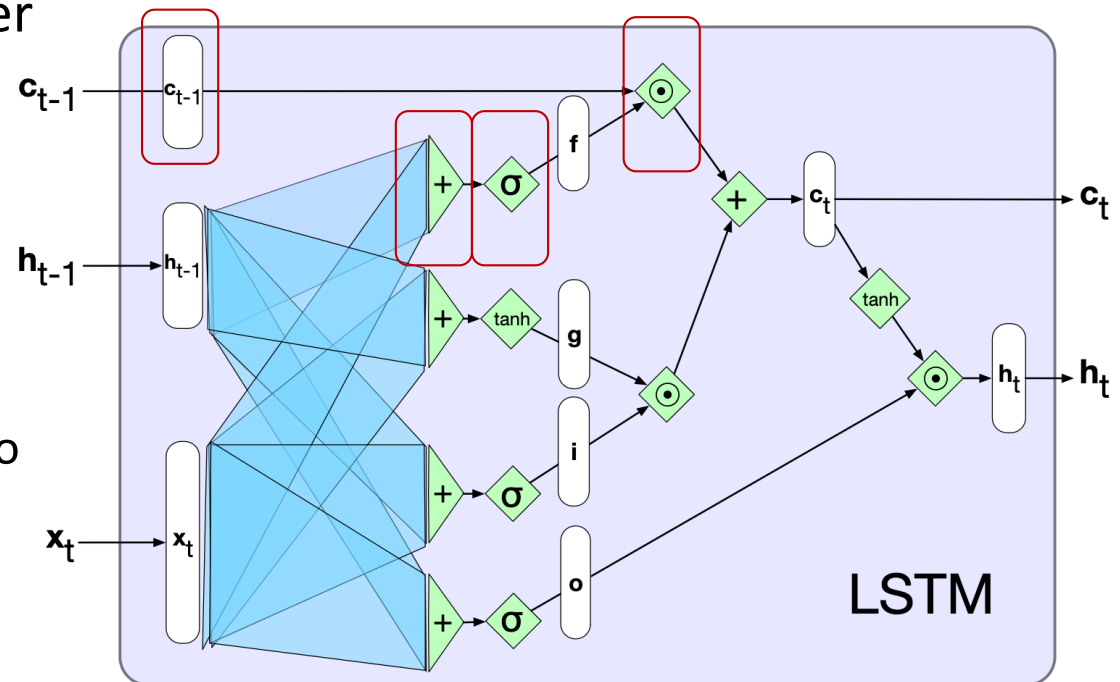
- Gli strati nascosti soffrono del problema dei **vanishing gradients** nel passaggio backward del training

Gate

- Le LSTM dividono il problema della gestione del testo in due sottoproblemi:
 - Rimuovere informazioni non più necessarie dal contesto
 - Aggiungere informazioni che probabilmente saranno necessarie per decisioni future
- I Gates sono unità neurali progettate per imparare la gestione del contesto

Gates

- Viene aggiunto un nuovo layer di contesto c_t
- Un *gate* è composto da:
 - Un feed-forward layer
 - Un'attivazione sigmoidale
 - Una moltiplicazione elemento per elemento con il layer da controllare

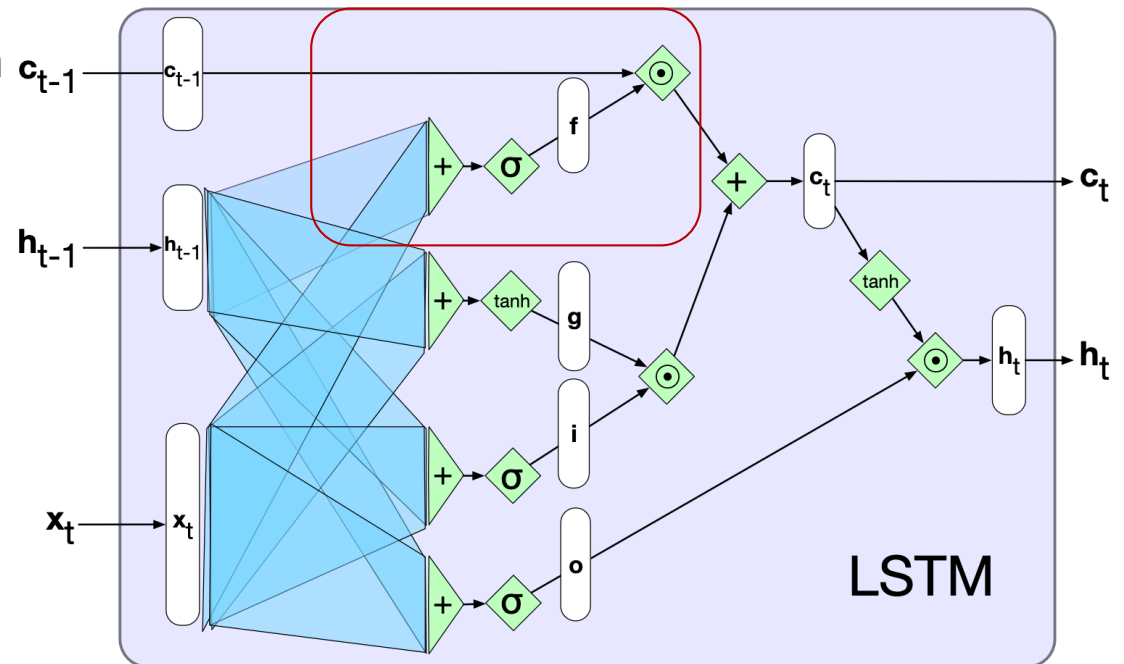


Forget Gate

- Elimina le informazioni dal contesto precedente che non sono più necessarie

$$\mathbf{f}_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t)$$

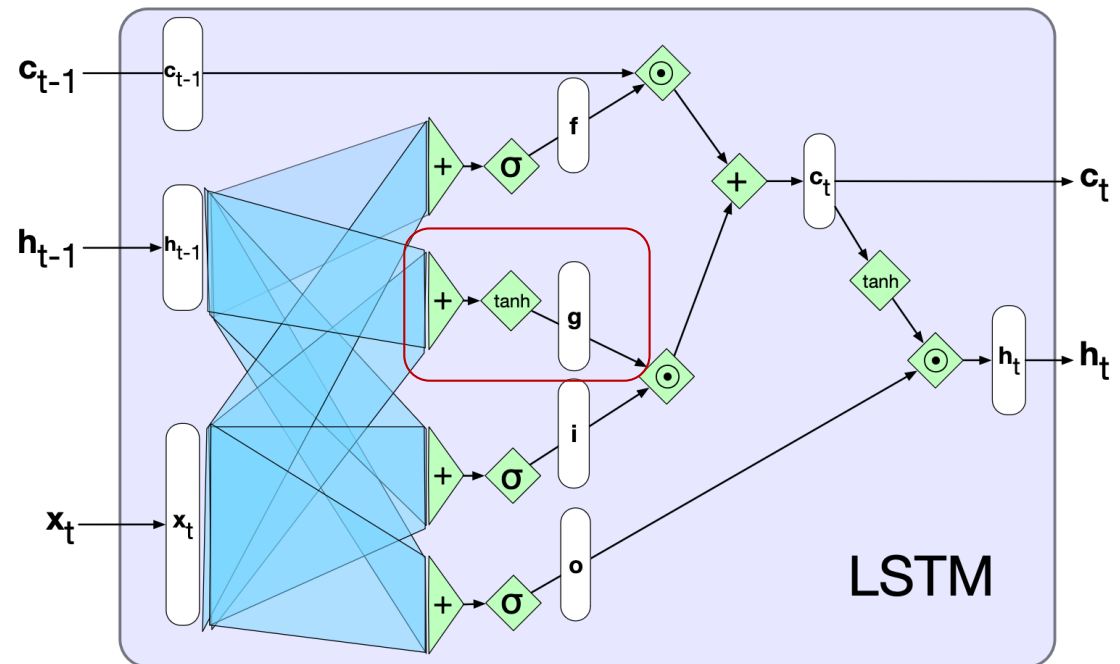
$$\mathbf{k}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t$$



Estrazione di informazioni al passo t

- Calcola le informazioni dallo stato nascosto precedente e dagli input correnti

$$\mathbf{g}_t = \tanh(\mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{W}_g \mathbf{x}_t)$$



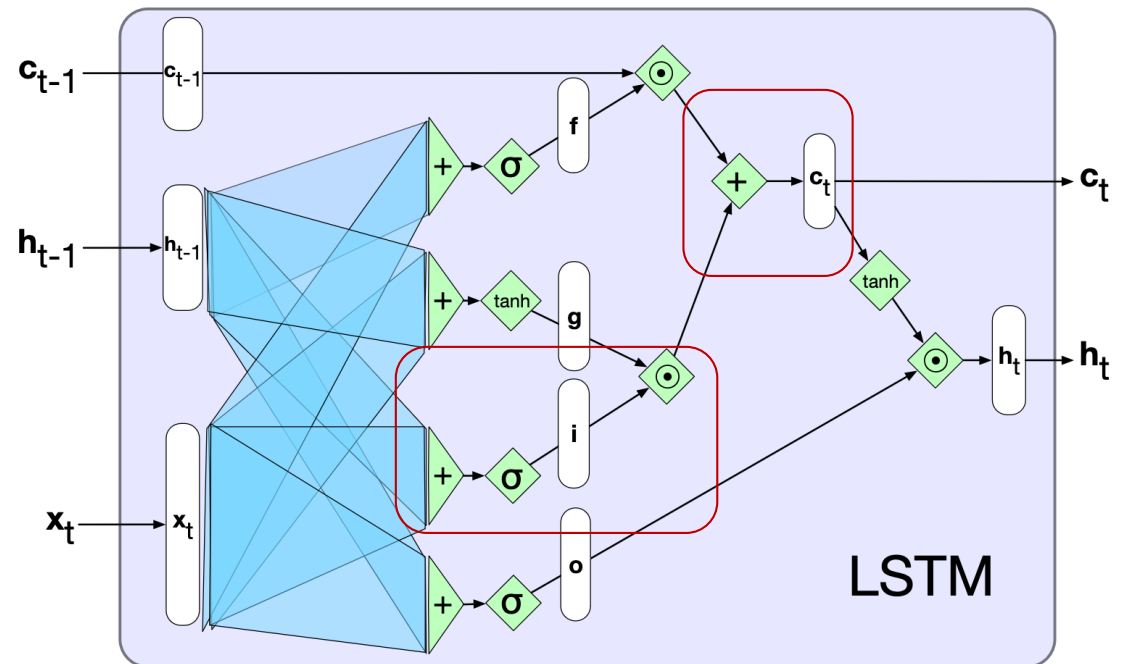
Add gate

- Seleziona le informazioni rilevanti e le aggiunge al contesto corrente

$$\mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t)$$

$$\mathbf{j}_t = \mathbf{g}_t \odot \mathbf{i}_t$$

$$\mathbf{c}_t = \mathbf{j}_t + \mathbf{k}_t$$

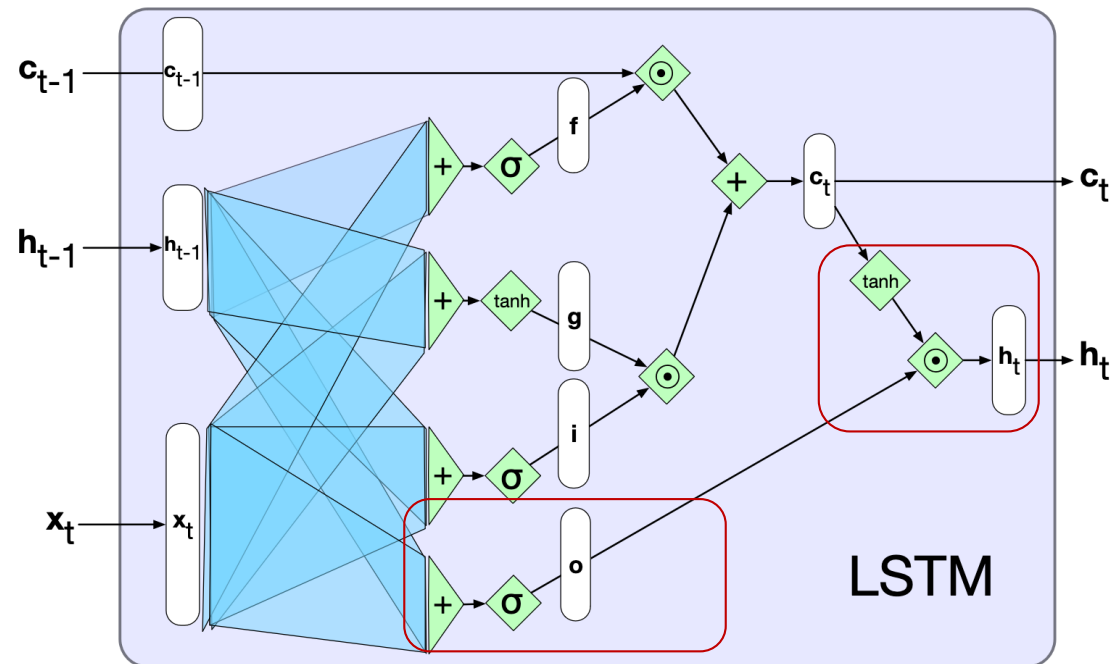


Output gate

- Decide quali sono le informazioni richieste per lo stato nascosto corrente

$$\mathbf{o}_t = \sigma(\mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{W}_o \mathbf{x}_t)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$



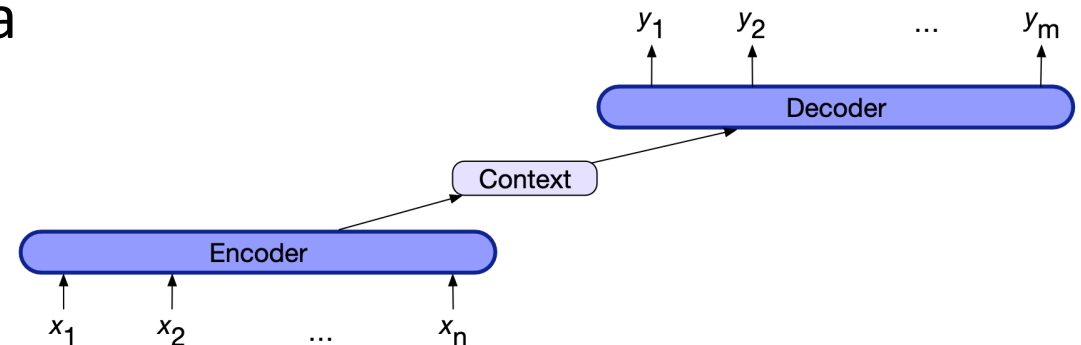
Il Modello Encoder-Decoder con RNN

Modello Encoder-decoder

- Task come la machine translation sono ancora task di sequence labeling ma...
 - La sequenza di input e output non hanno la stessa lunghezza
 - Il mapping tra coppie di token input/output può essere indiretto
 - I verbi hanno posizioni diverse in lingue diverse

Modello Encoder-decoder

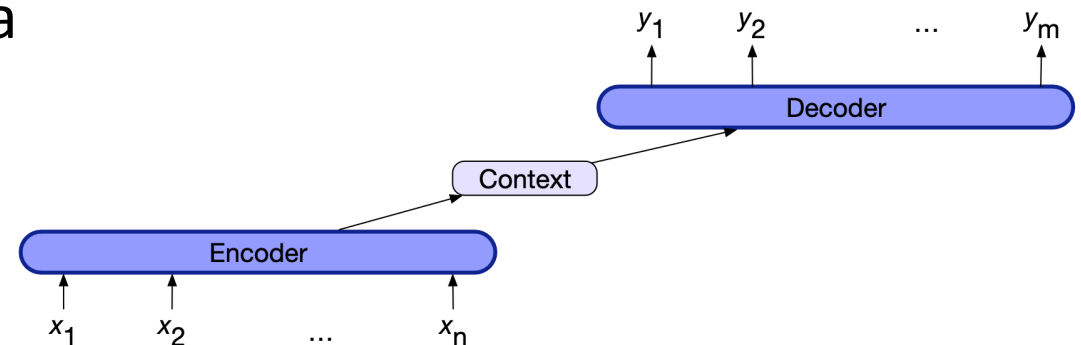
- Le reti *encoder-decoder* (note anche come *sequence-to-sequence*) affrontano questo problema



- Encoder: CNN/LSTM/Transformer che mappa $x_{1:n} \rightarrow h_{1:n}$

Modello Encoder-decoder

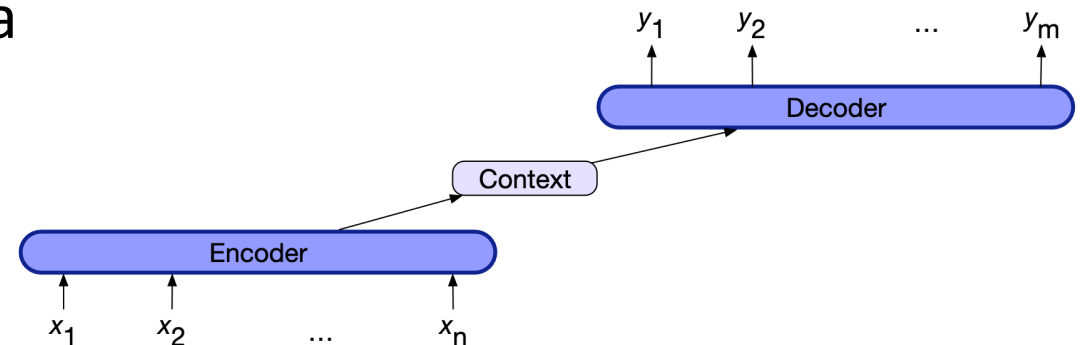
- Le reti *encoder-decoder* (note anche come *sequence-to-sequence*) affrontano questo problema



- Vettore di contesto (Context vector):
- $c = g(h_{1:n})$

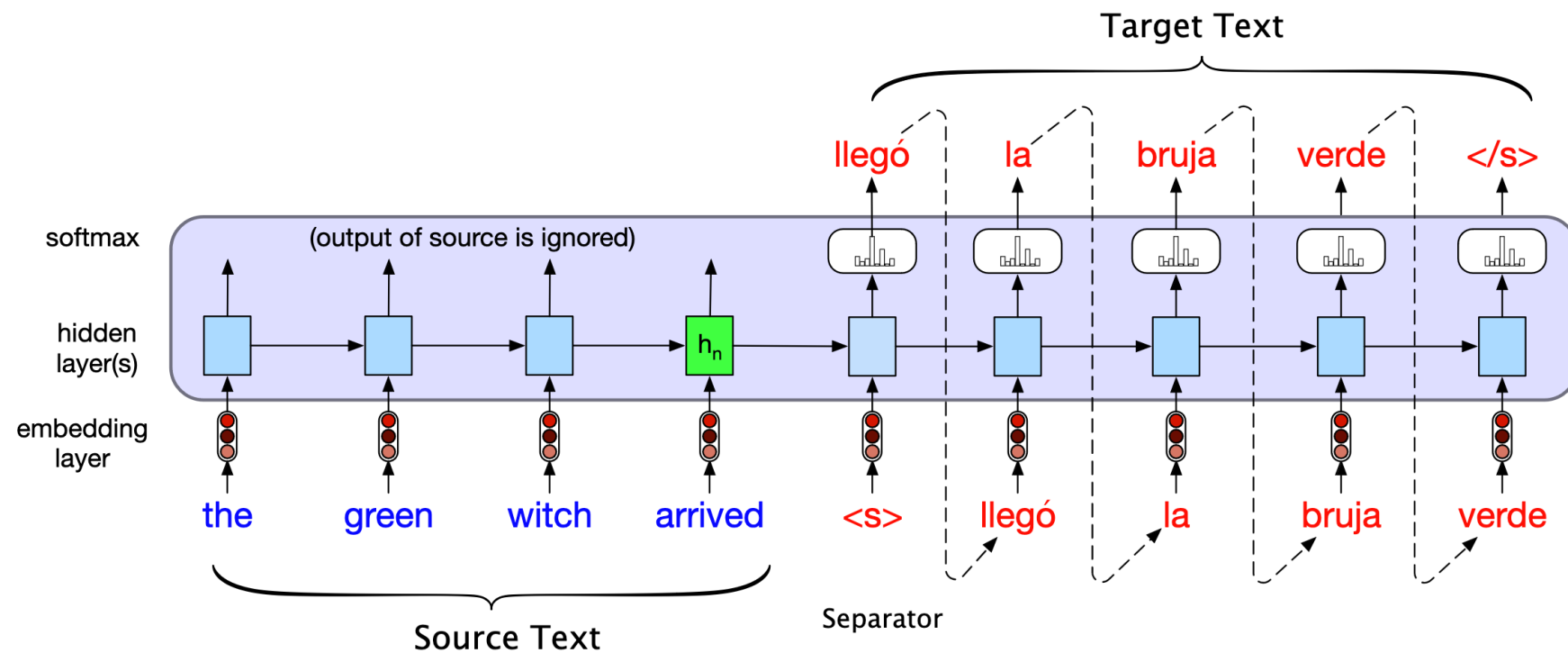
Modello Encoder-decoder

- Le reti *encoder-decoder* (note anche come *sequence-to-sequence*) affrontano questo problema



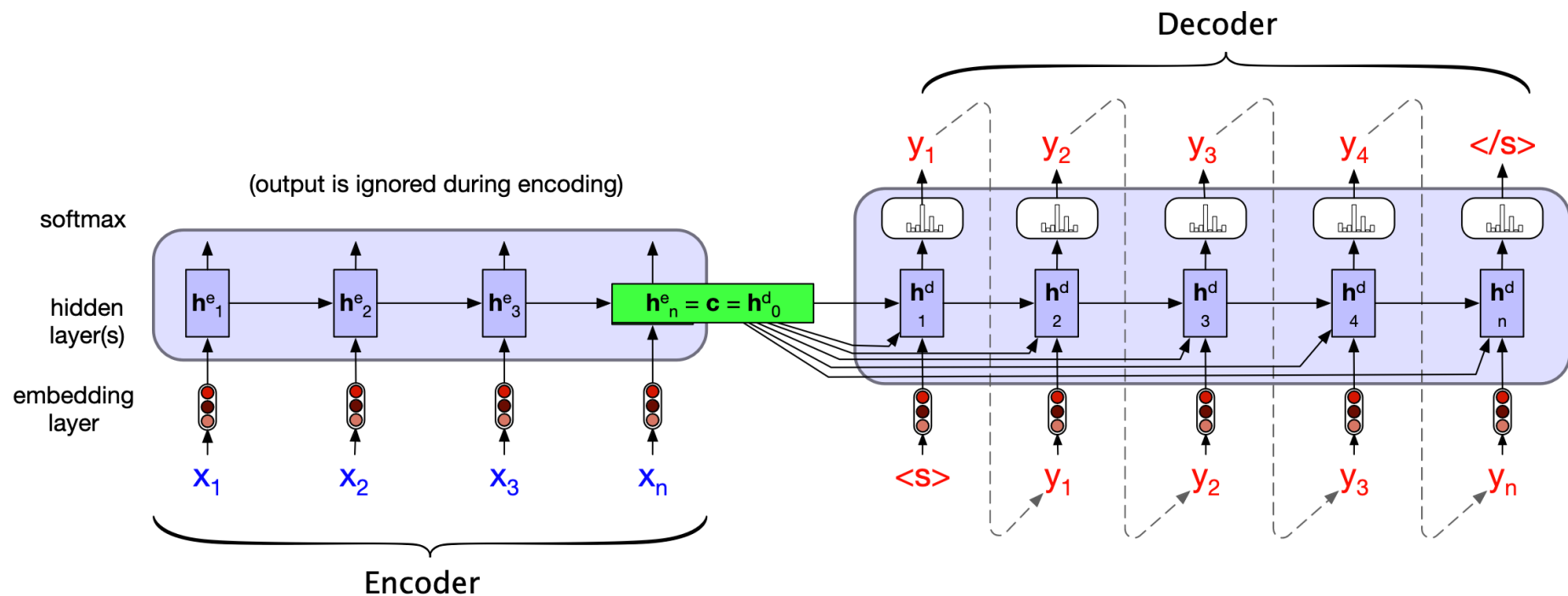
- Decoder: CNN/LSTM/Transformer che mappa $h_{1:m} = f(c)$, $h_{1:m} \rightarrow y_{1:m}$

Reti Encoder-decoder che usano RNN



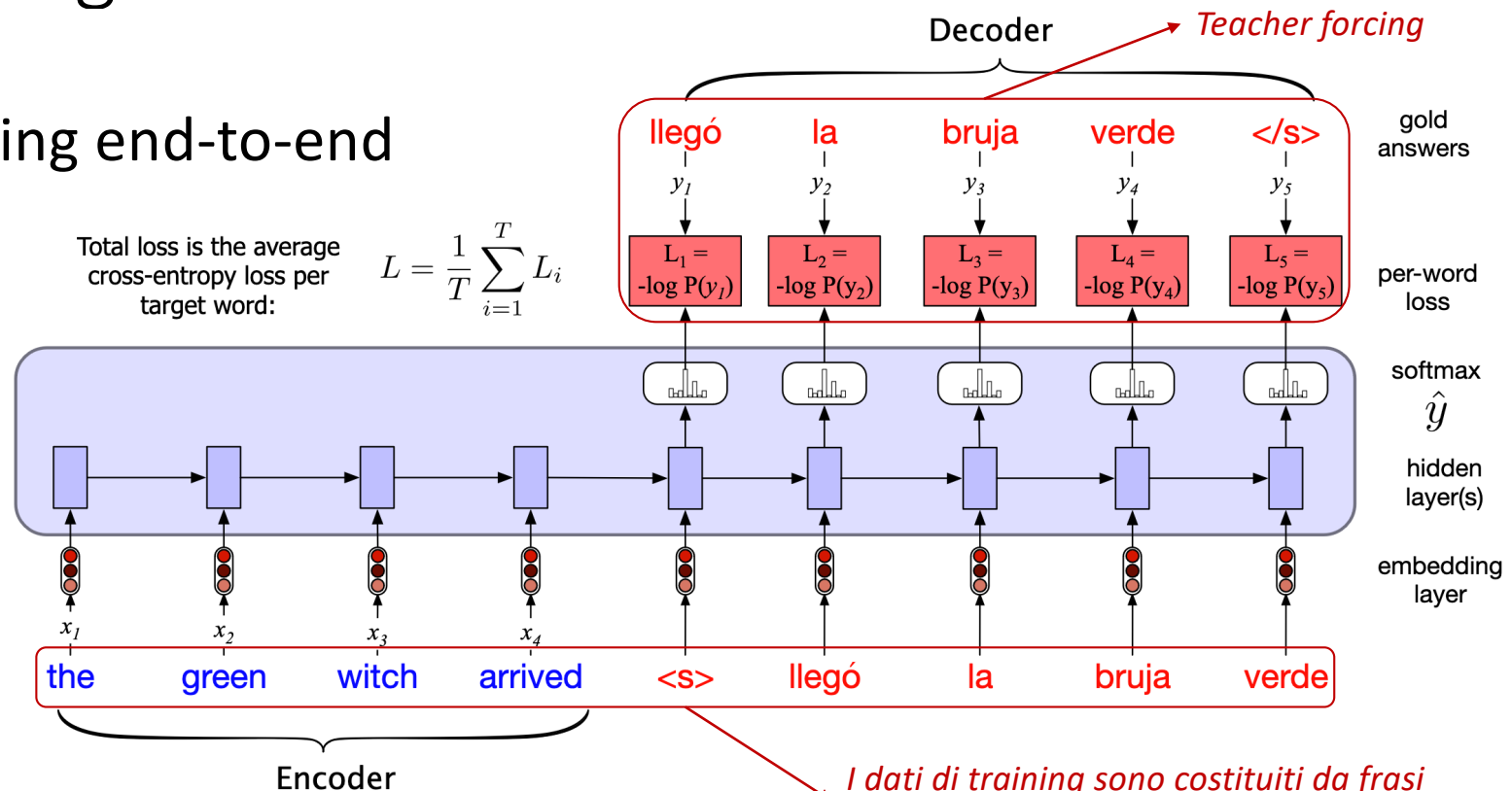
Inferenza

Reti Encoder-decoder che usano RNN



Training del modello encoder-decoder

- Training end-to-end



Attenzione



Università
degli Studi
di Palermo

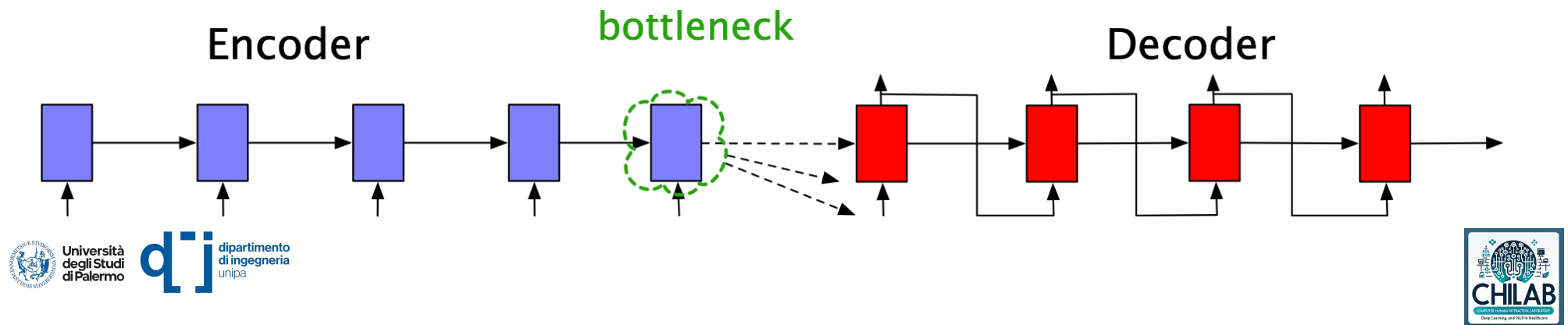


dipartimento
di ingegneria
unipa



Il collo di bottiglia del contesto

- **c** deve rappresentare *tutte* le informazioni provenienti dal testo sorgente
- Le dipendenze a lunga distanza potrebbero non essere ben rappresentate



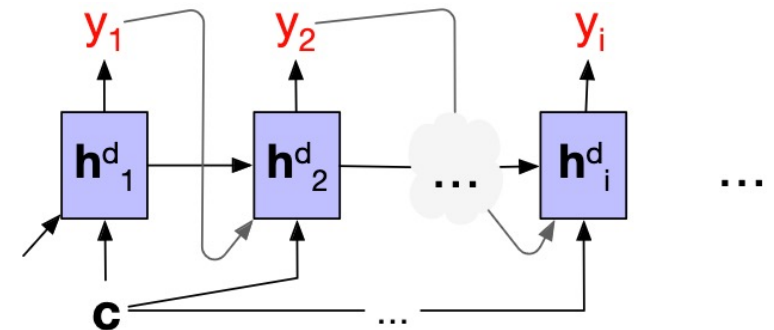
Meccanismo di Attenzione

- Volto a ottenere informazioni da tutti gli hidden states dell'encoder $\mathbf{c} = f(\mathbf{h}_1^e \dots \mathbf{h}_n^e)$
 - Vettore di lunghezza fissa calcolato come somma pesata di tutti gli hidden states dell'encoder
 - I pesi si focalizzano su una parte specifica del testo sorgente che è rilevante per il token che il decoder sta attualmente producendo

Meccanismo di Attenzione

- Il context vector è generato ad ogni passo di decodifica i

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$



- Iniziamo calcolando un set di scores che misurano quanto ogni hidden state dell'encoder sia rilevante per l'hidden state del decoder al passo $i-1$

$$score(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e)$$

Dot-product Attention

- Lo score più semplice è il prodotto scalare

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e$$

- Il dot product è uno scalare che riflette il grado di similarità tra i due vettori
- $\mathbf{h}_j^e$ e $\mathbf{h}_{i-1}^d$ devono avere la stessa dimensionalità

Pesi del context vector

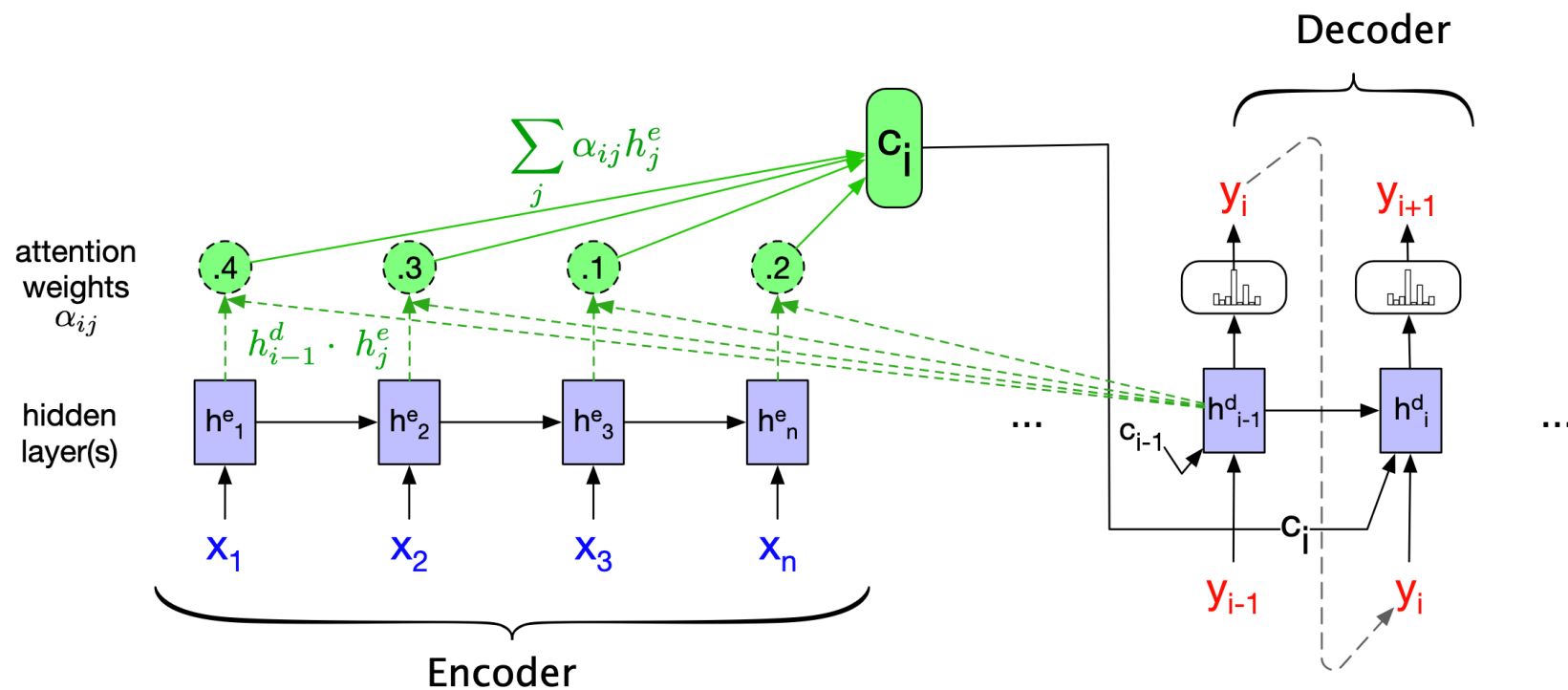
- Usiamo la softmax per normalizzare gli score e calcolare i pesi effettivi α_{ij}

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) \quad \forall j \in e)$$

$$= \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))}$$

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$

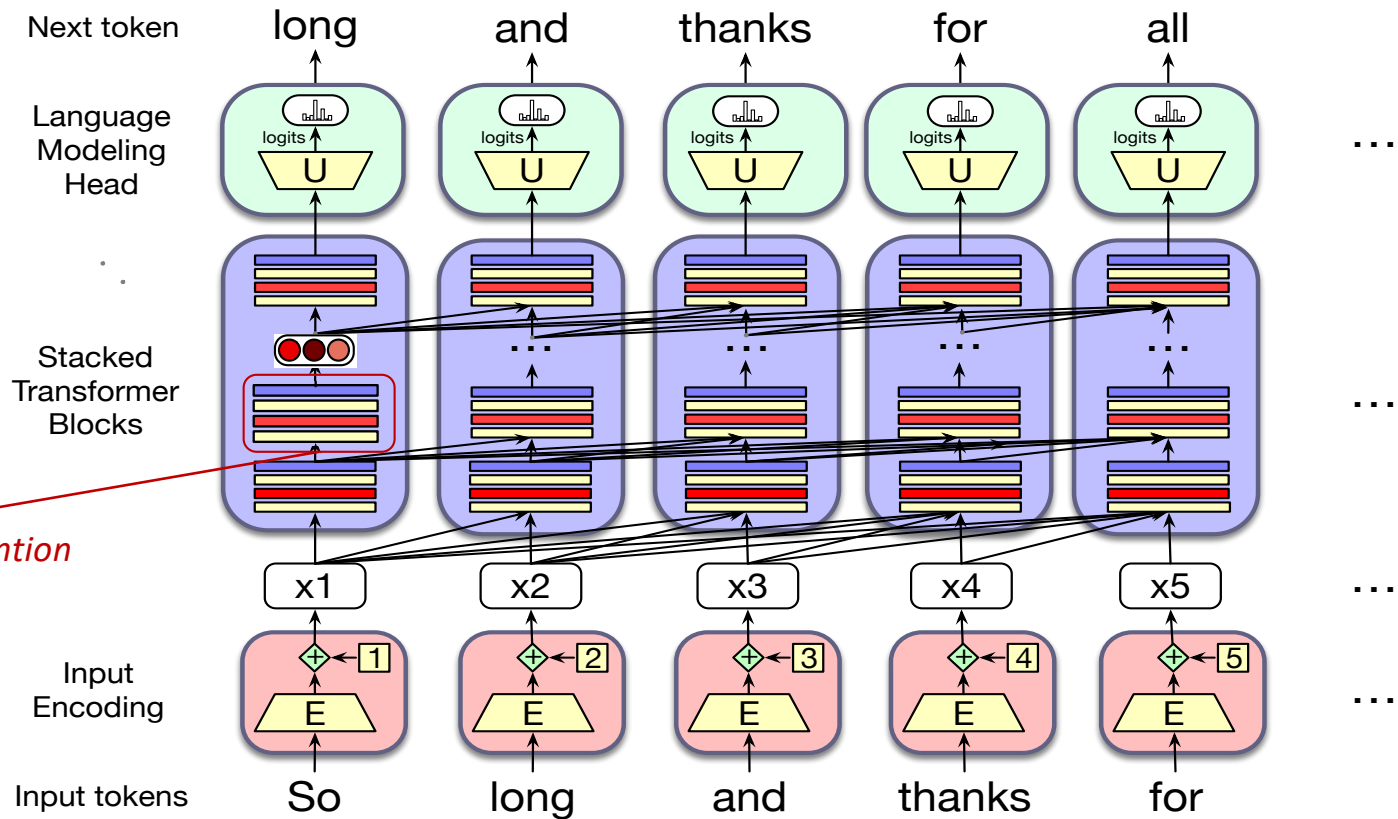
Rete Encoder-decoder con attenzione



Trasformer

Consideriamo gli **embedding** di una singola parola provenienti da un layer specifico.

Multi-head attention layer



Problemi con gli embedding statici (word2vec)

- Sono **statici**!

L'embedding di una parola non riflette come il suo significato cambi a seconda del contesto.

- The chicken didn't cross the road because **it** was too tired

- Qual è il significato rappresentato nell'embedding statico della parola "*it*"?

Embedding contestuali

- Intuizione: la rappresentazione del significato di una parola dovrebbe essere ***diversa nei diversi contesti!***
- ***Embedding contestuale***: ogni parola ha un ***vettore differente*** che esprime ***significati diversi*** a seconda delle parole che la circondano.
- Come calcolare gli embedding contestuali?
👉 ***Self-attention***

Embedding contestuali

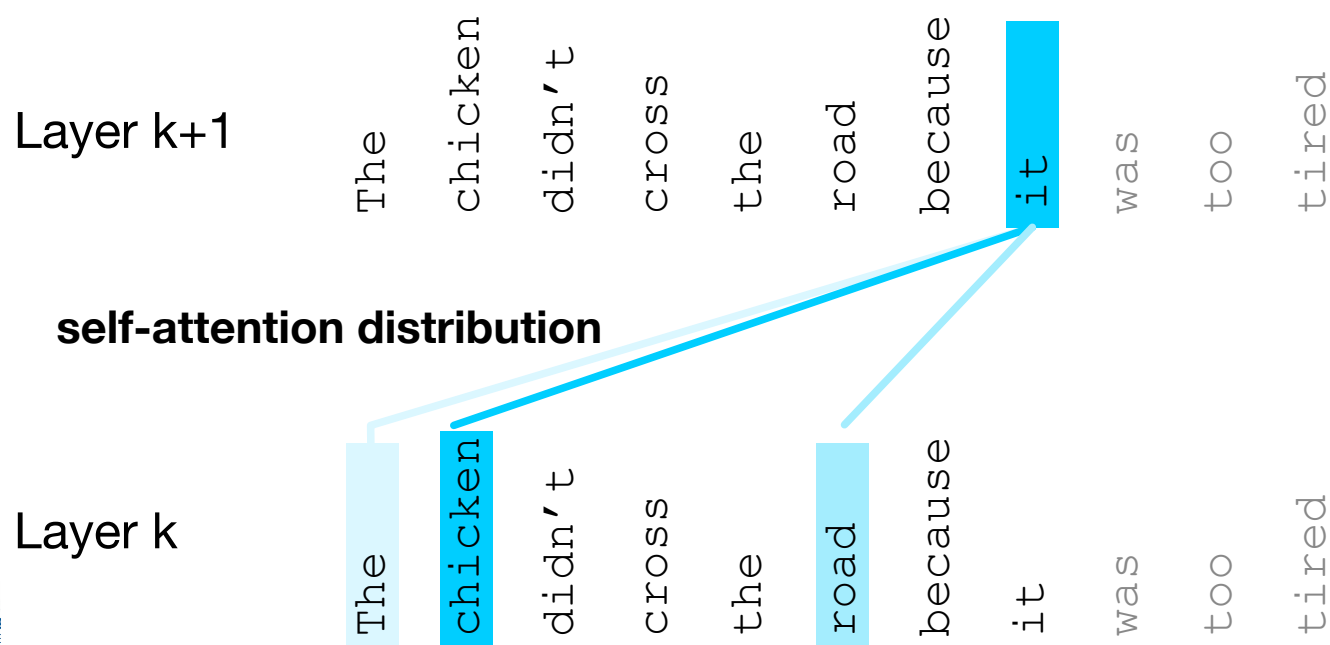
- The chicken didn't cross the road because it
- Quali potrebbero essere le proprietà di “it”?
- The chicken didn't cross the road because it was too **tired**
- The chicken didn't cross the road because it was too **wide**
- A questo punto della frase, probabilmente si riferisce o al pollo o alla strada.

Intuizione della self-attention

- Costruire l'embedding contestuale di una parola integrando selettivamente le informazioni provenienti da tutte le parole vicine.
- Si dice che una parola “presta attenzione” (*attends to*) ad alcune parole vicine ***più che ad altre***.

Intuizione della self-attention

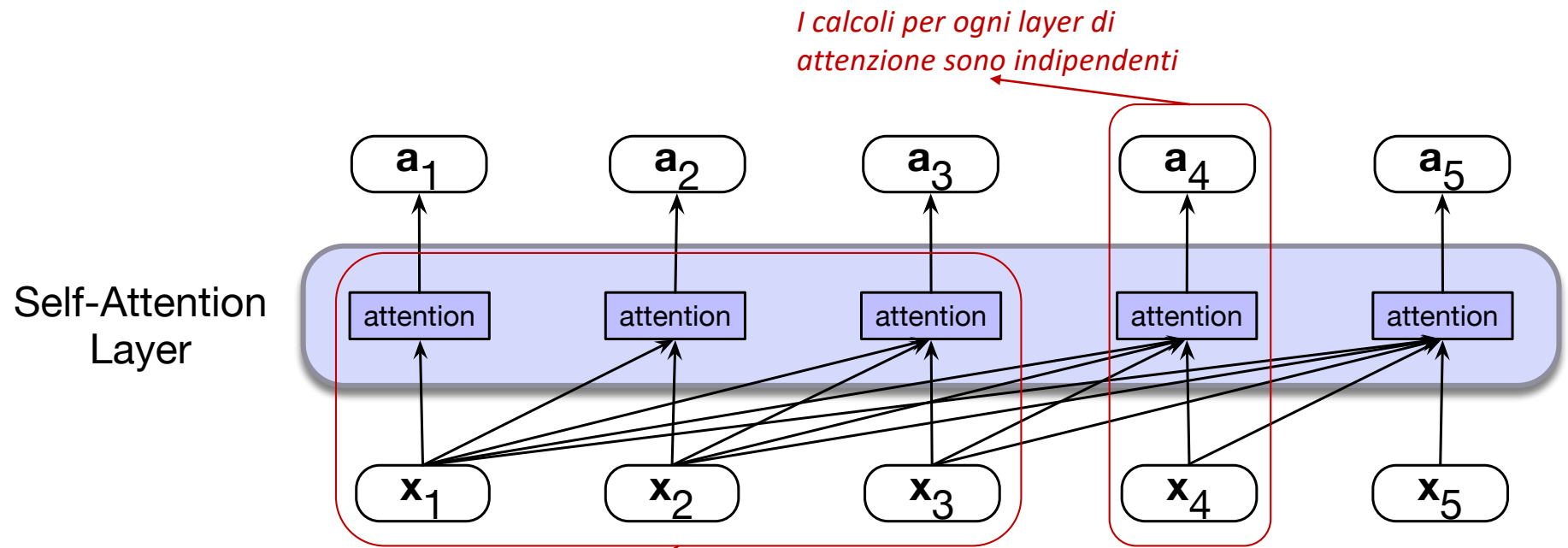
columns corresponding to input tokens



Definizione di attenzione

- Un meccanismo che aiuta a calcolare l'embedding di un token prestando selettivamente attenzione e integrando le informazioni provenienti dai token circostanti (nel livello precedente).
- Più formalmente: un metodo per eseguire una somma pesata di vettori.

L'attenzione va da sinistra a destra



Versione semplificata del meccanismo di attenzione

Data una sequenza di embedding di token:

$$\mathbf{x}_1 \quad \mathbf{x}_2 \quad \mathbf{x}_3 \quad \mathbf{x}_4 \quad \mathbf{x}_5 \quad \mathbf{x}_6 \quad \mathbf{x}_7 \quad \mathbf{x}_i$$

Produci $\mathbf{a}_i$ = somma pesata da $\mathbf{x}_1$ a $\mathbf{x}_7$ (e $\mathbf{x}_i$)
pesati rispetto alla loro similarità con $\mathbf{x}_i$

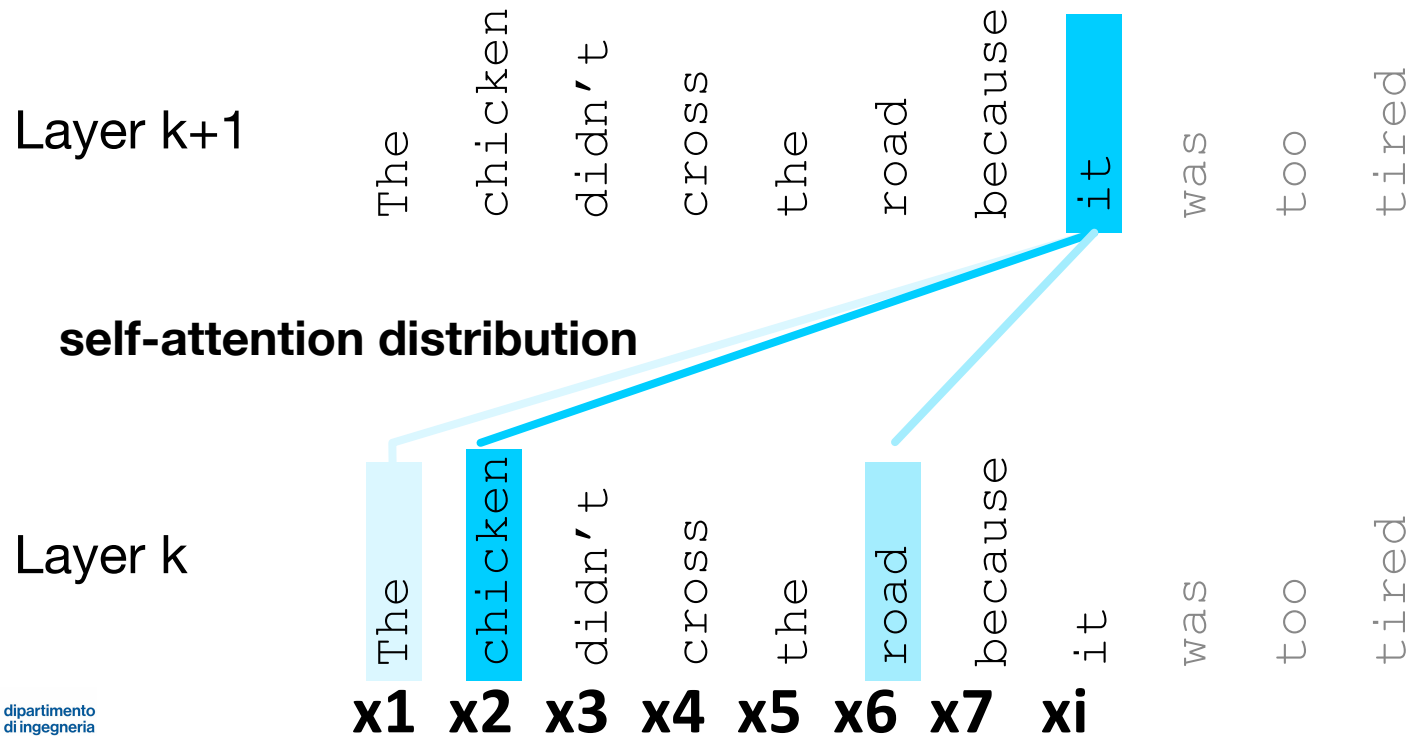
$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

Intuizione della self-attention

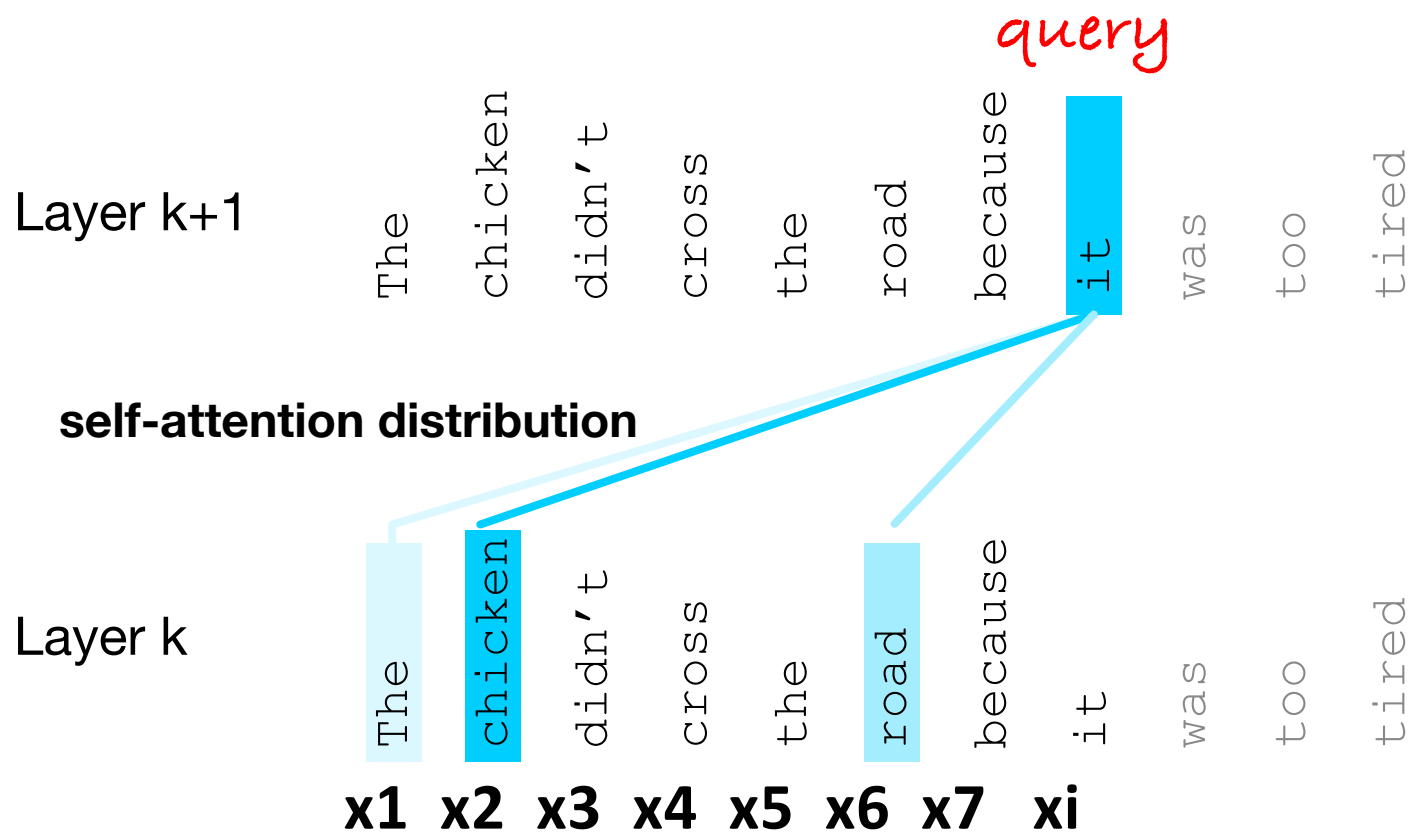
columns corresponding to input tokens



Una vera testa di attenzione: leggermente più complessa

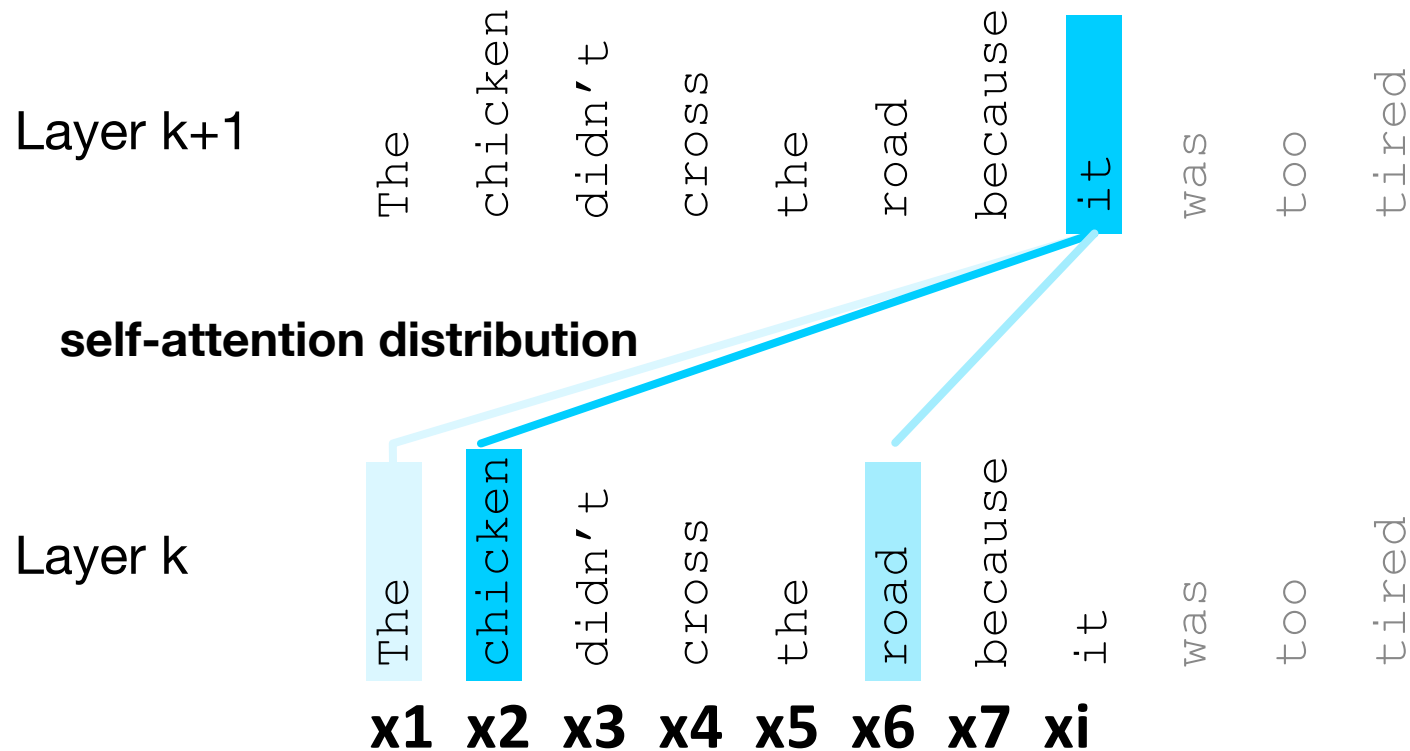
- Idea generale: invece di usare direttamente i vettori (come x_i e x_4), rappresenteremo *tre ruoli distinti* che ciascun vettore x_i svolge:
- **Query:** rappresenta l'*elemento corrente* che viene confrontato con gli input precedenti.
- **Key:** rappresenta un *input precedente* che viene confrontato con l'elemento corrente per determinare una misura di somiglianza.
- **Value:** rappresenta il *valore di un elemento precedente* che viene pesato e sommato per calcolare l'output dell'elemento corrente.

Intuizione della self-attention



Intuizione della self-attention

query



Una vera testa di attenzione: leggermente più complessa

- Useremo matrici per proiettare ogni vettore $\mathbf{x}_i$ nella rappresentazione del suo ruolo come query, chiave e valore:

- **query:** $\mathbf{W}^Q$

Implementazione originale

- **key:** $\mathbf{W}^K$

- **value:** $\mathbf{W}^V$

$d = 512$ Dimensionalità del modello

$d_k = d_v = 64$

$\mathbf{x}_i: (1, d)$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

$(1, d_k)$

(d, d_k)

$(1, d_k)$

(d, d_k)

$(1, d_v)$

(d, d_v)

Una vera testa di attenzione: leggermente più complessa

- Date queste tre rappresentazioni di $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

- Per calcolare la somiglianza tra l'elemento corrente $\mathbf{x}_i$ e un elemento precedente $\mathbf{x}_j$
- useremo il prodotto scalare tra $\mathbf{q}_i$ e $\mathbf{k}_j$.
- E invece di sommare $\mathbf{x}_j$, sommiamo $\mathbf{v}_j$

Equazioni finali per una testa di attenzione

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K; \quad \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V$$



*La scalatura previene
valori troppo grandi per
il prodotto scalare*



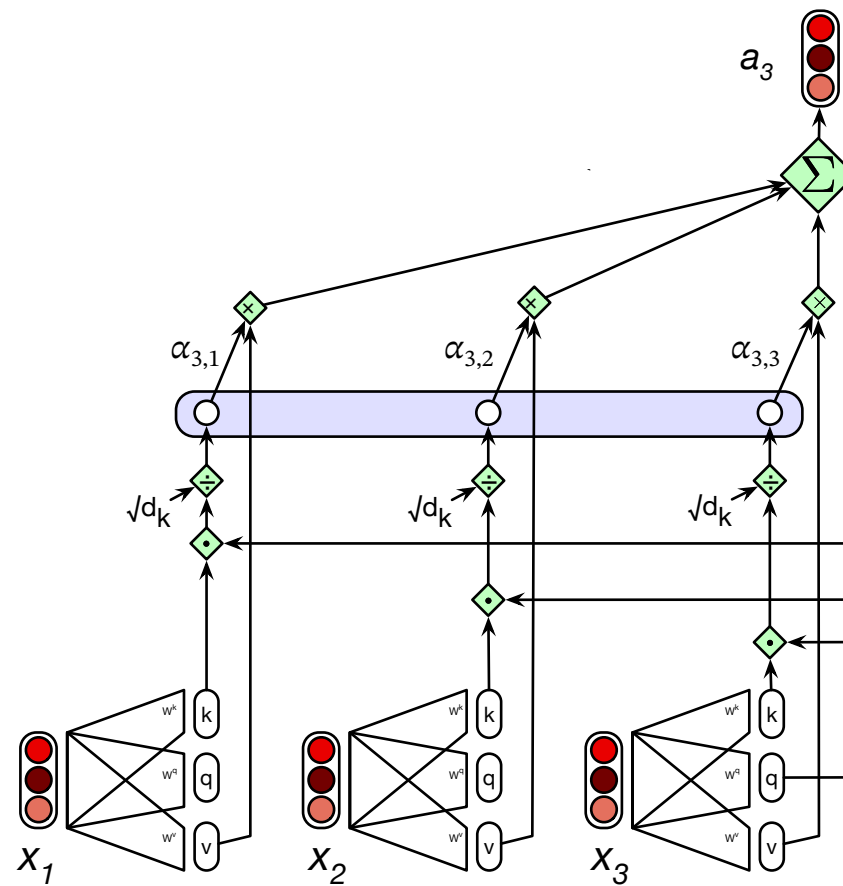
Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Calcolo del valore di a_3



Una vera attenzione: ancora più complessa

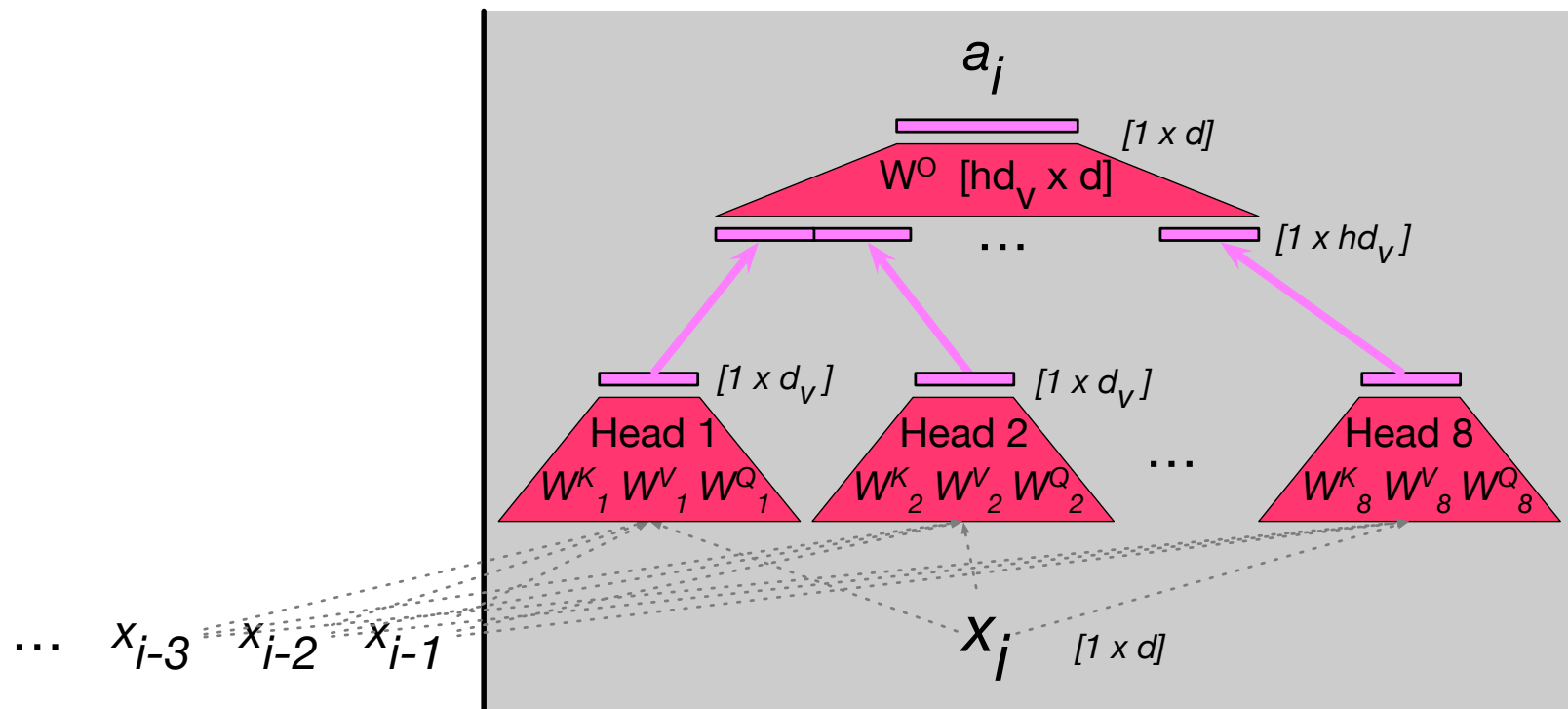
- Invece di una sola testa di attenzione, ne avremo

molte!

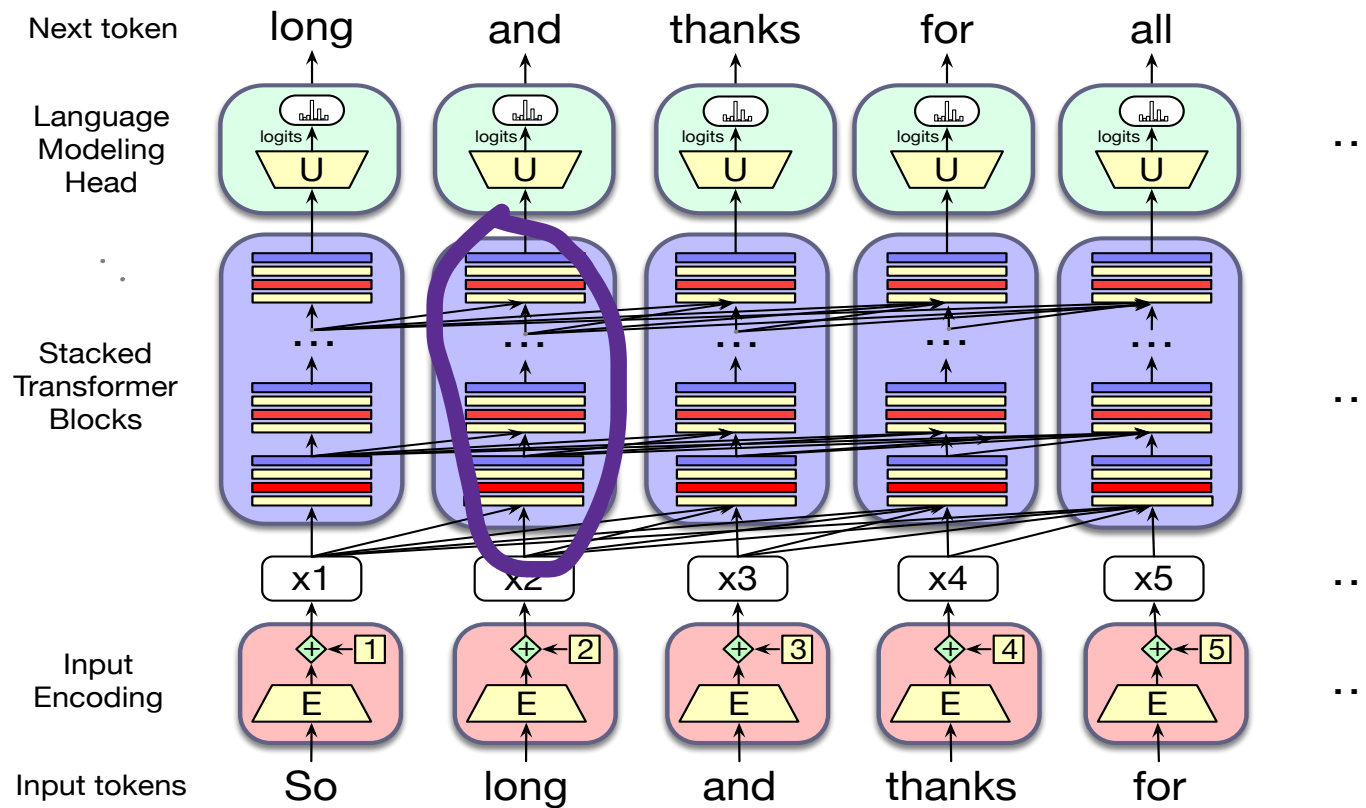
$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}c}; \quad \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{K}c}; \quad \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{V}c}; \quad \forall c \quad 1 \leq c \leq h$$

- Intuizione: ogni testa potrebbe **prestare attenzione al contesto per scopi diversi**, cogliendo relazioni linguistiche o schemi differenti presenti nel contesto.

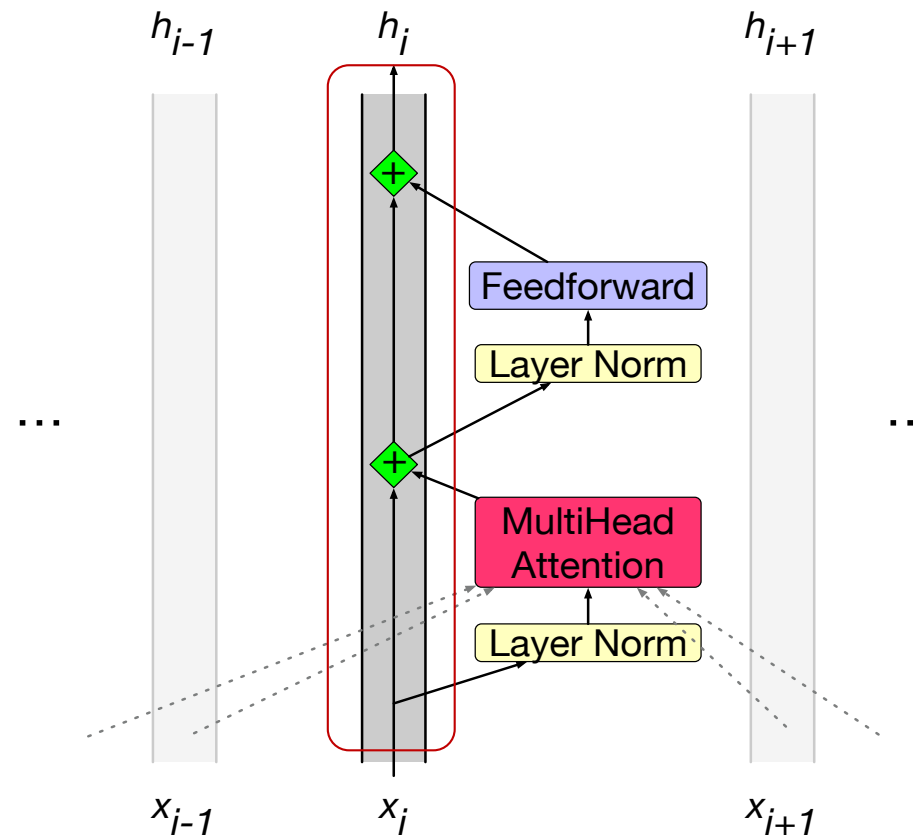
Multi-head self-attention



Ricordiamoci il Transformer LM

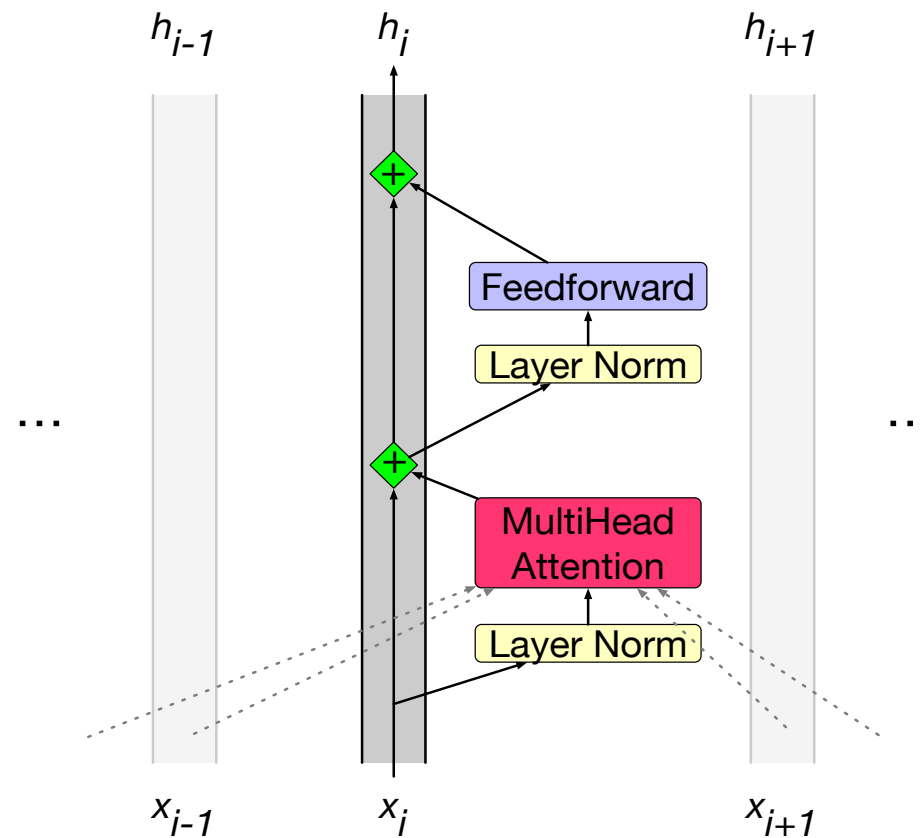


Il “residual stream”: ogni token viene trasmesso ai livelli superiori e modificato

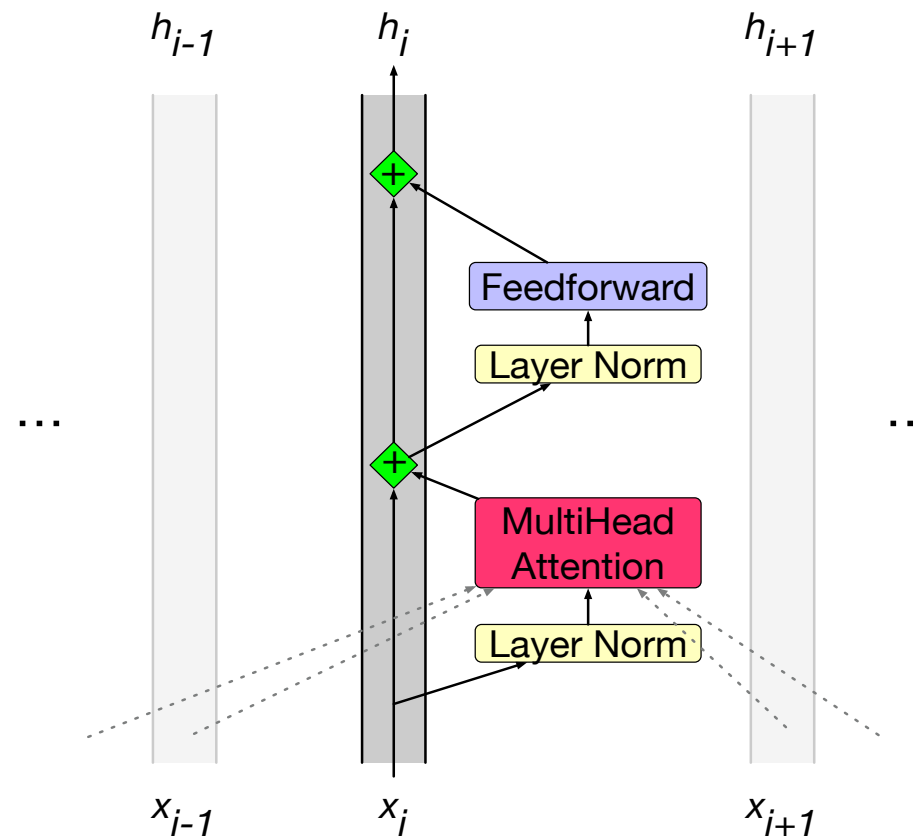


Ci servono non linearità e quindi inseriamo un layer feedforward

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i \mathbf{W}_1 + b_1) \mathbf{W}_2 + b_2$$



Layer normalization: il vettore x_i viene normalizzato due volte



Layer normalization

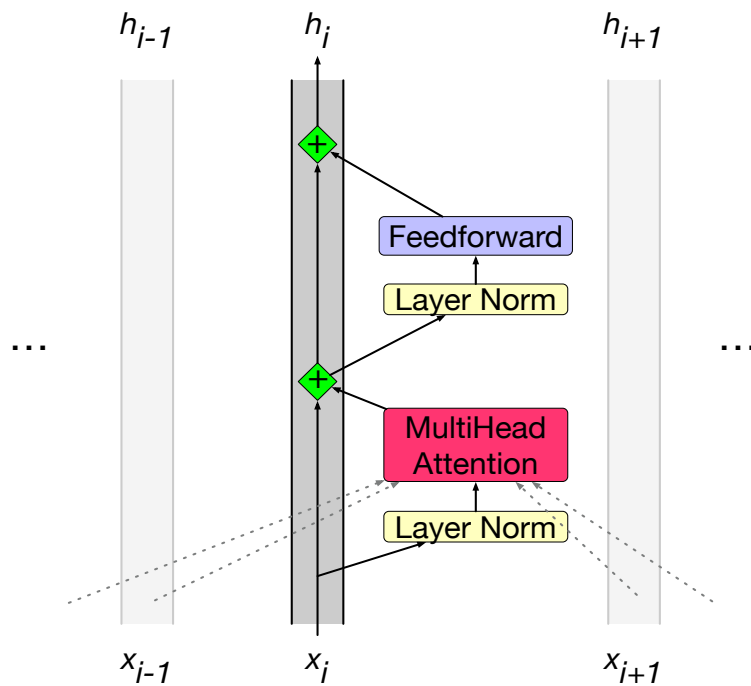
- La layer normalization è una variazione dello z-score usato in statistica, applicata a un singolo vettore in un livello nascosto.

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i \quad \text{Lungo le dimensioni} \quad \hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

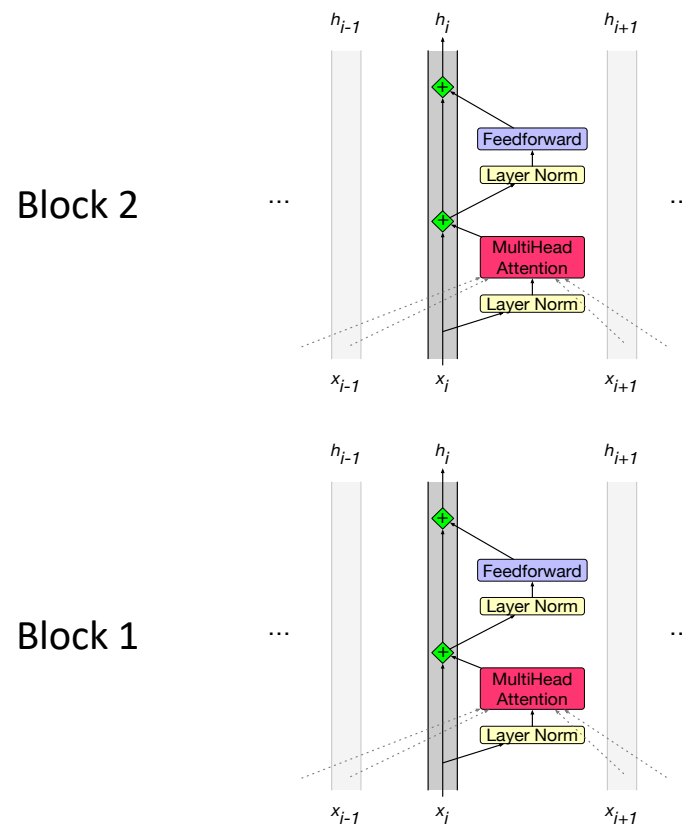
$$\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2} \quad \text{Per ogni vettore di embedding}$$

$$\text{LayerNorm}(\mathbf{x}) = \gamma \frac{(\mathbf{x} - \mu)}{\sigma} + \beta \quad \text{Le operazioni sono elemento per elemento}$$

Combiniamo tutti gli elementi in un singolo blocco del transformer.



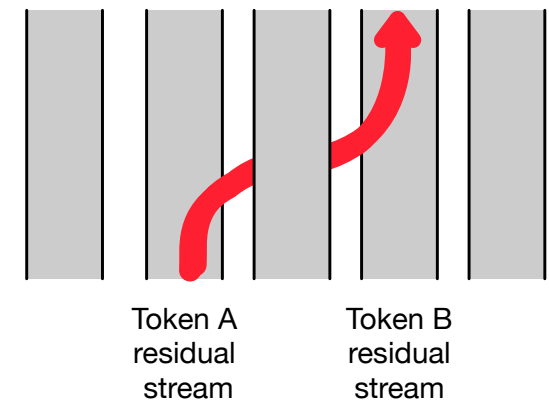
Un transformer è una struttura composta da uno stack di questi blocchi



tutti i vettori hanno la stessa dimensionalità d

Stream residuali e attenzione

- Si noti che tutte le parti del blocco del transformer si applicano a un unico flusso residuo (cioè un token)
- *Ad eccezione del meccanismo di attenzione*, che utilizza informazioni provenienti da altri token.
- Elhage et al. (2021) mostrano che è possibile interpretare le teste di attenzione come elementi che spostano letteralmente informazioni dal flusso residuo di un token vicino verso il flusso corrente.



Parallelizzazione del calcolo usando $\mathbf{X}$

- Nel blocco di attenzione/transformer, finora abbiamo calcolato un singolo output in un singolo passo temporale i all'interno di un unico flusso residuale.
- Ma possiamo “impacchettare” N token della sequenza di ingresso in una singola matrice $\mathbf{X}$ di dimensione $[N \times d]$.
- Ogni riga di $\mathbf{X}$ è l'embedding di un token di input.
- $\mathbf{X}$ può avere da 1K a 32K righe, ognuna pari alla dimensionalità degli embedding d

$$\mathbf{Q} = \mathbf{XW}^Q; \quad \mathbf{K} = \mathbf{XW}^K; \quad \mathbf{V} = \mathbf{XW}^V$$

$$QK^T$$

- Possiamo fare un'unica moltiplicazione di matrice per combinare Q and K^T

N

q1•k1	q1•k2	q1•k3	q1•k4
q2•k1	q2•k2	q2•k3	q2•k4
q3•k1	q3•k2	q3•k3	q3•k4
q4•k1	q4•k2	q4•k3	q4•k4

N



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Parallelizzare l'attenzione

- Scaliamo gli score, eseguiamo la softmax e moltiplichiamo il risultato per $\mathbf{V}$ ottenendo una matrice di dimensione $N \times d$
 - Un vettore attenzione per ogni token di input

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \right) \mathbf{V}$$



Mascheriamo il futuro

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

- A che serve questa funzione di mascheratura?
- $\mathbf{QK}^T$ ha uno score per ogni prodotto query-chiave, *inclusi quelli che seguono la query.*
- Indovinare la prossima parola è veramente semplice se già la conosciamo!

Mascheriamo il futuro

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

- Aggiungiamo $-\infty$ alle celle del triangolo superiore
- La softmax calcolerà 0 in questi punti

N

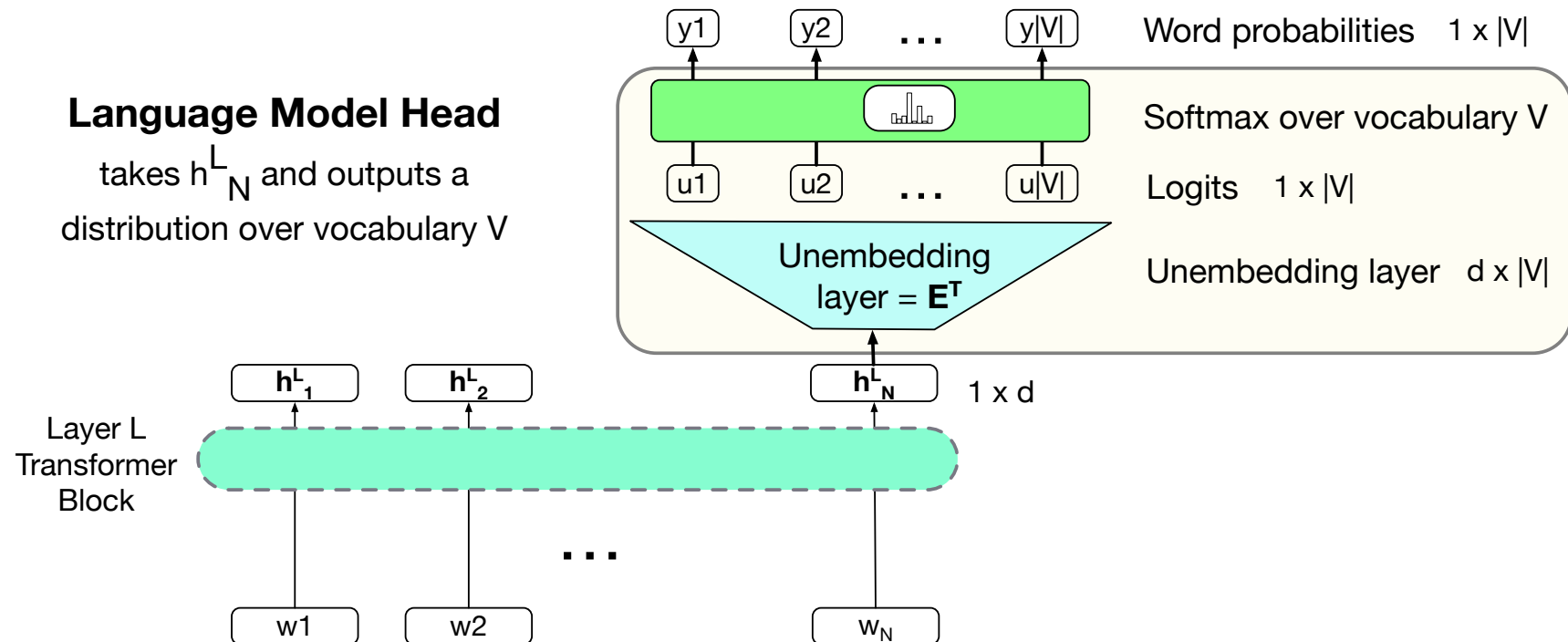
q1•k1	$-\infty$	$-\infty$	$-\infty$
q2•k1	q2•k2	$-\infty$	$-\infty$
q3•k1	q3•k2	q3•k3	$-\infty$
q4•k1	q4•k2	q4•k3	q4•k4

N

Token e Position Embedding

- La matrice $\mathbf{X}$ (di dimensione $[N \times d]$) contiene un embedding per ogni parola nel contesto.
- Questo embedding viene creato **sommando due embedding distinti** per ciascun input:
 - token embedding
 - Embedding posizionali

Testa di LM

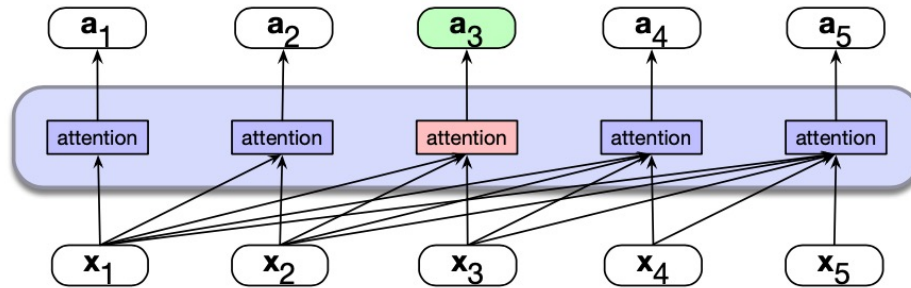


Transformer encoder bidirezionale

- BERT, RoBERTa, SpanBERT, XLM ...

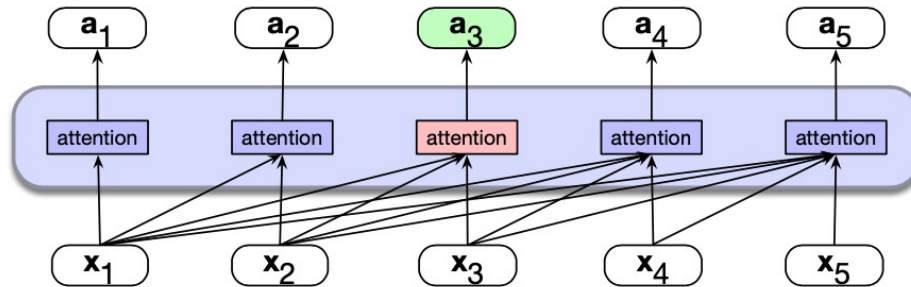
- Sono utilizzati in applicazioni «a valle» (downstream) attraverso il *fine-tuning*
- LM preaddestrati sono usati come ingresso per una rete posta sull'ultimo layer che viene addestrata per il task
- Il processo di pretraining/fine-tuning è un tipo di *transfer learning*
- Gli embedding generati dai Masked Language Model (MLM) sono *contestuali*

Transformer encoder bidirezionale

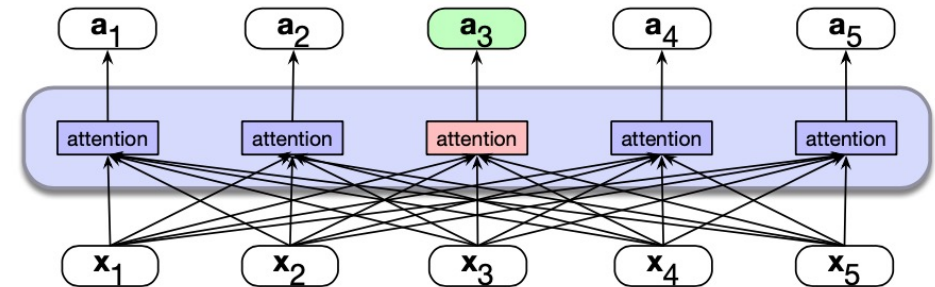


Architettura transformer left-to-right

Transformer encoder bidirezionale



Left-to-right transformer architecture



Arichitettura transformer encoder bidirezionale

Transformer encoder bidirezionale

- Implementazione parallela della self-attention

N

$q1 \cdot k1$	$-\infty$	$-\infty$	$-\infty$
$q2 \cdot k1$	$q2 \cdot k2$	$-\infty$	$-\infty$
$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$-\infty$
$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$

N

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Transformer encoder bidirezionale

- Implementazione parallela della self-attention

N

q1•k1	$-\infty$	$-\infty$	$-\infty$
q2•k1	q2•k2	$-\infty$	$-\infty$
q3•k1	q3•k2	q3•k3	$-\infty$
q4•k1	q4•k2	q4•k3	q4•k4

N

N

q1•k1	q1•k2	q1•k3	q1•k4
q2•k1	q2•k2	q2•k3	q2•k4
q3•k1	q3•k2	q3•k3	q3•k4
q4•k1	q4•k2	q4•k3	q4•k4

N

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \right) \mathbf{V} \quad \mathbf{A} = \text{softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

Bidirectional Encoder Representations from Transformers (BERT)

- Blocchi transformer multi-layer
 - 768 unità in ogni strato nascosto
 - BERT_{BASE}: 12 layer transformer, 12 teste di self-attention per layer, 110M parametri
 - BERT_{LARGE}: 24 layer transformer, 16 teste di self-attention per layer, 304M parametri

Masked Language Modeling

- Il language modeling causale diventa banale se rimuoviamo la maschera causale
- Masked Language Modeling usa un approccio di «riempimento degli spazi vuoti»

○ The *cloze task* Please turn your homework ____ .



Please turn ____ homework in.]



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Masked Language Modeling

- Durante l'addestramento, al modello vengono rimossi uno o più elementi da una sequenza di input.
- Il modello deve quindi generare una distribuzione di probabilità sul vocabolario per ciascuno degli elementi mancanti.
- Successivamente, si utilizza la cross-entropy loss di ciascuna delle predizioni del modello per guidare il processo di apprendimento.

Masked Language Modeling

- Il masking può essere generalizzato a *qualsiasi metodo che corrompe l'input di addestramento* e poi richiede al modello *di ricostruire l'input originale*.
 - Mascheramenti
 - Sostituzioni
 - Riordinamenti
 - Cancellazioni
 - Inserimenti di elementi estranei nel testo di addestramento

Mascherare le parole

- BERT utilizza corpora testuali non annotati
- Un campione casuale di token viene selezionato per essere utilizzato nel task di addestramento

Mascherare le parole

- Una volta selezionato, un token viene utilizzato in uno dei tre modi seguenti:
 - Viene sostituito con il token unico [MASK] del vocabolario
 - Viene sostituito con un altro token del vocabolario, scelto casualmente in base alle probabilità di unigramma dei token
 - Viene lasciato invariato

Mascherare le parole

- In BERT il 15% dei token viene campionato per l'addestramento
 - 80% è sostituito dal token [MASK]
 - 10% è sostituito da un token casuale
 - 10% viene lasciato invariato
- Tutti i token partecipano al meccanismo di self-attention, ma solo quelli selezionati vengono utilizzati per calcolare la loss tramite la softmax.

Mascherare le parole

$$L_{MLM}(x_i) = -\log P(x_i | \mathbf{h}_i^L)$$

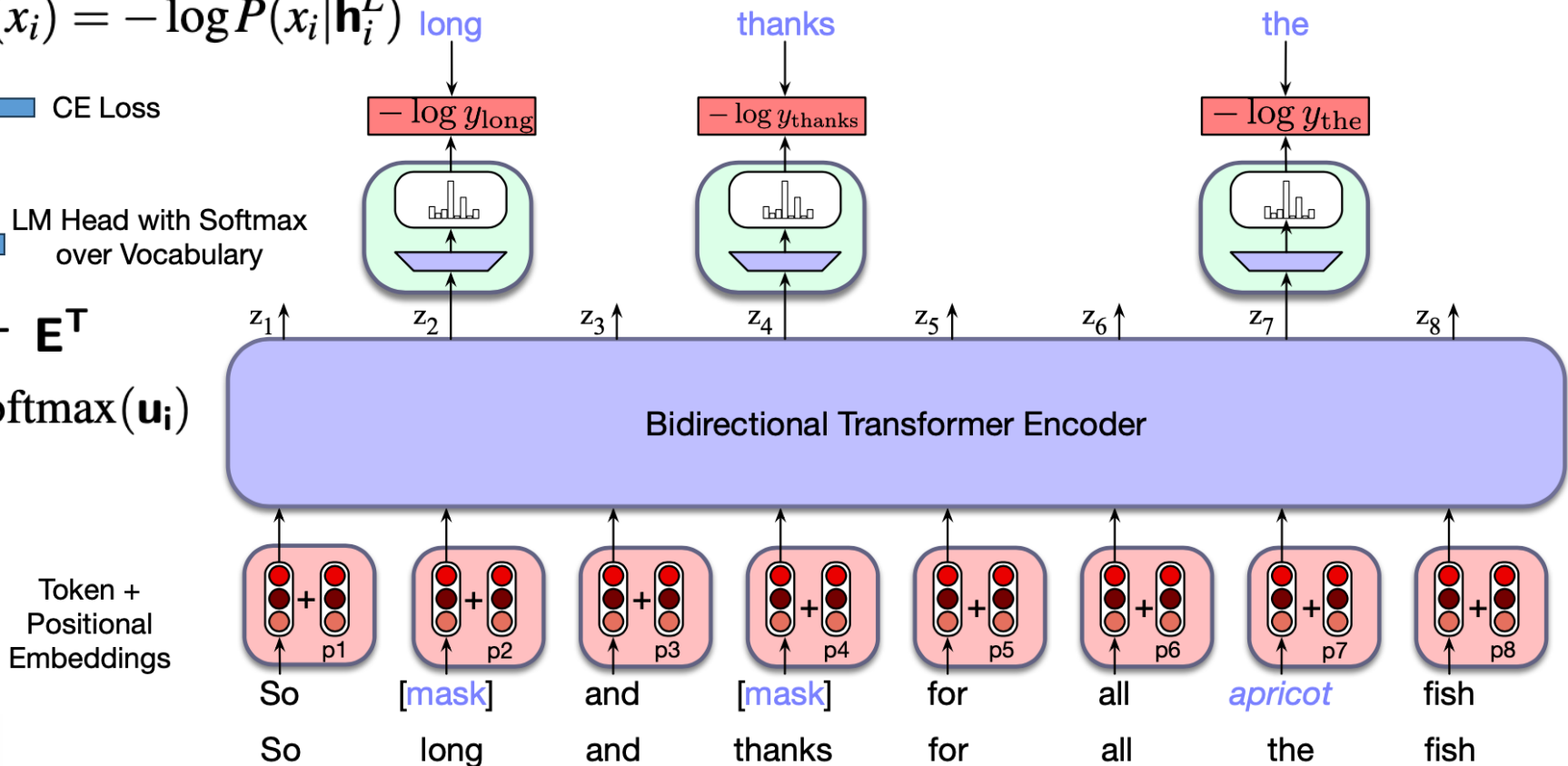
CE Loss

LM Head with Softmax
over Vocabulary

$$\mathbf{u}_i = \mathbf{h}_i^L \mathbf{E}^T$$

$$\mathbf{y}_i = \text{softmax}(\mathbf{u}_i)$$

$$L_{MLM} = -\frac{1}{|M|} \sum_{i \in M} \log P(x_i | \mathbf{h}_i^L)$$



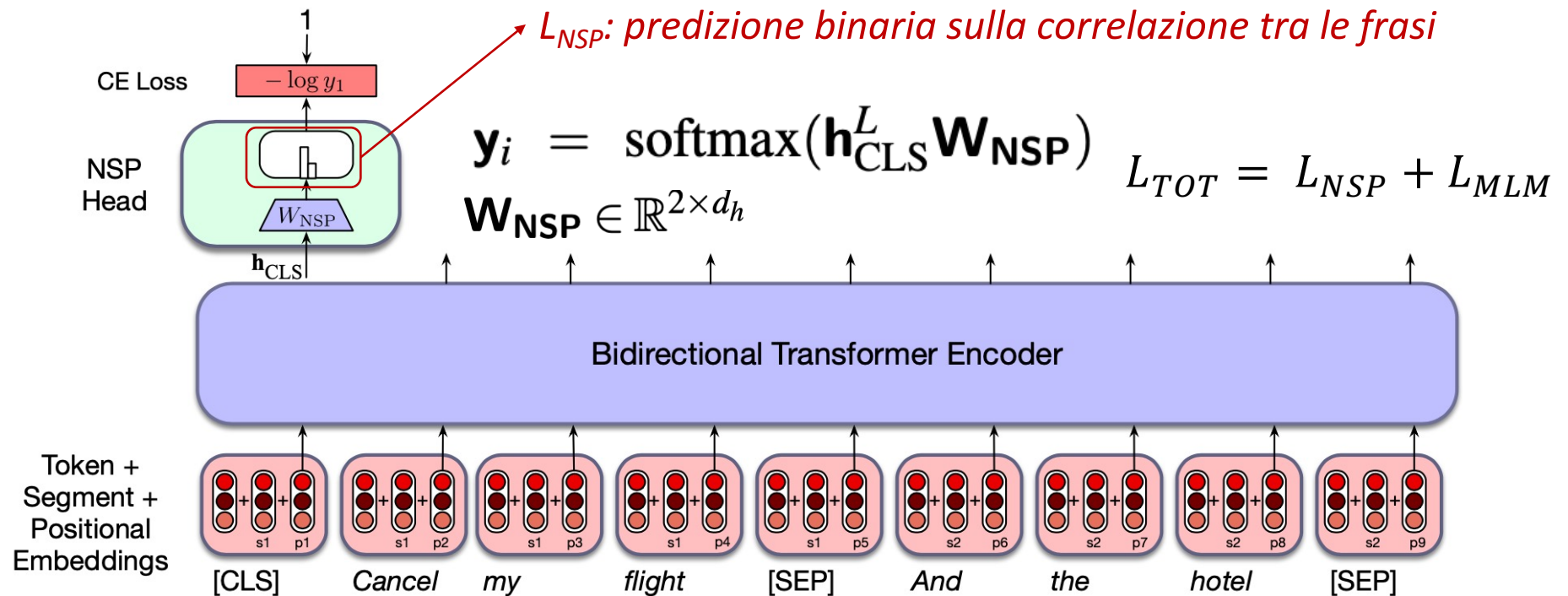
Next sentence prediction

- Molti task di NLP includono la determinazione della relazione tra coppie di frasi
 - Individuazione di parafrasi (paraphrase detection)
 - Textual entailment (stabilire se il significato di una frase implica o contraddice quello dell'altra)
 - Coerenza del discorso tra due frasi adiacenti (discourse coherence)
 - ...

Next sentence prediction

- BERT esegue la predizione della frase successiva (Next Sentence Prediction - NSP) usando le seguenti procedure di addestramento:
 - Split 50% - 50% tra coppie positive di frasi (correlate) e negative (non correlate)
 - Vengono introdotti due nuovi token:
 - [CLS] inserito all'inizio della coppia di frasi
 - [SEP] inserito nel mezzo tra le due frasi e dopo il token finale

Next sentence prediction



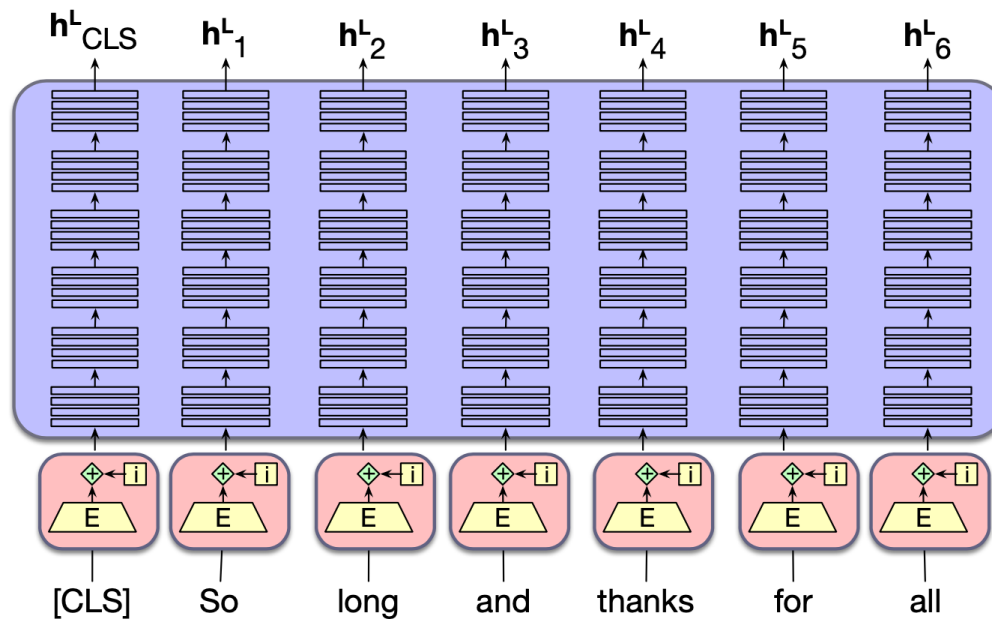
Addestramento di BERT

- Addestrato su
 - BookCorpus (800M parole)
 - Wikipedia inglese (2500M parole)
- Addestramento della NSP usando lo schema 50% - 50%
 - Le coppie di frasi sono state campionate per entrare nella dimensione di input di 512 token
- 40 epoche di addestramento usando $L_{TOT} = L_{MLM} + L_{NSP}$

Embedding contestuali

- L'uscita fornita dal layer finale di un LM pre-addestrato per ogni token x_i di una nuova frase può essere considerato un *embedding contestuale*
 - **Embedding contestuali:** vettori che rappresentano alcuni aspetti del significato di un'istanza di una parola nel proprio contesto
 - Spesso il contextual embedding si ottiene facendo la media, token per token, dei corrispondenti stati nascosti negli ultimi quattro layer del modello

Embedding contestuali



- Embedding statici $\rightarrow$ significato dei tipi delle parole
- Embedding contestuali $\rightarrow$ significato delle istanze delle parole

ContextualEmbedding(«long») $\rightarrow h^L_2$

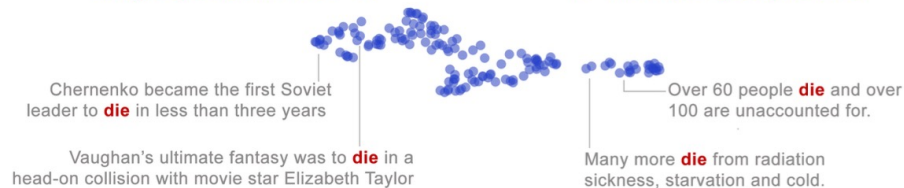
Embedding contestuali e senso delle parole

- Ricordiamo che le parole sono *ambigue*

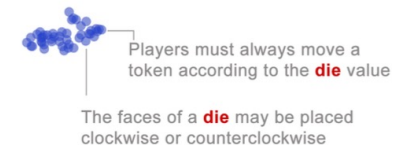
German article “die”



single person dies ← → multiple people die



a playing die



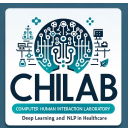
Varianti di BERT



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Multilingual BERT

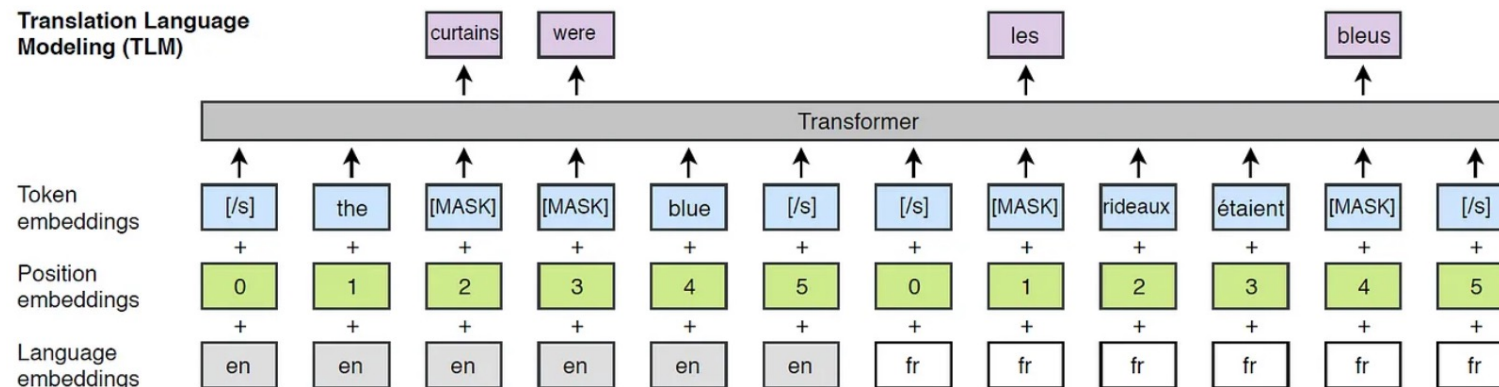
- Addestrato su 104 lingue contemporaneamente per costruire una rappresentazione condivisa del significato di una parola in ognuna di esse
- Vocabolario di 110K token condivisi per WordPiece
- Non il migliore nei task cross-linguistici

RoBERTa

- Sviluppato da Facebook (ora Meta)
- Addestrato su cinque corpora inglesi (160GB di testo non compresso)
- Addestramento efficace utilizzando *dynamic masking*
 - Il mascheramento dei token cambia ad ogni sequenza di input
 - BERT esegue un solo masking in fase di preprocessing

XLM (cross lingual Language Model)

- Sviluppato da Facebook (pre-training su 64 GPUs)
- Tecnica del *Translation Language Modelling* (TLM)



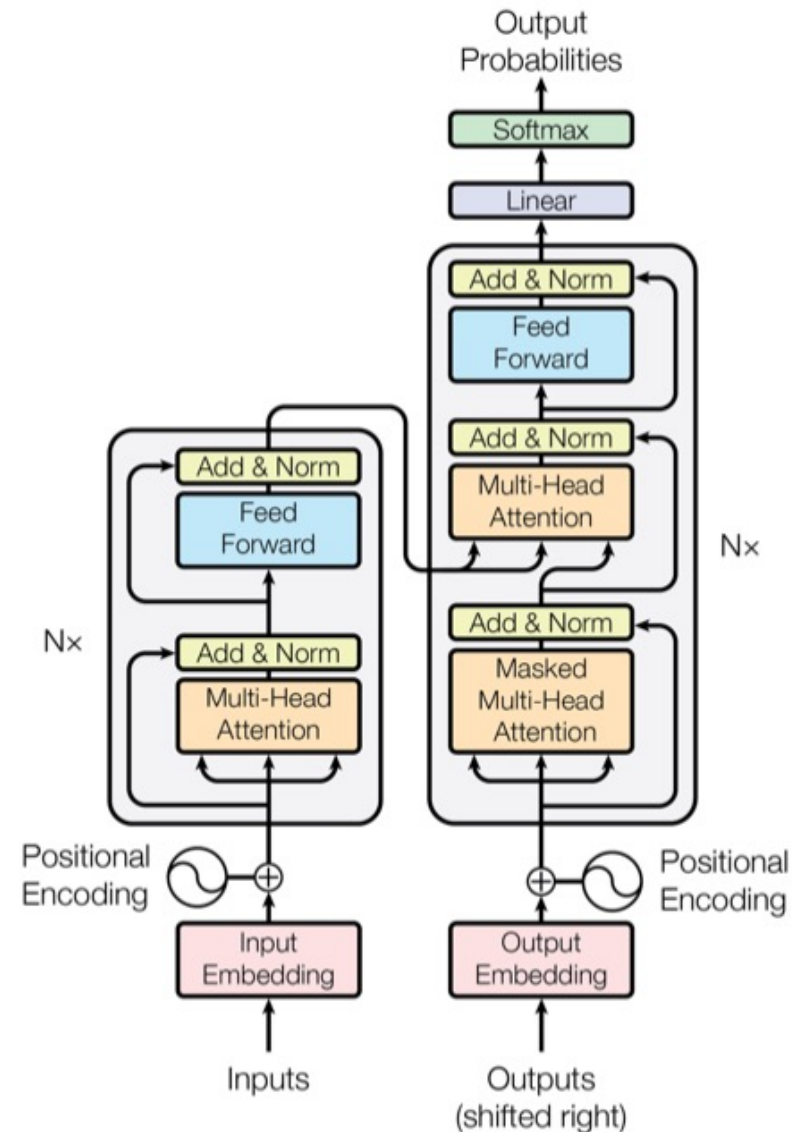
Lample, G., & Conneau, A. (2019). Cross-lingual language model pretraining. *arXiv preprint arXiv:1901.07291*.

XLM-RoBERTa

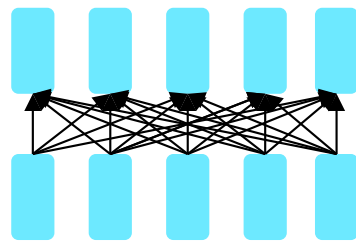
- Sviluppato da Facebook
- Ancora più potere computazionale (pre-training su 500 GPUs) e ancora più dati
- Non utilizza i language embedding per abilitare un veloce switch da una lingua all'altra

Large Language Models

- L'architettura di base è sempre la stessa, il transformer è composto da un encoder e un decoder
- A seconda dei modelli, cambia come le componenti encoder-decoder interagiscono fra di loro e il metodo di attenzione utilizzato

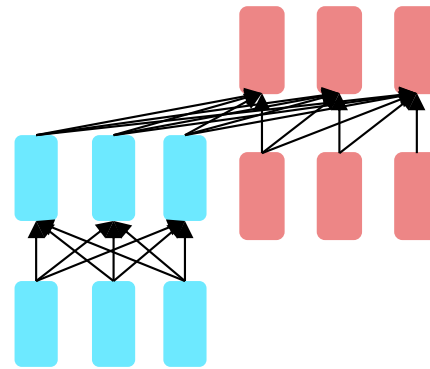


Large Language Models



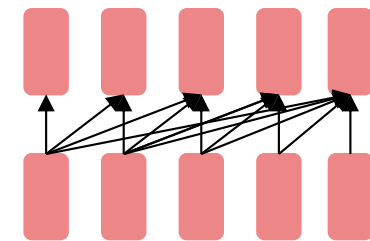
Encoders

BERT family



Encoder-decoders

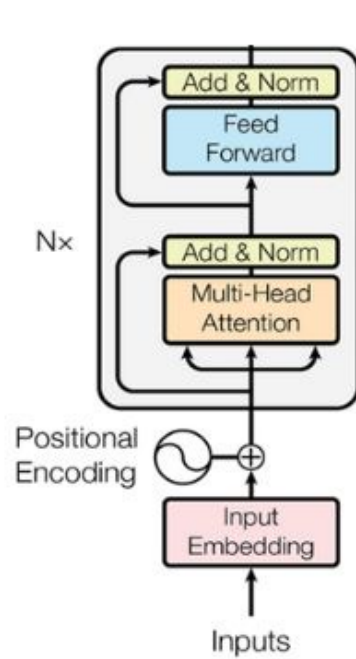
Flan-T5, Whisper



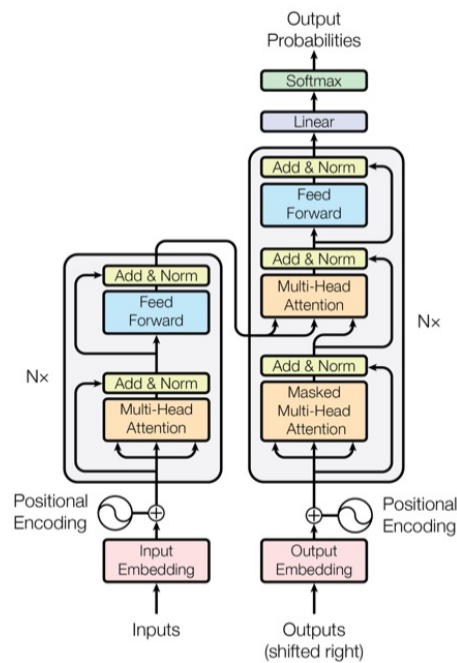
Decoders

GPT, Claude,
Llama, Mixtral

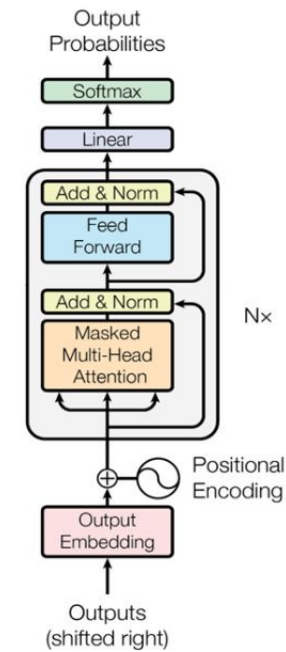
Large Language Models



Encoder-only

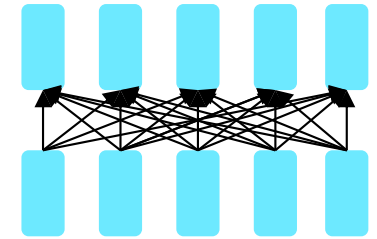


Encoder-decoder



Decoder-only

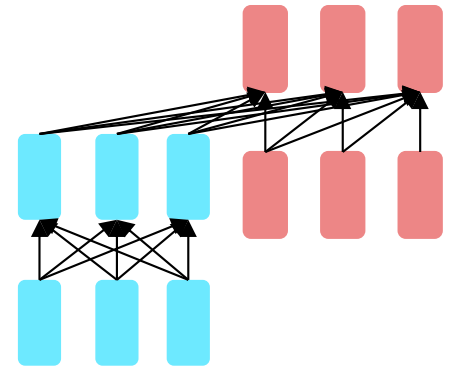
Encoder-only



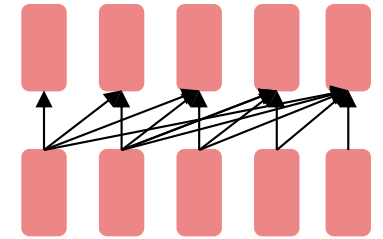
- Molte varietà!
 - Masked Language Models (MLMs)
 - Modelli BERT-like
- Addestrati a partire da una parola target e quelle nel suo intorno destro e sinistro
- Sono solitamente finetuned (addestrati su dati supervised) per compiti di classificazione

Encoder-Decoders

- Addestrati a mappare da una sequenza a un'altra
- Molto popolari per:
 - machine translation (mappare da una lingua all'altra)
 - speech recognition (mappare dall'acustica alle parole)



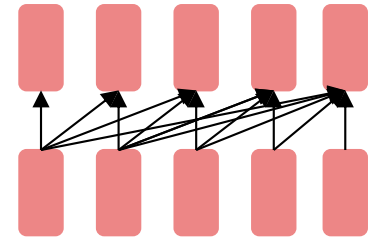
Decoder-only



- Cosa possono fare questi modelli rispetto ai precedenti?
- Tutti i task precedenti possono essere riformulati in maniera generativa
 - Task di tipo next-word prediction

Decoder-only

- Chiamati anche
 - Causal LLMs
 - Autoregressive LLMs
 - Left-to-right LLMs
- Predicono le parole da sinistra a destra



Decoder-only – Autoregressione

- In fase di training il token corretto viene iniettato nel modello a prescindere dalla predizione effettuata dallo stesso (teacher forcing)
- In fase di inferenza lo stesso token generato è ridato in input al modello
- Se l'output dipende dai token precedenti, posso specificare un'istruzione da dare al modello così da generare il testo desiderato

Decoder-only – Autoregressione

- L'algoritmo di autoregressione è utilizzato in fase di inferenza, per condizionare la generazione dell'output con l'output precedentemente generato

Algorithm 1 Overview of the autoregressive algorithm

$i \leftarrow 1$

$w_i \sim P(w)$

while $w_i \neq \text{EOS}$ **do**

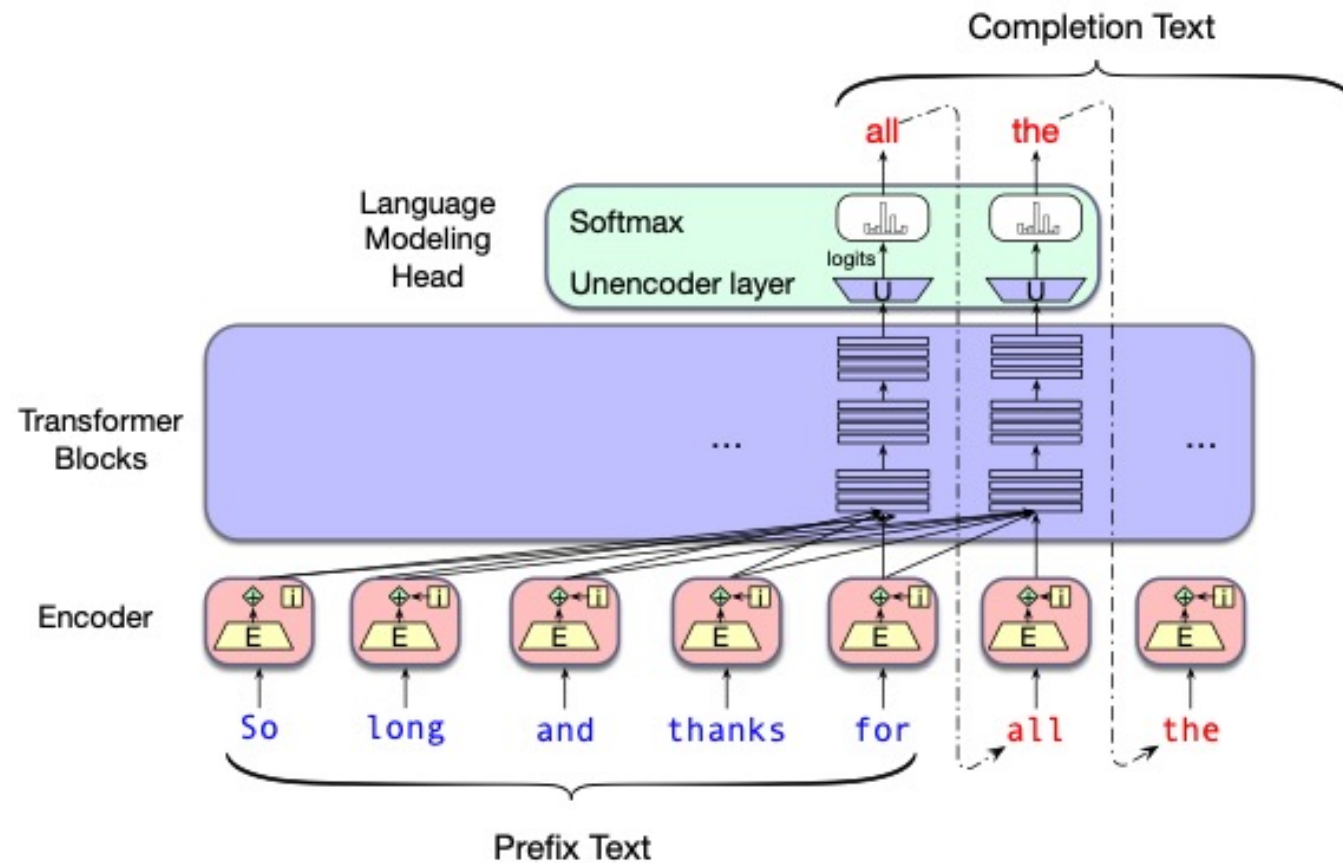
$i \leftarrow i + 1$

$w_i \sim P(w_i | w_{<i})$

end while

$P(w)$ è la *strategia* con cui vengono *campionati* e quindi generati i token

Decoder-only – Autoregression



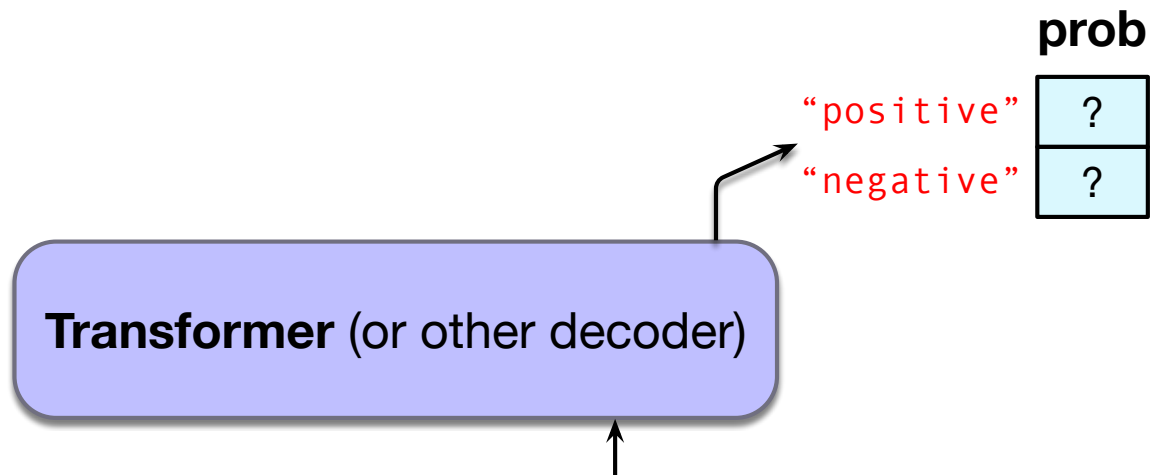
Decoder-only – Autoregressione

- Se l'output dipende dai token precedenti, posso specificare un *prompt* (istruzione) da dare al modello così da generare l'output desiderato.

Decoder-only – Autoregressione

- In quest'ottica i modelli generativi possono svolgere una serie di compiti tradizionali dei modelli encoder-only (*classificazione*) e encoder-decoder (*traduzione*) ma anche nuovi (*generazione*)

Sentiment analysis



The sentiment of the sentence "I like Jackie Chan" is:



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Sentiment analysis

- Diamo al language model questa stringa:

The sentiment of the sentence "I like Jackie Chan" is:

- E vediamo quale parola pensa che venga dopo:

$P(\text{positive} | \text{The sentiment of the sentence "I like Jackie Chan" is:})$

$P(\text{negative} | \text{The sentiment of the sentence "I like Jackie Chan" is:})$

- Il modello genererà l'etichetta più probabile, che completa meglio il prompt, fornendo così la predizione

Question-Answering

QA: “Who wrote The Origin of Species”

- Diamo al language model questa stringa:

Q: Who wrote the book ‘‘The Origin of Species’’? A:

- E vediamo quale parola pensa che venga dopo:

$P(w|Q: \text{Who wrote the book ‘‘The Origin of Species’’? A:})$

- E iteriamo:

$P(w|Q: \text{Who wrote the book ‘‘The Origin of Species’’? A: Charles})$

Summarization

Testo originale

The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For \$89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says.

But not if you live in New England or surrounding states. “We will not ship snow to any states in the northeast!” says Waring’s website, ShipSnowYo.com. “We’re in the business of expunging snow!”

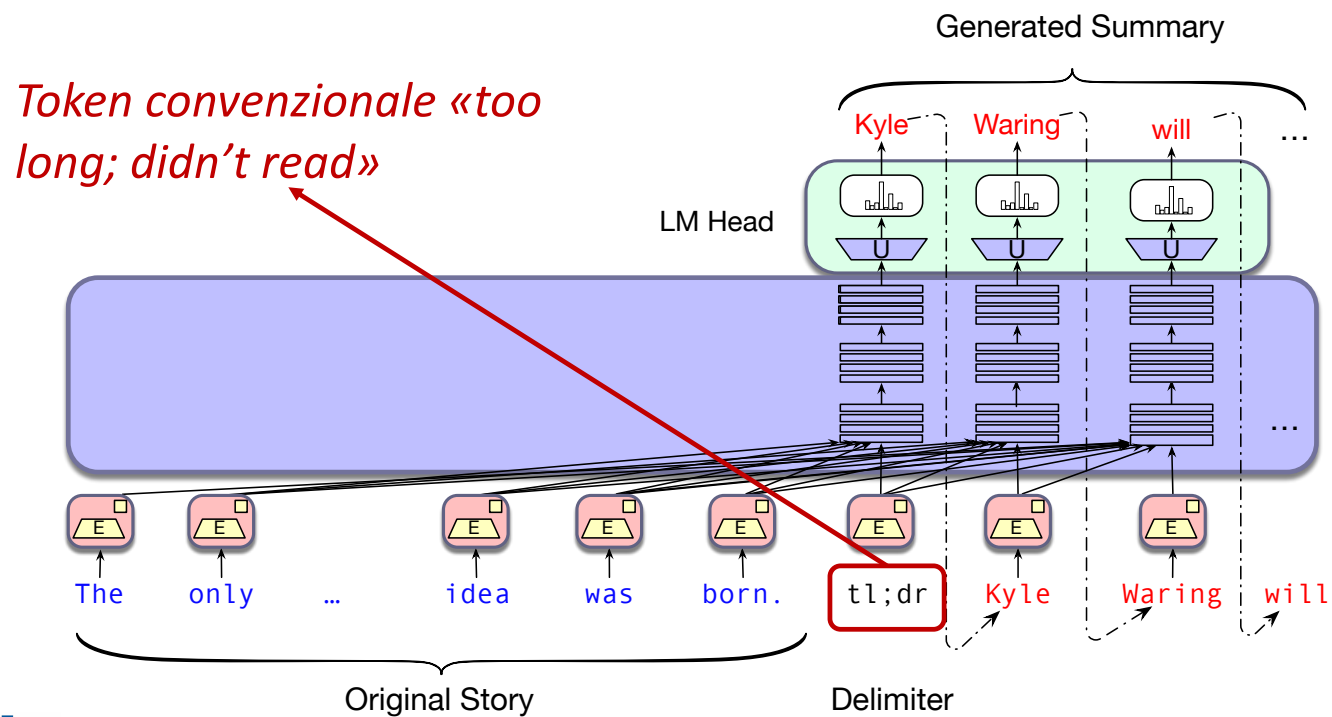
His website and social media accounts claim to have filled more than 133 orders for snow – more than 30 on Tuesday alone, his busiest day yet. With more than 45 total inches, Boston has set a record this winter for the snowiest month in its history. Most residents see the huge piles of snow choking their yards and sidewalks as a nuisance, but Waring saw an opportunity.

According to Boston.com, it all started a few weeks ago, when Waring and his wife were shoveling deep snow from their yard in Manchester-by-the-Sea, a coastal suburb north of Boston. He joked about shipping the stuff to friends and family in warmer states, and an idea was born. [...]

Riassunto

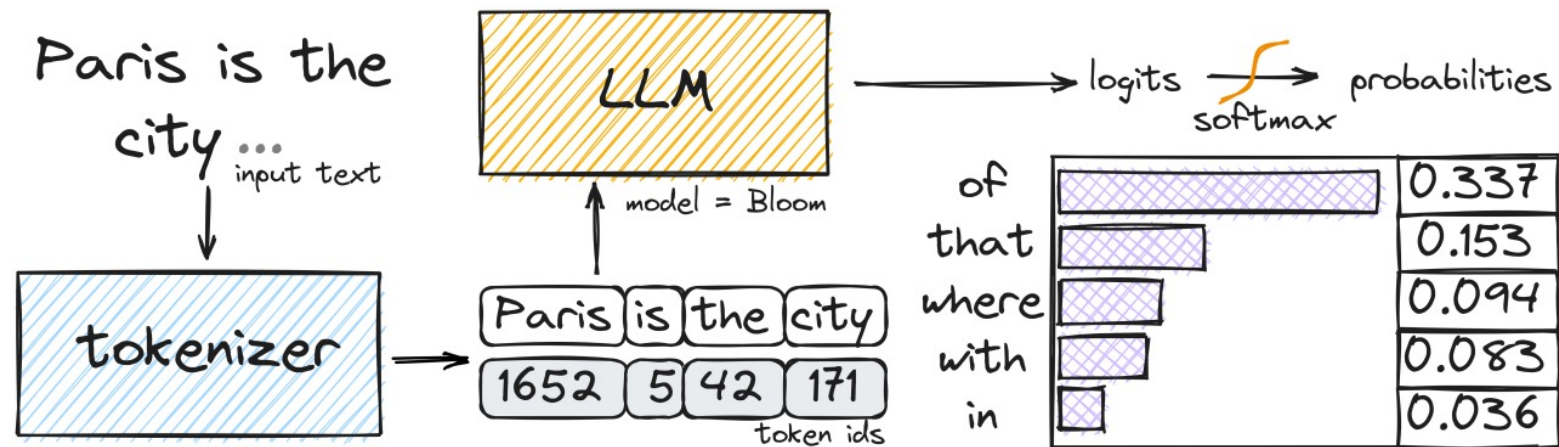
Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

Summarization



Decoding

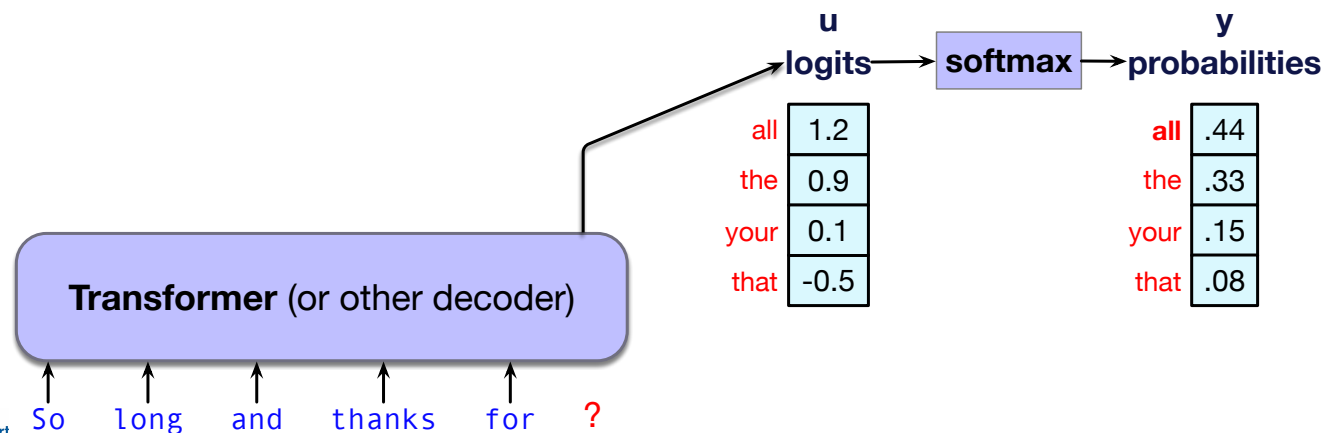
Testo → tokens → LLM → token predetto → testo predetto



Come seleziono i token che formano il testo in output?

Decoding

- I layer neurali degli LLM generano i logit per ogni token nel vocabolario.
- Il vettore di punteggio $\mathbf{u}$ con shape $[1 \times |V|]$ viene trasformato in un vettore di probabilità $\mathbf{y}$ tramite softmax



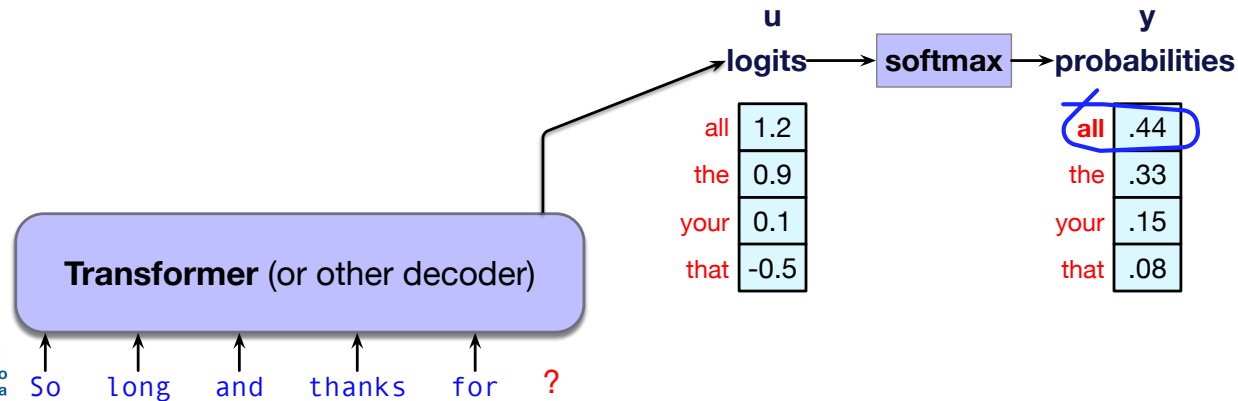
Decoding and Sampling

- Il decoding è la scelta della parola da generare basandosi sulle su in criterio in base alle probabilità fornite in output dal modello
- Decoding deterministico
 - Greedy search, beam search
- Decoding stocastico (sampling)
 - Random sampling, Top-k sampling, Top-p sampling (nucleus sampling), Temperature sampling, contrastive search

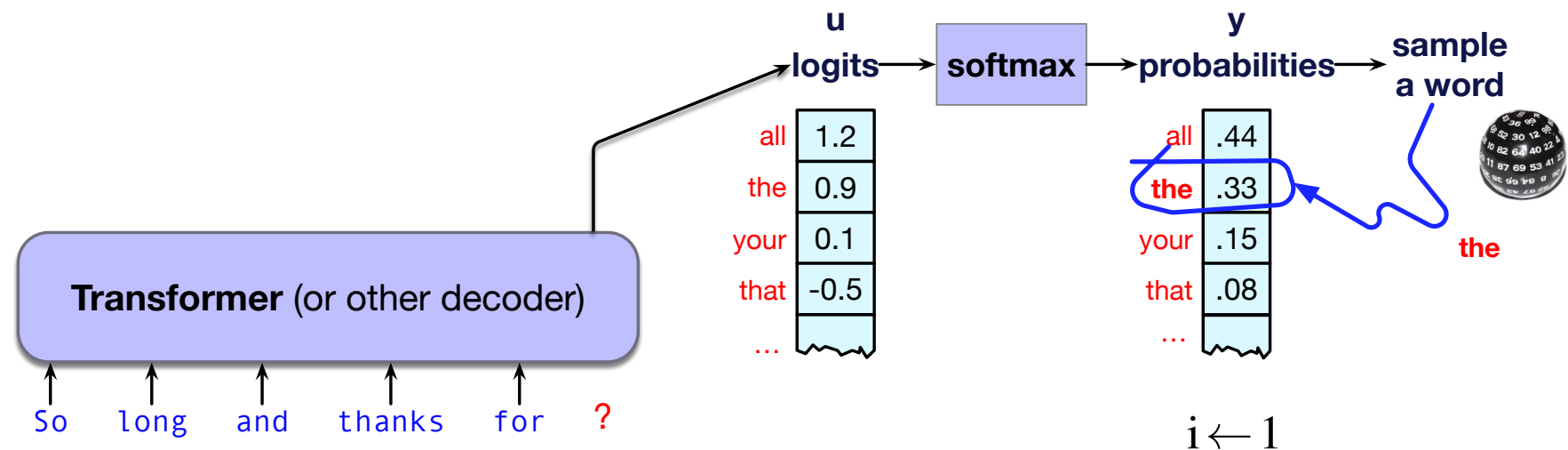
Greedy Search

- La Greedy search è il più semplice metodo di decoding
- Seleziona la parola con maggiore probabilità di essere selezionata come parola successiva

$$w_t = \operatorname{argmax}_w P(w \mid w_{1:t-1})$$



Random sampling



Dopo ogni token campioneremo parole da generare in base alla loro probabilità condizionata dai token precedentemente predetti

$i \leftarrow 1$

$w_i \sim p(w)$

while $w_i \neq \text{EOS}$

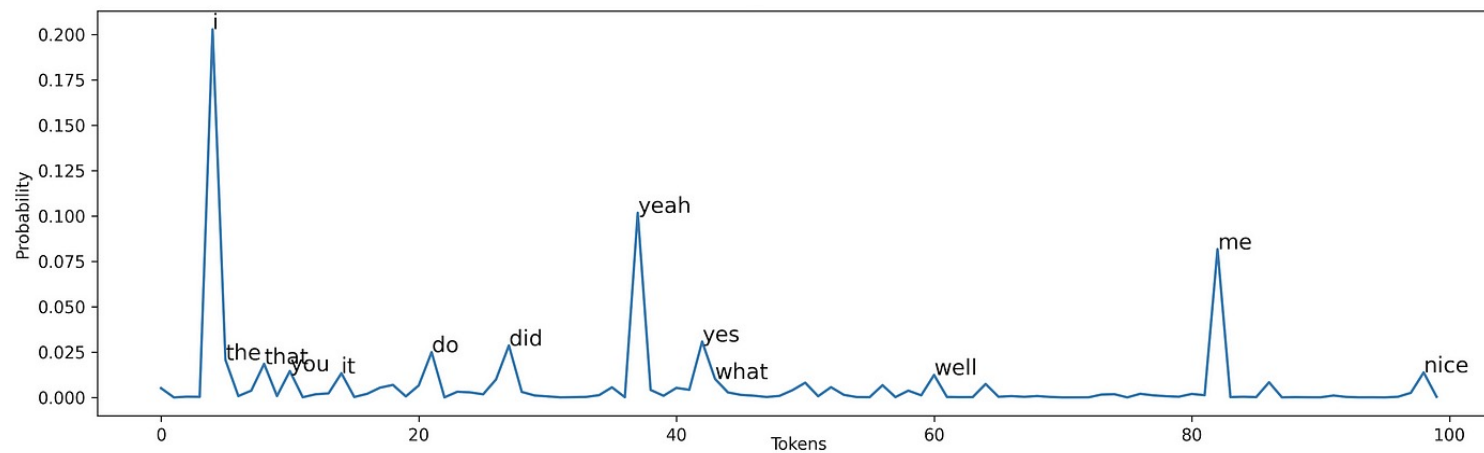
$i \leftarrow i + 1$

$w_i \sim p(w_i \mid w_{<i})$

Random sampling

Considerando il seguente contesto:

I love watching movies



I, yeah e me sono i token più probabili

Random sampling

- Anche se il random sampling genera per lo più parole sensate e ad alta probabilità, ci sono molte parole strane e a bassa probabilità nella distribuzione
- Ognuna ha bassa probabilità, ma sommate costituiscono una grande porzione della distribuzione
- Ma possono essere selezionate e portare alla generazione di frasi strane

Random sampling – qualità e diversità

- Enfatizzare parole ad alta probabilità
 - + quality: più accurate, coerenti e fattuali
 - diversity: noiose, ripetitive
- Enfatizzare parole a media probabilità
 - + diversity: più creative, diverse
 - quality: meno fattuali, incoerenti



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa

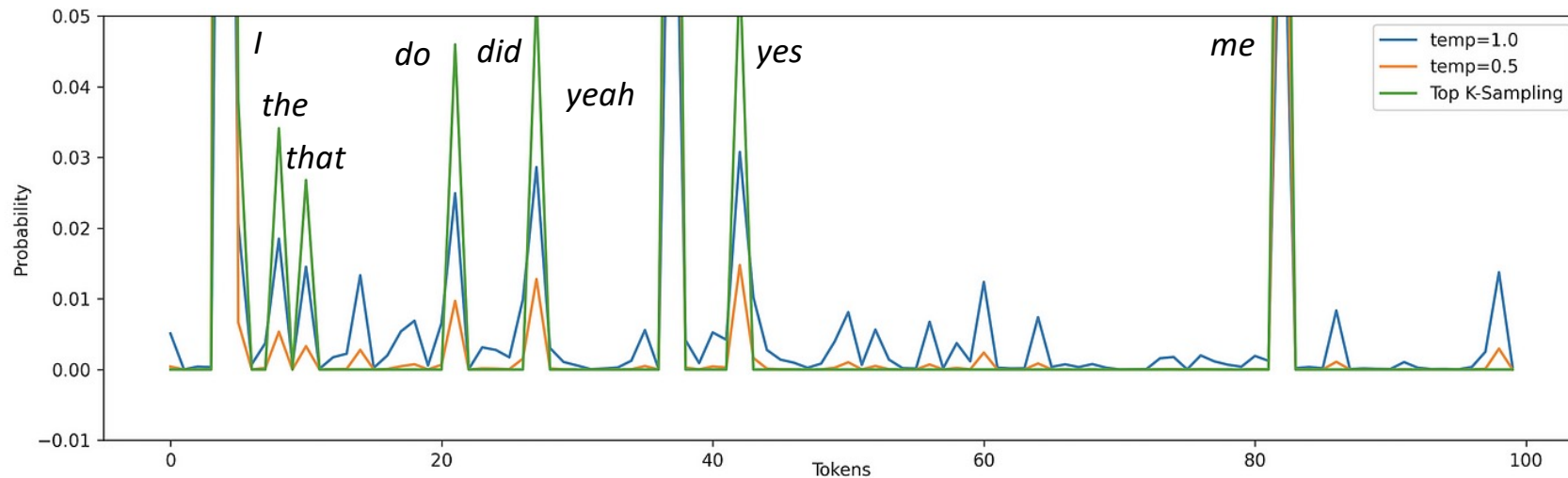


Top- k sampling

1. Scegli il numero di parole k
2. Per ogni parola nel vocabolario V , usa il LLM per calcolare la verosimiglianza (likelihood) di questa parola dato il contesto $p(w_t | \mathbf{w}_{<t})$
3. Ordina le parole per verosimiglianza, mantieni solo le k parole più probabili
4. Rinormalizza i punteggi delle k parole per ottenere una distribuzione di probabilità legittima
5. Campiona casualmente una parola all'interno di queste k parole più probabili rimanenti in base alla sua probabilità

Top- k sampling

- Nel nostro esempio in corso abbiamo campionato con $k=8$
- Confrontiamo con la linea blu (random sampling)



Top- p sampling (Holtzman et al., 2020)

- Detto anche nucleus sampling
- Nel caso di top-k sampling, il valore k è fisso quindi può coprire quantità molto diverse di massa di probabilità in situazioni diverse

Top- p sampling (Holtzman et al., 2020)

💡 Nel top- p sampling viene mantenuta fissa la massa di probabilità del top p percento

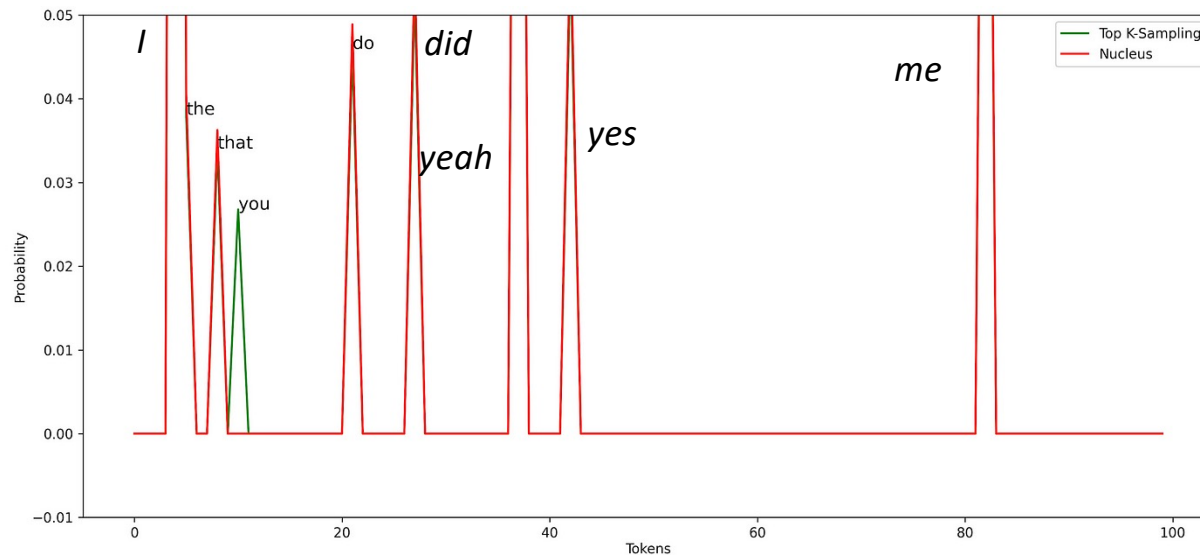
- Data una distribuzione $P(w_t | \mathbf{w}_{<t})$, il vocabolario top- p $V(p)$ è l'insieme più piccolo di parole tale che

$$\sum_{w \in V(p)} P(w | \mathbf{w}_{<t}) \geq p$$

*Implementa una sorta di
selezione adattiva del valore di k*

Top- p sampling (Holtzman et al., 2020)

- Nel nostro esempio in corso, «you» non è stato campionato a causa del valore della soglia p



Temperature sampling

- Rimodella la distribuzione invece di troncarla secondo un parametro di *temperatura* τ
- Intuizione dalla termodinamica
 - un sistema ad alta temperature è flessibile e può esplorare molti possibili stati
 - un sistema a bassa temperature è probabile che esplori un sottoinsieme di stati ad energia inferiore (migliore)

Temperature sampling

- Nel *low temperature sampling*, ($\tau \leq 1$)
 - aumentiamo in modo uniforme la probabilità delle parole più probabili
 - diminuiamo la probabilità delle parole rare

Temperature sampling

- Si divide il logit per il parametro di temperatura τ prima di passarlo attraverso la softmax

Invece di ~~$\mathbf{y} = \text{softmax}(u)$~~

calcolo $\mathbf{y} = \text{softmax}(u/\tau)$

Temperature sampling

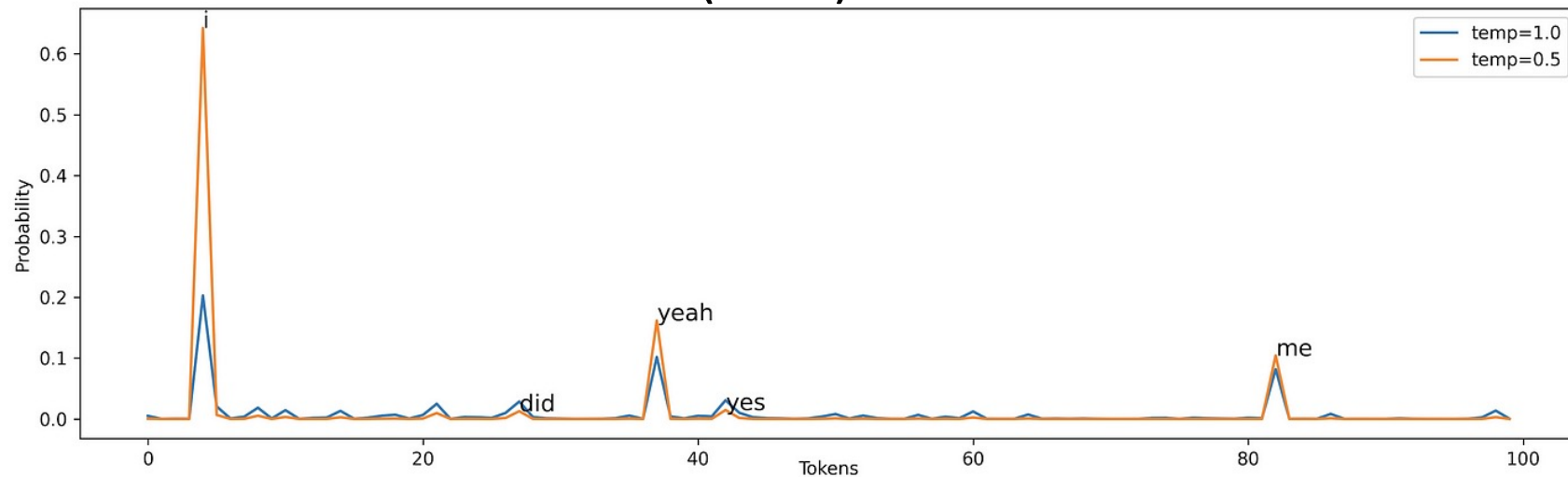
- Perché funziona?
 - Quando τ è vicino a 1, la distribuzione non cambia molto
 - Più basso è τ , più grandi sono i punteggi passati alla softmax
 - La softmax spinge i valori alti verso 1 e i valori bassi verso 0

Temperature sampling

- Perché funziona?
 - Grandi input spingono le parole ad alta probabilità più in alto e le parole a bassa probabilità più in basso, rendendo la distribuzione più greedy
 - Mentre τ si avvicina a 0, la probabilità della parola più probabile si avvicina a 1

Temperature sampling

- Nel nostro esempio, *I*, *yeah*, e *me* hanno una maggiore probabilità di essere selezionati
- Confronta con la linea blu ($\tau = 1$)



Contrastive Search

- Dato il contesto $\mathbf{x}_{<t}$, il token successivo $\mathbf{x}_t$ è generato come

$$x_t = \arg \max_{v \in V^{(k)}} \left\{ (1 - \alpha) \times \underbrace{p_{\theta}(v|\mathbf{x}_{<t})}_{\text{model confidence}} - \alpha \times \underbrace{(\max\{s(h_v, h_{x_j}) : 1 \leq j \leq t-1\})}_{\text{degeneration penalty}} \right\}$$

- $V^{(k)}$ è l'insieme dei top- k token predetti dal modello secondo la sua distribuzione di probabilità $p_{\theta}(v|\mathbf{x}_{<t})$, ovvero la *model confidence* cioè la probabilità che il modello generi v

Contrastive Search

- Dato il contesto $\mathbf{x}_{<t}$, il token successivo $\mathbf{x}_t$ è generato come

$$x_t = \arg \max_{v \in V^{(k)}} \left\{ (1 - \alpha) \times \underbrace{p_{\theta}(v | \mathbf{x}_{<t})}_{\text{model confidence}} - \alpha \times \underbrace{(\max\{s(h_v, h_{x_j}) : 1 \leq j \leq t-1\})}_{\text{degeneration penalty}} \right\}$$

- La *degeneration penalty* misura quanto è discriminativo v , rispetto al contesto precedente.
- La funzione $s(\cdot, \cdot)$ calcola la massima similarità coseno tra gli embedding dei token.

Contrastive Search

- Dato il contesto $\mathbf{x}_{<t}$, il token successivo $\mathbf{x}_t$ è generato come

$$x_t = \arg \max_{v \in V^{(k)}} \left\{ (1 - \alpha) \times \underbrace{p_{\theta}(v|\mathbf{x}_{<t})}_{\text{model confidence}} - \alpha \times \underbrace{(\max\{s(h_v, h_{x_j}) : 1 \leq j \leq t-1\})}_{\text{degeneration penalty}} \right\}$$

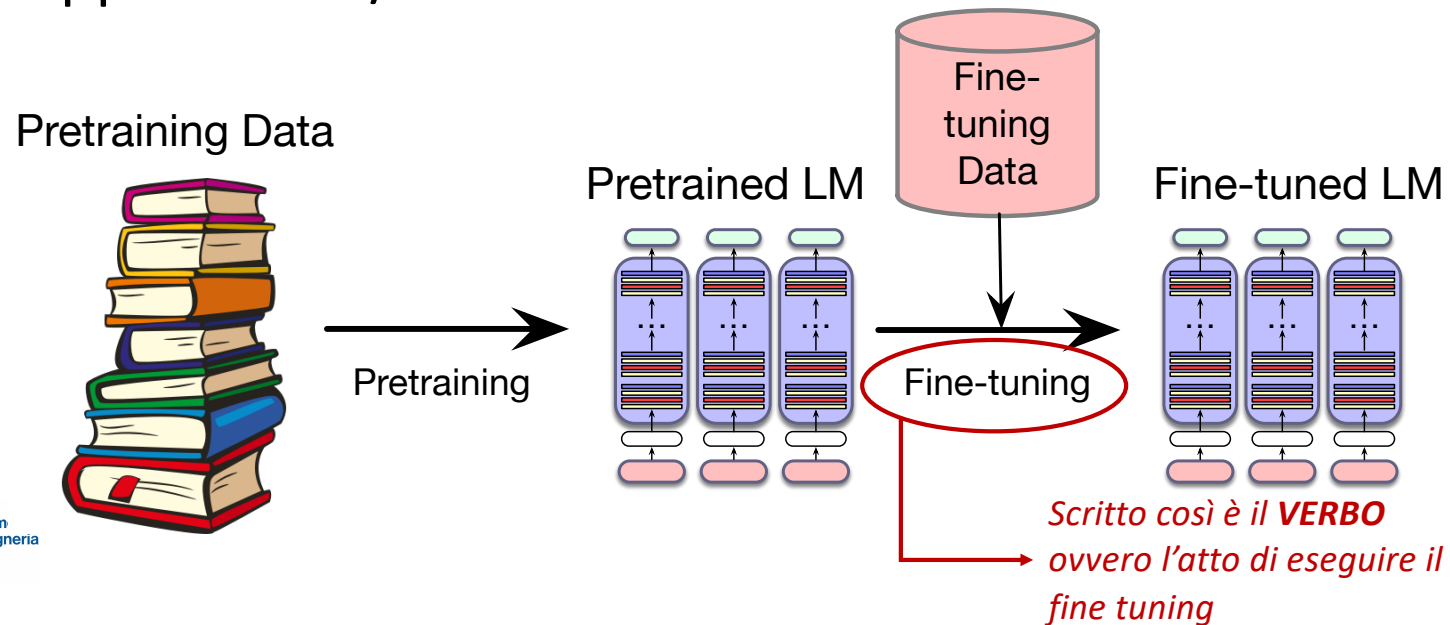
- Maggiore il valore della degeneration penalty, più il token generato è simile al contesto nello spazio degli embedding.
- Questo comporta la possibilità di riscontrare un peggioramento nel testo generato dal modello.

Fine tuning

- Cosa succede se abbiamo bisogno che il nostro LLM funzioni bene in un dominio che non ha visto nel pretraining?
 - Dominio medico, legale, finanziario, dati aziendali
- Oppure, nel caso di un LLM multilingua, è necessario che questo venga addestrato su una lingua che era rara nel pretraining?

Fine tuning

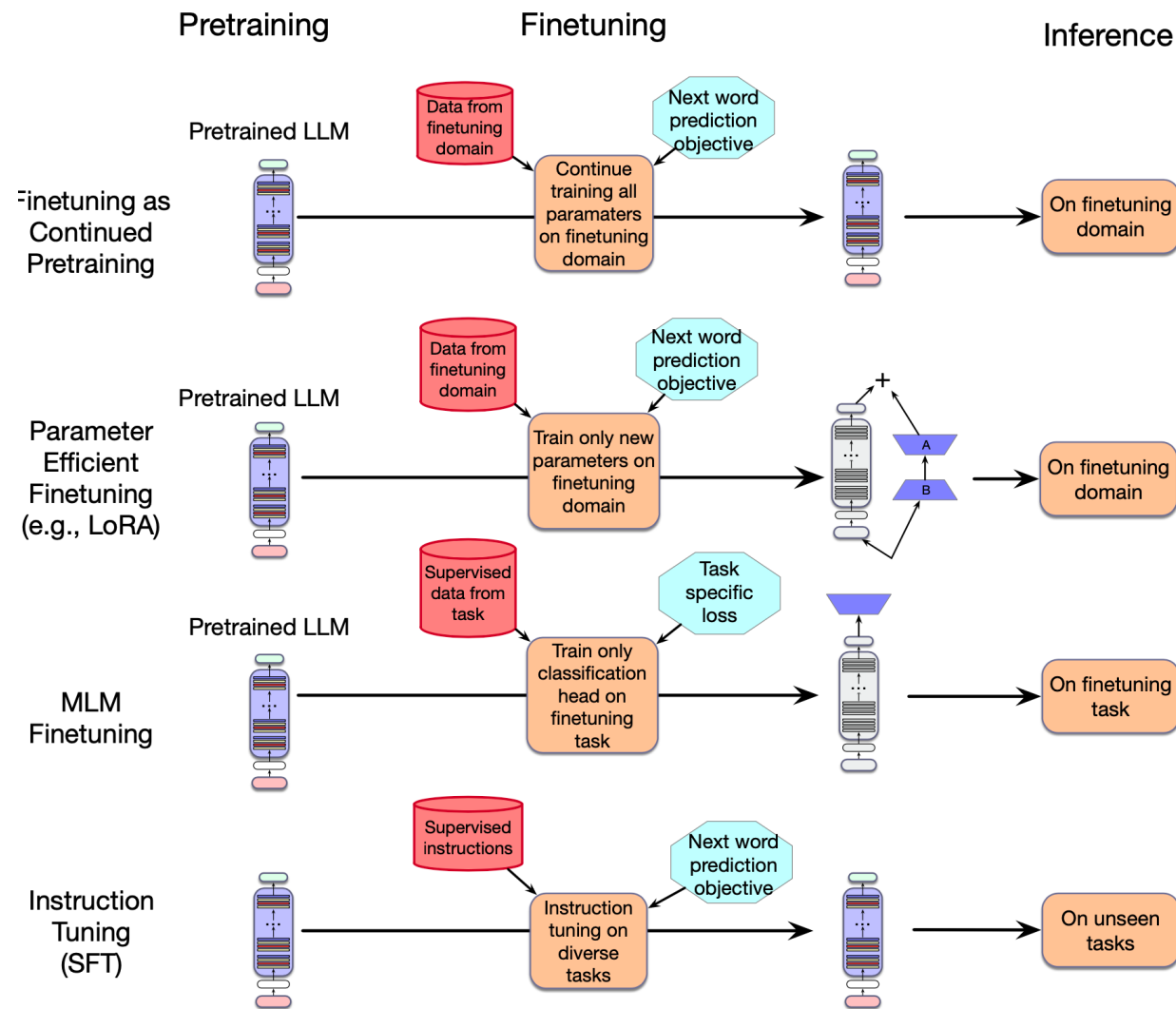
- Il fine tuning è il processo in cui un modello pre-addestrato è sottoposto ad ulteriori *pass* di training su nuovi dati specifici per un'applicazione/dominio.



Fine tuning – tipologie

- Aggiungere circuiti neurali extra (*una testa ulteriore*) dopo lo strato superiore del modello per usarlo come classificatore
- Continued pretraining
- Parameter-efficient fine tuning (PEFT)
- Supervised fine tuning (SFT)

Fine tuning – tipologie



Continued pretraining

- Addestrare ulteriormente *tutti i parametri* del modello su nuovi dati.
- Stessa strategia di addestramento e funzione di loss usati per il pretraining
 - Next-word prediction, Cross-Entropy Loss
- È come se i nuovi dati fossero in coda ai dati di pretraining.

Parameter-Efficient Fine Tuning

- L'adattamento a un nuovo dominio tramite *continued pretraining* (fine tuning) è un problema con i grandi LLM.
 - Notevole quantità di parametri da addestrare.
 - Per ogni batch, si deve fare un passaggio di gradient descent e fare backpropagation attraverso i layer di tutto il modello.
- Costi notevoli legati alla potenza di calcolo richiesta, memoria e tempo.

Parameter-Efficient Fine Tuning

- Il modello per *la sola inferenza* utilizza circa un numero di GB pari a un po' più di 2 volte il numero di parametri.

LLaMA-2 7B → fino a 16GB

- # parametri in 16-bit (2 bytes) → per 7B parametri servono 14 GB
- Embedding models → 1GB
- Input + output al modello ca 1MB/token (512 token → 1GB)

Parameter-Efficient Fine Tuning

- In fase di fine-tuning, non ho il solo modello in memoria

Finetuning di LLaMA-2 7B (*batch size* = 1)

- Parametri del modello (*float* a 16 *bit*) $\rightarrow 7B \times 2 =$ 14GB
- *Gradienti* (*float* a 16 *bit*) $\rightarrow 7B \times 2 =$ 14GB
- *Stati dell'ottimizzatore* (2× parametri *float* a 32 *bit*) $\rightarrow 7B \times 4 \times 2 =$ 56GB

TOTALE: 84GB

Parameter-Efficient Fine Tuning

- Le tecniche di Parameter-Efficient Fine Tuning (PEFT) consentono di:
 - Selezionare in modo efficiente un sottoinsieme di parametri da aggiornare durante il *fine-tuning*.
 - Congelare alcuni dei parametri.
 - Aggiornare solo alcuni pochi parametri.
- Generalmente si addestrano gli ultimi layer del modello

LoRA (Low-Rank Adaptation)

- I Transformers hanno molti layer di moltiplicazione di matrici dense, come gli strati $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$, $\mathbf{W}^O$ nei layer di attenzione.
- Invece di aggiornare questi layer durante il fine-tuning:
 - Congeliamoli
 - Aggiorniamo una loro approssimazione a basso rango (Low-Rank) con meno parametri.

LoRA (Low-Rank Adaptation)

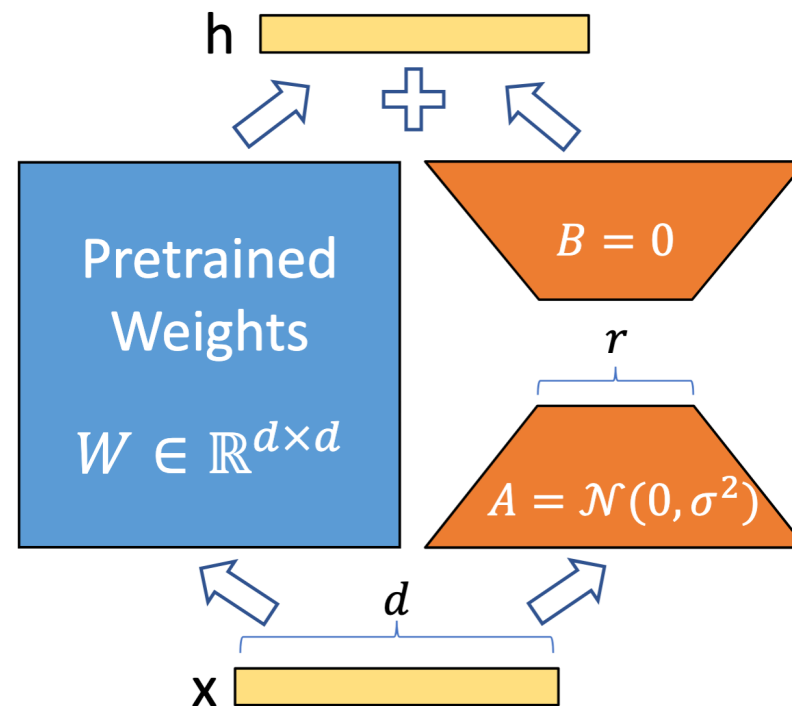
- Considerata una matrice $\mathbf{W}$ $[N \times d]$ che deve essere aggiornata durante il fine-tuning tramite gradient descent.
- Normalmente gli aggiornamenti sono di tipo additivo $\mathbf{W} \rightarrow \mathbf{W} + \Delta\mathbf{W}$, dove $\Delta\mathbf{W}$ $[N \times d]$
- Nel forward pass, dopo l'aggiornamento, calcoliamo $\mathbf{h} = \mathbf{x}\mathbf{W}$

LoRA (Low-Rank Adaptation)

- In LoRA, congeliamo $\mathbf{W}$ e aggiorniamo invece una decomposizione $\mathbf{AB}$ a *basso rango* di $\mathbf{W}$ in cui:
 - $\mathbf{A} [N \times r]$
 - $\mathbf{B} [r \times d]$, con $r \ll \min(N, d)$
- Durante il fine-tuning aggiorniamo $\mathbf{A}$ e $\mathbf{B}$ invece di $\mathbf{W}$.
- Sostituiamo $\mathbf{W} + \Delta\mathbf{W}$ con $\mathbf{W} + \mathbf{AB}$.
- Nel forward pass, invece di $\mathbf{h} = \mathbf{xW}$ calcoliamo:

$$\mathbf{h} = \mathbf{xW} + \mathbf{xAB}$$

LoRA (Low-Rank Adaptation)



Supervised Fine Tuning

- Usato per l'instruction finetuning, in cui vogliamo che un LLM sottoposto a pretraining impari a seguire istruzioni di testo.
 - Foundational LLM → Instruct LLM
- Essendo supervisionato, è necessario avere un data set annotato in cui ci sia siano coppie input-output
 - Question-Answering task

Supervised Fine Tuning

- Creiamo un *data set di prompt* e risposte desiderate
 - domande e loro risposte, o comandi e loro esecuzioni.
- Addestriamo il modello utilizzando la cross-entropy loss per predire iterativamente *ogni token nella risposta desiderata*, cioè per produrre la risposta desiderata dal comando nel prompt.

Prompting

- Prompt: istruzione data in input ad un LLM.
 - La stringa del prompt dell'utente viene passata al LLM, che genera in modo iterativo token condizionati dal prompt.
 - Il prompt crea un contesto che guida gli LLM a generare output utili per raggiungere un obiettivo dell'utente.
- Prompt Engineering: il processo per trovare prompt efficaci per un determinato task

Prompting

- I prompt possono essere usati per una grande varietà di task
- Summarization
- Translation
- (Fine-grained) sentiment analysis
- ...

Template

- Se vogliamo risolvere un task, non vogliamo dover creare un nuovo prompt ogni volta che lo facciamo: possiamo usare i template

Basic Prompt Templates

Summarization	<code>{input} ; tldr;</code>
Translation	<code>{input} ; translate to French:</code>
Sentiment	<code>{input}; Overall, it was</code>
Fine-Grained-Sentiment	<code>{input}; What aspects were important in this review?</code>

Template

- I prompt (e anche i template) devono essere progettati in modo non ambiguo, in modo che qualsiasi continuazione ragionevole raggiunga il task desiderato.

A prompt consisting of a review plus an incomplete statement

Human: Do you think that “input” has negative or positive sentiment?

Choices:

(P) Positive

(N) Negative

Assistant: I believe the best answer is: (



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Template

LLM Outputs for Basic Prompts

Original Review (\$INPUT)	Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life.
Sentiment	Prompt: \$INPUT + In short, our stay was Output: not enjoyable
Fine-grained Sentiment	Prompt: \$INPUT + These aspects were important to the reviewer: Output: 1. Poor service 2. Unpleasant location 3. Noisy and busy area
Summarization	Prompt: \$INPUT + tl;dr Output: I had a bad experience with the hotel's service and the location was loud and busy.
Translation	Prompt: \$INPUT + Translate this to French Output: Je n'ai pas aimé le service qui m'a été offert lorsque je suis entré dans l'hôtel. Je n'ai également pas aimé la zone dans laquelle se trouvait l'hôtel. Trop de bruit et d'événements pour que je me sente détendu et loin de la vie citadine.



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Few-shot prompting

- Migliorare un prompt includendo alcuni esempi etichettati nel prompt template (dimostrazioni).
- Al contrario, lo *zero-shot prompting* utilizza prompt che non contengono alcun esempio etichettato.
- Possiamo avere zero- one- n- shot prompting o, in generale, few-shot prompting

Few-shot prompting

Definition: This task is about writing a correct answer for the reading comprehension task. Based on the information provided in a given passage, you should identify the shortest continuous text span from the passage that serves as an answer to the given question. Avoid answers that are incorrect or provides incomplete justification for the question.

Passage: Beyoncé Giselle Knowles-Carter (born September 4, 1981) is an American singer, songwriter, record producer and actress. Born and raised in Houston, Texas, she performed in various singing and dancing competitions as a child, and rose to fame in the late 1990s as lead singer of R&B girl-group Destiny's Child. Managed by her father, Mathew Knowles, the group became one of the world's best-selling girl groups of all time. Their hiatus saw the release of Beyoncé's debut album, *Dangerously in Love* (2003), which established her as a solo artist worldwide, earned five Grammy Awards and featured the Billboard Hot 100 number-one singles "Crazy in Love" and "Baby Boy".

Examples:

Q: In what city and state did Beyoncé grow up?

A: Houston, Texas

Q: What areas did Beyoncé compete in when she was growing up?

A: singing and dancing

Q: When did Beyoncé release *Dangerously in Love*?

A: 2003

Q: When did Beyoncé start becoming popular?

A:



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Few-shot prompting

Quanti esempi?

- Non troppi: un numero casuale e piccolo.
- I maggiori guadagni di *performance* tendono a provenire dal primo esempio di *training* e diminuiscono per quelli successivi.
- Sono utili a dimostrare il *task* al LLM insieme al formato della sequenza.

Few-shot prompting

Come selezionare gli esempi?

- Automaticamente
 - Selezionate i top-T embeddings più simili nel training set all'embedding dell'esempio corrente.
- Programmaticamente
 - Utilizzare una misura di performance del task per selezionare gli esempi che la massimizzano.

In-Context Learning

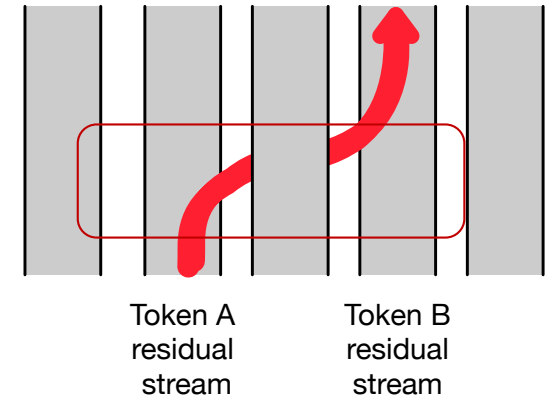
- In generale i prompt con o senza esempi, possono essere visti come una sorta di processo di *learning*
 - Più un modello *legge* il prompt, più tende a migliorare nella previsione dei tokens successivi.
 - L'informazione nel contesto aiuta a dare al modello maggiori capacità predittive.
- Questa capacità di apprendimento dei LLM generativi tramite prompt è chiamata *in-context learning*.

Induction head

- Diverse ipotesi sono state effettuate circa il funzionamento del in-context learning.
- In particolare, le *induction head* sono delle particolari zone della rete costituite da alcune delle attention head che assumono un ruolo particolare nell'aiutare il modello a fare previsioni, basandosi sul riconoscimento di pattern del tipo:

$$\{A, B, \dots, A\} \rightarrow B$$

Induction head



- Le *previous token head* sono attention head responsabili di prestare attenzione al token precedente e copiarlo nel residual stream del token a cui si presta attenzione.
- Le induction head eseguono quindi un'operazione di match-and-copy, cercando una corrispondenza tra una query derivata dal token corrente e una key derivata dall'output della token head precedente.

Model Alignment

- Il potere del in-context learning è limitato
 - Spesso, la tendenza autoregressiva del modello lo spinge ad ingorare il condizionamento datogli dal prompt.
 - I LLM pretrained generano testi che possono essere dannosi (falsi, unsafe, toxic, ...)
- Model alignment: metodi progettati per adattare meglio i LLM alle esigenze umane per generare testo utile e non dannoso.

Model Alignment

- Due strategie principali per il model alignment:
 - *Instruction tuning* (Supervised Fine Tuning - SFT): i modelli sono sottoposti a finetuning su un corpus di istruzioni e domande con le loro risposte corrispondenti.
 - *Preference alignment*: un modello separato viene addestrato per decidere quanto una risposta candidata si allinea alle preferenze umane; questo modello viene quindi utilizzato per il fine tuning del base model.
 - Il Reinforcement Learning from Human Feedback (RLHF), è la più nota tecnica di preference alignment

Instruction tuning

- Instruction tuning (instruction fine tuning, instruct, IT) è un metodo per rendere un LLM migliore nel seguire le istruzioni per una serie di tasks.
- Un LLM pretrained viene sottoposto a finetuning su un corpus di istruzioni e risposte utilizzando sempre la language modeling loss (CE Loss).
- Il modello risultante non solo apprende quei tasks, ma si impegna anche in una forma di meta-learning: migliora la sua capacità di seguire le istruzioni in generale.

Istruzioni come dati di training

- L'Instruction tuning è in realtà una procedura di training.
- Di conseguenza, devono essere sviluppati appositamente corpora di istruzioni etichettate.
- Esistono diversi corpora scritti da umani.
 - Un task che richiede molto tempo.

Chain-of-Thought Prompting

- Chain-of-Thought prompting (CoT) è una tecnica per migliorare la performance su task di reasoning difficili in cui i LLM tendono a fallire.
- Le persone risolvono compiti complessi suddividendoli in step.
- L'istruzione nel prompt incoraggerà i LLM a suddividerli allo stesso modo.

Chain-of-Thought Prompting

- Ciascun esempio nel few-shot prompting è arricchito con del testo che spiega alcuni passaggi di ragionamento, generando una dimostrazione.
- L'obiettivo è indurre il language model sia a produrre dei passaggi di ragionamento analoghi per il problema che si sta risolvendo sia a generare la risposta corretta come output.

Chain-of-Thought Prompting

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅



Università
degli Studi
di Palermo

dj
dipartimento
di ingegneria
unipa



Chain-of-Thought Prompting

